



UNIVERSITÀ DEGLI STUDI DI TRENTO

Dipartimento di Ingegneria e Scienza dell'Informazione

Appunti di Programmazione Funzionale

A cura di:

Mirco Negri

Versione Light Mode. Disponibile anche la [versione Dark Mode](#).

luglio 2026

Indice

1	Introduzione alla Programmazione Funzionale	4
1.1	Spiegazione teorica	4
2	Espressioni e tipi	4
2.1	Introduzione a ML	4
2.2	Espressioni e Comandi	5
2.3	Tipi di dato e astrazione	6
2.4	Espressioni, tipi e operatori in ML	6
2.5	Sistemi di tipi: Sicurezza e Controlli	8
3	Sistemi di tipi, variabili e funzioni	8
3.1	Spiegazione teorica	8
3.2	Implementazione del codice	10
3.2.1	Conversioni di tipo esplicite in ML	10
3.2.2	Ambiente e Shadowing	10
3.2.3	Manipolazione di Liste e Stringhe	10
3.2.4	Dichiarazione e uso di funzioni	11
4	Laboratorio ML 1	11
4.1	Correzione di Type Errors (Esercizi 1.3 - 1.5)	11
4.2	Istanze di Tipi Complessi (Esercizio 1.6 - 1.7)	13
5	Regole di scope	13
5.1	Funzioni e inferenza di tipi	13
5.2	Ambiente (Environment) e Blocchi	14
5.3	Scoping Statico vs Dinamico	14
6	Laboratorio ML 2	15
6.1	Esercizi base sulle Tuple	15
6.2	Esercizi base sulle Liste e sulle Stringhe	16
7	Astrazione del controllo e ricorsione	16
7.1	Spiegazione teorica	16
7.2	Implementazione del codice	17
7.2.1	Mutua Ricorsione in ML	17
8	Laboratorio ML 3	17
8.1	Esercizi base sulla Ricorsione Matematica	18
8.2	Esercizi base sulla Ricorsione e Liste	18
9	Pattern ed environment locale in ML	19
9.1	Spiegazione teorica	19
9.2	Implementazione del codice	20
9.2.1	Merge e uso di <code>as</code>	20
9.2.2	Utilizzo del <code>case</code> e <i>wildcard</i>	20
10	Laboratorio ML 4	21
10.1	Destrutturazione avanzata di liste e tuple	21

10.2 Insiemi Logici tramite Liste (Esercizi L4.6 - L4.8)	22
11 Input-output, eccezioni e funzioni polimorfe	22
11.1 Spiegazione teorica	22
11.2 Implementazione del codice	25
11.2.1 Esempio di I/O e gestione dei flussi di lettura da File	25
11.2.2 Eccezioni personalizzate e meccanismi di recupero (handle)	25
11.2.3 Polimorfismo puro vs Polimorfismo di uguaglianza	26
12 Laboratorio ML 5	26
12.1 Ottimizzazione dell'Ambiente Locale (let val)	26
12.2 Uso delle espressioni case	27
12.3 Esercizi di Input/Output e Stampa	27
13 Funzioni di ordine superiore e curried	28
13.1 Spiegazione teorica	28
13.2 Implementazione del codice	29
13.2.1 Funzioni di Ordine Superiore Classiche (Implementate a mano) . . .	29
13.2.2 Curryng e Applicazione Parziale	30
14 Laboratorio ML 6	30
14.1 Elaborazione di Flussi di Testo (I/O)	30
14.2 Esercizi dal Tutorato: I/O Sicuro ed Eccezioni	31
14.3 Combinare I/O e Ordine Superiore	31
15 Tipi di dati e composizione di funzioni	32
15.1 Spiegazione teorica	32
15.2 Implementazione del codice	33
15.2.1 Composizione e Primitive List	33
15.2.2 Costrutti Datatype e Union Types	33
16 Laboratorio ML 7	34
16.1 Curryng e Composizione Pratica (Esercizio L7.1 - L7.4)	34
16.2 Tipi di dato per gli Alberi Binari (Esercizi L7.5 - L7.10)	34
16.3 Esercizi e approfondimenti dal tutorato: Analisi di Struttura sugli Alberi .	35
17 Abstract Data Types in ML e Funtori	36
17.1 Spiegazione teorica	36
17.2 Implementazione del codice: Trasparenza vs Opacità	38
18 Laboratorio ML 8 - Alberi Binari di Ricerca	38
18.1 Creazione di un BST tramite Functor (MakeBST)	39
18.2 Esercizi e approfondimenti dal tutorato (Strutture e BST)	40
19 Lambda Calcolo	41
19.1 Spiegazione teorica	41
19.2 Implementazione e Riduzioni Logiche	42
19.2.1 Esempio 1: Applicazione di Base	43
19.2.2 Esempio 2: Valutazione di un Booleano con operatore	43
19.3 Esercizi e approfondimenti dal tutorato: Computazione in Forma Normale	43

19.3.1	Esercizio L11.12: Riduzione annidata	43
19.3.2	Esercizio L11.13: Riduzione con Catture Evitate	44
20	Programmazione Logica e Prolog	45
20.1	Spiegazione teorica	45
20.2	Implementazione del codice	47
20.2.1	Liste, Ricorsione e Backtracking (append e member)	47
20.2.2	Tic-Tac-Toe in Prolog (Tris Interattivo)	48
21	Domande Generali Teoria	49
22	Domande Lambda Calcolo	77
23	Domande Calcolo Indirizzo Array	83
24	Domande Tipi Funzioni ML	89
25	Domande Output Codici	100
26	Simulazioni d’Esame: Esercizi Pratici in ML	119
26.1	Esercizio 1: Estrazione dei percorsi su Alberi (Appello 15/07/2025)	119
26.2	Esercizio 2: Astrazione del Controllo (Il ciclo FOR)	120
26.3	Esercizio 3: Somma Filtrata per Parità (Prog 1)	121
26.4	Esercizio 4: Manipolazione di stringhe (Prefissi e Suffissi)	122
26.5	Esercizio 5: Addizione Binaria Nativa su Liste	122
26.6	Esercizio 6: Aggregazione su Strutture Ramificate (get_words)	123

1 Introduzione alla Programmazione Funzionale

1.1 Spiegazione teorica

La prima lezione del corso è introduttiva e mira a fornire una panoramica sugli argomenti trattati, le modalità di svolgimento e un inquadramento storico dei linguaggi di programmazione.

Il paradigma funzionale si distingue da quello imperativo e orientato agli oggetti per il suo approccio matematico alla computazione, basato sulla valutazione di espressioni e sull'uso di funzioni pure.

I punti chiave affrontati in questa introduzione includono:

- **Obiettivi del corso:** Comprendere i principi base della programmazione funzionale, studiare il sistema dei tipi, le funzioni di ordine superiore, l'astrazione dei dati e il polimorfismo. Verranno inoltre introdotti il Lambda Calcolo, la programmazione logica (Prolog) e accenni a Scala.
- **Storia dei linguaggi funzionali e logici:**
 - **Anni '50 - '70:** Nascita del LISP (lista processing), capostipite dei linguaggi funzionali (untyped), seguito dalle sue evoluzioni come Scheme e Racket.
 - **Anni '70 - '80:** Sviluppo dei theorem prover (LCF) da cui deriva la famiglia ML (Meta Language), caratterizzata da *call-by-value*, tipizzazione forte e polimorfismo. Nascono Caml e Miranda.
 - **Anni '90 - oggi:** Consolidamento di Standard ML, OCaml e nascita di Haskell (linguaggio funzionale puro *lazy*). Più recentemente, la programmazione funzionale ha influenzato linguaggi moderni come Scala e F#.
 - Sviluppo di strumenti basati sui tipi dipendenti (come Coq) nati negli anni '80.

Trattandosi di una lezione introduttiva sul contesto e sulla storia del paradigma, non sono stati affrontati esercizi pratici o costrutti sintattici del linguaggio ML.

2 Espressioni e tipi

2.1 Introduzione a ML

ML (Meta Language) è un linguaggio di programmazione general-purpose, nato originariamente come metalinguaggio per sistemi di verifica formale (come il theorem prover LCF negli anni '70). Esistono diversi dialetti derivati, tra cui Standard ML, OCaml, F#, ed elementi che hanno influenzato linguaggi come Scala e Haskell.

Perché studiare ML? Studiare la programmazione funzionale impone un cambio di paradigma: i linguaggi mainstream si concentrano sulla gestione degli "stati", mentre la programmazione funzionale si concentra sui **valori**. ML è un linguaggio pratico e compatto le cui idee possono rendere programmatori migliori in qualsiasi linguaggio.

Caratteristiche principali della Programmazione Funzionale in ML:

1. **Linguaggio funzionale:** Il modello di calcolo di base si fonda sulla definizione e sull'applicazione di funzioni. Le funzioni sono trattate come valori (cittadini di prima classe) e possono essere passate come parametri ad altre funzioni, supportando nativamente le *funzioni di ordine superiore* (Higher-order functions).
2. **Ricorsione:** Fortemente incoraggiata rispetto ai cicli `while` (costrutti iterativi sono disponibili, ma secondari).
3. **Programmazione basata su regole:** Costrutti come `if-then-else` sono spesso implementati o sostituiti dal *pattern matching*.
4. **Nessun side effect (effetti collaterali):** La computazione procede per valutazione di espressioni, non tramite l'assegnazione di valori a variabili che mutano lo stato. Se in C `a = b + c` modifica lo stato di `a`, in ML `b + c` crea un nuovo elemento. Gli effetti collaterali (es. stampare a video) sono permessi ma non sono il mezzo principale di calcolo.
5. **Strong typing (Tipizzazione forte):** Tutti i valori e le variabili hanno tipi determinabili a "tempo di compilazione" (*compile-time*). Rileva errori semantici preziosi; inoltre, non è solitamente necessaria la dichiarazione esplicita del tipo grazie alla *Type Inference*.
6. **Polimorfismo:** Una funzione può accettare argomenti di tipi differenti (es. una funzione di ordinamento generica).
7. **Abstract Data Types (Strutture):** Sistema di tipi elegante con la capacità di costruire nuovi tipi complessi.

Eseguire ML: L'interprete interattivo PolyML si avvia da linea di comando digitando `poly` (o `rlwrap poly` in Linux per avere la cronologia dei comandi). All'interno dell'ambiente interattivo (indicato dal prompt `>`), si possono valutare le espressioni chiudendole con il punto e virgola `;`. Per caricare un file esterno si usa `use "foo";`. Per uscire CTRL+D, per interrompere CTRL+C.

2.2 Espressioni e Comandi

Vi è una netta separazione tra espressioni e comandi:

- **Espressioni:** Entità sintattiche la cui valutazione restituisce un valore o non termina (espressione indefinita). Sono composte da costanti, variabili o operatori applicati a ulteriori espressioni. Notazioni utilizzabili: infissa (`a + b`), prefissa o polacca (`+ a b`), suffissa (`a b +`).
- **Comandi:** Entità sintattiche la cui valutazione non restituisce necessariamente un valore, ma provoca un effetto collaterale (modifica dello stato). Tipici dei linguaggi imperativi, sono del tutto assenti nei paradigmi puramente funzionali o logici.

Strategie di valutazione delle espressioni:

- **Eager evaluation (Valutazione ansiosa):** Si valutano prima tutti gli operandi, poi si applica l'operatore ai risultati.
- **Lazy evaluation (Valutazione pigra):** Gli operandi vengono valutati solo se e quando il loro valore è effettivamente necessario per il calcolo. Questo approccio previene errori in espressioni come `a == 0 ? b : b/a` (se `a` è 0, valutare ansiosamente `b/a` causerebbe una divisione per zero). Viene usata per operatori booleani (short-circuit evaluation), dove il risultato finale può essere noto prima di valutare tutti gli operandi.

In ML si utilizza principalmente la **eager evaluation**, tranne che per alcuni specifici costrutti come `andalso`, `orelse` e `if-then-else` che adottano una valutazione *lazy*.

2.3 Tipi di dato e astrazione

Un **tipo di dato** è una collezione di valori (omogenei) dotata di un insieme di operatori applicabili. I tipi servono a tre scopi principali:

1. **Livello concettuale (Design):** Organizzare l'informazione e documentare il codice (es. distinguere un `prezzo` da un `numero di stanza`).
2. **Livello di programma (Correctness):** Identificare ed evitare errori. I compilatori eseguono il type-checking: assegnare una stringa a un intero (`3 + "pippo"`) solleverà un errore semantico.
3. **Livello implementativo:** Permettere al compilatore ottimizzazioni. Conoscere il tipo permette di sapere quanta memoria allocare (un booleano occupa meno di un double) e calcolare gli offset di memoria per l'accesso ai campi di una struct.

2.4 Espressioni, tipi e operatori in ML

In ML, le espressioni possono essere valori o funzioni applicate a valori. Ogni espressione in PolyML termina con `;`. Esempio:

```
1 + 2 * 3;  
val it = 7: int
```

In questo output: `val` indica il valore dell'espressione, `it` è il nome automatico associato al risultato, e `int` è il tipo dedotto automaticamente.

Tipi Base in ML:

- **int**: Interi positivi e negativi. L'operatore unario per il negativo è la tilde `~` (es. `~3`). Supporta esadecimali (es. `0x124`).
- **real**: Numeri reali. Richiedono il punto decimale o la notazione esponenziale `E/e` (es. `~123.0`, `3.14e12`).
- **bool**: `true` o `false` (rigorosamente minuscoli, ML è case-sensitive).
- **char**: Singolo carattere indicato dal prefisso `#` (es. `#"a"`).
- **string**: Doppi apici (es. `"foo"`). Caratteri speciali supportati: `\n` (newline), `\t` (tab), `\\`, `\"`.
- **unit**: Possiede un unico valore `()`, utilizzato per indicare le espressioni che non restituiscono un valore utile (analogo a `void`).

Operatori:

- **Aritmetici**: `+`, `-`, `*` per `int` e `real`. La divisione per reali è `/`, mentre per gli interi si usano `div` (divisione troncata all'intero inferiore) e `mod` (resto).
- **Forma Infissa/Prefissa**: In ML, un operatore come `+` è di fatto una funzione prefissa che riceve in input una tupla numerica `+(1,2)`, ma consente la notazione infissa per comodità.
- **Stringhe**: La concatenazione avviene tramite l'operatore `^` (es. `"house" ^ "cat" → "housecat"`).
- **Confronto**: `=`, `<`, `>`, `<=`, `>=`, `<>` (diverso). Valgono per interi, caratteri e stringhe (ordine lessicografico). **Attenzione**: in ML non è permesso testare l'uguaglianza (o la disuguaglianza `<>`) tra due valori `real`, per prevenire problemi di approssimazione *floating-point*.
- **Logici**: `not`, `andalso` (AND lazy), `orelse` (OR lazy). `andalso` ha precedenza su `orelse`.

Costrutto If-then-else:

```
if then else ;
```

Poiché in ML tutto è un'espressione, l'utilizzo di `if` richiede **obbligatoriamente** la presenza del ramo `else`. Inoltre, per preservare la *type safety*, e devono avere lo stesso tipo ben definito. Un'espressione come `if 5<6 then 5 else 6.0`; solleva un *Type mismatch error* statico in compilazione.

2.5 Sistemi di tipi: Sicurezza e Controlli

I linguaggi di programmazione adottano strategie diverse per il *Type Checking*:

- **Static Type Checking:** Effettuato a tempo di compilazione (es. C, Java, Haskell, ML) prima dell'esecuzione. Vantaggi: maggiore efficienza a runtime e blocco preventivo di errori. Svantaggi: design del linguaggio più complesso e regole spesso molto rigide.
- **Dynamic Type Checking:** Eseguito durante l'esecuzione del codice (es. Python, JavaScript, Lisp). Vantaggi: massima flessibilità, non servono dichiarazioni di tipo. Svantaggi: gli errori di tipo vengono identificati solo a runtime, e causano perdita di efficienza dovuta ai continui controlli dinamici.

Inoltre, i linguaggi si dividono in:

- **Strongly typed (Tipizzazione forte):** Molto severi sui tipi. Il tipo di un valore non cambia in modo inaspettato. Le conversioni di tipo implicite (coercion) sono vietate (es. ML).
- **Weakly typed (Tipizzazione debole):** Molto flessibili, permettono la conversione implicita tra tipi differenti a runtime (es. JavaScript: `4 + "2" = "42"`, PHP, C/C++).

ML, a garanzia della sua **Type Safety**, è un linguaggio **fortemente e staticamente tipizzato** (statically and strongly typed). Tenta di risolvere l'inconveniente della verbosità grazie al potente meccanismo della *Type Inference* (in cui il compilatore deduce il tipo in automatico), rifiutandosi categoricamente di effettuare coercion nascoste (come `1 + 2.0` che genererà un errore).

3 Sistemi di tipi, variabili e funzioni

3.1 Spiegazione teorica

Il sistema dei tipi di un linguaggio è definito dai suoi tipi predefiniti, dai meccanismi per crearne di nuovi, dalle regole per determinare se il controllo è statico o dinamico e dalle regole di equivalenza, compatibilità e inferenza.

Static e Dynamic Type Checking:

- **Controllo Statico:** Avviene a tempo di compilazione (es. ML, Java, C). Previene l'esecuzione di codice con errori di tipo ed è molto efficiente a runtime, sebbene renda la progettazione e la scrittura del codice più rigida.
- **Controllo Dinamico:** Avviene durante l'esecuzione (es. Python, JavaScript). Garantisce grande flessibilità, ma gli errori vengono scoperti solo a runtime, impattando sulle prestazioni.

I linguaggi si dividono inoltre in **fortemente tipizzati** (strongly typed), dove il tipo di un valore non cambia in modo inaspettato (niente conversioni implicite, come in ML), e **debolmente tipizzati** (weakly typed), dove le conversioni implicite (coercion) sono permesse (es. `4 + "2" = "42"` in JavaScript). ML assicura un alto livello di *Type Safety*, ovvero previene la presenza di errori di tipo inosservati a runtime.

Equivalenza e Compatibilità: Due tipi possono essere considerati equivalenti per **nome** (se hanno letteralmente lo stesso nome) o per **struttura** (se, analizzando le definizioni, la struttura di base è identica). ML utilizza principalmente l'equivalenza strutturale (fatta eccezione per i tipi definiti con **datatype**).

In ML **non esistono conversioni automatiche** (coercion). Se si valuta `1 + 2.0`, il compilatore non converte automaticamente l'intero in reale, ma restituisce un errore di tipo statico. È necessario eseguire una conversione esplicita tramite apposite funzioni di sistema.

Nomi, Variabili e Ambiente (Environment): L'**Environment** è l'insieme delle associazioni tra identificatori (nomi) e valori. In ML, a differenza dei linguaggi imperativi come il C, **le variabili non sono modificabili**. Una variabile è semplicemente un identificatore legato a un valore matematico. Tramite il costrutto **val**, non si assegna un nuovo valore a una locazione di memoria, ma si crea un nuovo legame (binding) nell'ambiente. Se si riutilizza lo stesso nome, la nuova dichiarazione oscura la precedente (*shadowing*), ma il valore originale non viene sovrascritto.

Tipi Complessi: Tuple e Liste:

- **Tuple:** Aggregati di dimensione fissa che possono contenere tipi eterogenei. Esempio: `(4, 5.0, "sei")` è di tipo `int * real * string`. Si accede all'*n*-esimo elemento tramite l'operatore `#n`.
- **Liste:** Aggregati di dimensione variabile, ma rigorosamente omogenei (tutti gli elementi devono avere lo stesso tipo). Esempio: `[1, 2, 3]` è di tipo `int list`.

Funzioni come valori: In ML le funzioni sono trattate come "cittadini di prima classe", ovvero come normali valori. Possono essere anonime tramite il costrutto **fn** (che riflette il lambda calcolo), o associate a un nome con il costrutto **fun**. Una particolarità di ML è che **tutte le funzioni accettano esattamente un solo parametro**. Per simulare più parametri, si passa semplicemente una singola tupla contenente i vari argomenti.

3.2 Implementazione del codice

3.2.1 Conversioni di tipo esplicite in ML

```
> real(4);          (* Da intero a reale *)
val it = 4.0: real

> floor(3.5);       (* Arrotondamento per difetto *)
val it = 3: int

> ceil(3.5);        (* Arrotondamento per eccesso *)
val it = 4: int

> round(3.5);       (* Arrotondamento al pari più vicino *)
val it = 4: int

> trunc(3.5);       (* Troncamento *)
val it = 3: int

> ord("#a");        (* Da char ad ASCII *)
val it = 97: int

> chr(97);          (* Da ASCII a char *)
val it = #a: char

> str("#a");        (* Da char a string *)
val it = "a": string
```

3.2.2 Ambiente e Shadowing

```
> val pi = 3.14159;
val pi = 3.14159: real

> val pi = 3;
val pi = 3: int
(* Crea un nuovo 'pi' di tipo intero che nasconde il precedente,
   ma non lo sovrascrive fisicamente *)
```

3.2.3 Manipolazione di Liste e Stringhe

```
> val L = [2, 3, 4];
> hd(L);           (* Head: estrae il primo elemento *)
val it = 2 : int

> tl(L);           (* Tail: estrae il resto della lista *)
val it = [3, 4] : int list

> [1, 2] @ [3, 4];  (* Concatenazione di liste, stesso tipo! *)
val it = [1, 2, 3, 4] : int list
```

```

> 1 :: [2, 3];      (* Cons: aggiunge l'elemento in testa alla lista *)
val it = [1, 2, 3] : int list

> explode("abcd"); (* Da stringa a lista di caratteri *)
val it = ["a", "b", "c", "d"] : char list

> implode(["a", "b"]); (* Da lista di caratteri a stringa *)
val it = "ab" : string

```

3.2.4 Dichiarazione e uso di funzioni

```

(* Sintassi della funzione anonima associata a una variabile *)
> val increment = fn n => n + 1;
val increment = fn: int -> int

(* Zucchero sintattico (più comune) *)
> fun increment n = n + 1;
val increment = fn: int -> int

(* Funzione a finti "parametri multipli" (prende in realtà una tupla) *)
> fun max3(a:real, b, c) =
    if a > b then
        if a > c then a else c
    else
        if b > c then b else c;
val max3 = fn: real * real * real -> real

```

4 Laboratorio ML 1

Nel primo laboratorio si mette in pratica la sintassi base affrontata nelle lezioni teoriche, concentrandosi sulla correzione dei "Type Mismatch" e sulla familiarizzazione con i tipi complessi (*list*, *tuple* e relative combinazioni).

4.1 Correzione di Type Errors (Esercizi 1.3 - 1.5)

ML è un linguaggio in cui la correttezza del tipo (*Type Safety*) viene imposta in modo rigoroso. Analizziamo come correggere espressioni sintatticamente o logicamente invalide.

```

(* ERRORE: ceil accetta solo real, non int *)
ceil(4);
(* CORREZIONE *)
ceil(4.0);

(* ERRORE: i rami dell'if devono restituire esattamente lo stesso tipo *)
if true then 5 + 6 else 7.0;
(* CORREZIONE (convertito l'intero in reale) *)
if true then real(5 + 6) else 7.0;

(* ERRORE: la condizione dell'if deve essere strettamente di tipo bool *)
if 0 then 1 else 2;
(* CORREZIONE *)
if false then 1 else 2;

(* ERRORE: ord si applica ai caratteri (prefisso #), non alle stringhe *)
ord("a");
(* CORREZIONE *)
ord("#a");

(* ERRORE: explode prende in input una singola stringa, non una lista *)
explode ["bar"];
(* CORREZIONE *)
explode("bar");

(* ERRORE: implode accetta una lista di char, non una tupla *)
implode (#"a", #"b");
(* CORREZIONE (sostituisco le parentesi tonde con le quadre) *)
implode (["a", "b"]);

(* ERRORE: :: (cons) unisce UN elemento a una LISTA.
    Qui a sinistra c'è una lista e a destra un'altra lista *)
["r"]::["a","t"];
(* CORREZIONE: (rimuovo le parentesi a sinistra) *)
"r"::["a","t"];

```

4.2 Istanze di Tipi Complessi (Esercizio 1.6 - 1.7)

Si richiede di creare valori legali per le seguenti firme di tipo, dimostrando di saper annidare correttamente tuple (parentesi tonde) e liste (parentesi quadre).

```
(* TIPO: int list list list *)
(* Una lista che contiene liste di liste di interi *)
val es1 = [[[1, 2]], [[3]]];

(* TIPO: (int * char) list *)
(* Una lista di tuple formate da intero e carattere *)
val es2 = [(1, #"a"), (2, #"b")];

(* TIPO: string list * (int * (real * string)) *)
(* Una macro-tupla contenente una lista di stringhe e un'altra tupla *)
val es3 = (["ciao", "mondo"], (42, (3.14, "pi")));

(* TIPO: ((int * int) * (bool list) * real) * (real * string) *)
val es4 = (((1, 2), [true, false], 5.0), (3.14, "test"));
```

5 Regole di scope

5.1 Funzioni e inferenza di tipi

In ML, le funzioni sono considerate valori (cittadini di prima classe) a tutti gli effetti. Sono rappresentate tramite espressioni parametrizzate.

- **Sintassi base (fn):** La definizione di una funzione anonima si basa sulla sintassi `fn <parametro> => <espressione>`, che corrisponde concettualmente all'astrazione λ nel lambda calcolo. Esempio: `fn n => n + 1`.
- **Dichiarazioni con nome:** È possibile associare una funzione a un identificatore usando la parola chiave `val` (es. `val increment = fn n => n + 1`), ma ML offre anche un costrutto semplificato (zucchero sintattico) estremamente comune: `fun increment n = n + 1`.
- **Singolo parametro e Tuple:** In ML, **tutte le funzioni hanno esattamente un solo parametro**. Tuttavia, se abbiamo bisogno di passare molteplici argomenti logici, il parametro può essere strutturato come una tupla. Ad esempio, `fun max3(a:real, b, c) = ...` accetta in input un'unica tupla contenente tre elementi di tipo `real`.

Type Inference (Inferenza dei tipi): Il compilatore di ML deduce automaticamente i tipi delle variabili basandosi sul contesto delle espressioni:

- In un'espressione aritmetica come $(a + b) * 2.0$, sapendo che 2.0 è reale e l'operatore $*$ impone operandi compatibili, il risultato di $a+b$ e, di conseguenza, le variabili a e b vengono forzate e dedotte come tipo **real**.
- In presenza di costrutti condizionali, le espressioni associate ai rami **then** ed **else** e all'intera espressione condizionale stessa devono rigorosamente possedere lo stesso tipo.
- Qualora non vi siano operatori che forzino un tipo specifico e non vi sia alcun modo per determinarne il tipo, il compilatore assegna un **tipo generico** (indicato per esempio con `'a` e `'b`).

5.2 Ambiente (Environment) e Blocchi

L'**Environment** (ambiente) è la collezione delle associazioni tra identificatori (nomi) e oggetti denotabili che si viene a creare a runtime in uno specifico momento del programma. Una *dichiarazione* è il meccanismo che crea tale associazione all'interno dell'ambiente. L'ambiente può essere logicamente suddiviso in:

- **Ambiente locale:** Variabili dichiarate internamente e parametri formali creati all'ingresso di un blocco.
- **Ambiente non-locale:** Associazioni ereditate da blocchi gerarchicamente esterni a quello corrente.
- **Ambiente globale:** Costituisce quella porzione dell'ambiente non-locale che contiene associazioni e dichiarazioni visibili in tutti i blocchi.

I **blocchi** (in ML definiti dal costrutto `let ... in ... end`) gestiscono l'organizzazione dell'ambiente. Una dichiarazione locale in un blocco è visibile nel blocco stesso e in tutti i blocchi annidati all'interno, a patto che in questi ultimi non vi sia un'ulteriore dichiarazione con il medesimo nome che ne occulta (shadowing) la precedente.

5.3 Scoping Statico vs Dinamico

Come viene risolto il riferimento a una variabile "non locale"? La letteratura dei linguaggi di programmazione definisce due approcci fondamentali:

- **Scoping Statico (o Lexical):** Il riferimento viene risolto nel blocco che **include testualmente (sintatticamente)** la chiamata. Si basa esclusivamente sulla struttura del codice scritto ed è indipendente dai nomi locali del blocco chiamante. I vantaggi dello scoping statico includono una maggiore facilità di lettura da parte del programmatore, migliore efficienza a runtime, e la possibilità per il compilatore di effettuare stringenti controlli di correttezza e ottimizzazioni. È adottato in linguaggi come C, Java, Pascal e dallo stesso ML.

- **Scoping Dinamico:** Il riferimento a un nome non-locale viene risolto all'interno del blocco che è **stato attivato per ultimo** durante il flusso di esecuzione (a runtime) e non ancora terminato. Tale approccio consente di alterare il comportamento di una funzione semplicemente modificando l'ambiente del chiamante, ma genera architetture fragili in cui i programmi diventano ostici da comprendere e meno efficienti.

6 Laboratorio ML 2

Nel secondo laboratorio si prendono i primi passi pratici scrivendo funzioni pure per elaborare primitive di ML e strutture complesse quali Tuple e Liste, verificandone il corretto type checking.

6.1 Esercizi base sulle Tuple

Esercizio L2.2 - Minimo di una tupla: Scrivere una funzione `min3` che restituisca l'elemento più piccolo all'interno di una tupla composta da tre numeri interi.

```
fun min3 (a: int, b: int, c: int) =
  if a < b andalso a < c then a
  else if b < c then b
  else c;
```

Esercizio L2.4 - Inversione di Tupla: Scrivere una funzione `reverse` che prenda in ingresso una tupla di 3 elementi invertendone gli oggetti (la funzione deve poter accettare Tuple eterogenee, es. `reverse(1.0, 2, "a")`).

```
fun reverse (a, b, c) = (c, b, a);
```

Esercizi L2.7 e L2.8 - Identificazione ed Ordinamento: Scrivere una funzione `min_max_pair` che data una tupla di 3 interi restituisca in uscita una nuova tupla (coppia) limitata al valore più piccolo e al valore più grande estratti. Successivamente, implementare una funzione `sort` in grado di restituire una lista ordinata degli elementi.

```
fun min_max_pair (a:int, b, c) =
  let
    val minimo = if a<b andalso a<c then a else if b<c then b else c
    val massimo = if a>b andalso a>c then a else if b>c then b else c
  in
    (minimo, massimo)
  end;
```

```
fun sort (a:int, b, c) =
  let
    val (minimo, massimo) = min_max_pair(a, b, c)
    val mediano = if (a <> minimo andalso a <> massimo) then a
                  else if (b <> minimo andalso b <> massimo) then b
                  else c
  in
    [minimo, mediano, massimo]
  end;
```


6.2 Esercizi base sulle Liste e sulle Stringhe

Esercizio L2.3 - Terzo elemento: Estrarre il terzo elemento da una lista di dati. L'esercizio prevede l'uso in sequenza delle primitive `hd` (head) e `tl` (tail) senza la necessità, in questa fase, di far operare correttamente la funzione su liste inferiori alla lunghezza prevista.

```
fun third L = hd(tl(tl(L)));
```

Esercizio L2.6 - Cycle: Scrivere una funzione `cycle` che prende una lista originaria `[a1, ..., an]` ed esegue un "giro", producendo come risultato la dislocazione `[a2, ..., an, a1]` (l'esercizio assume liste non vuote).

```
fun cycle L = tl(L) @ [hd(L)];
```

Esercizio L2.10 - Rimuovere il secondo elemento: Scrivere la funzione `rem` in grado di accettare in ingresso una lista e ritornare la struttura inalterata privata del secondo elemento (ad esempio, la lista `[1,2,3,4]` si traduce in `[1,3,4]`).

```
fun rem L = hd(L) :: tl(tl(L));
```

7 Astrazione del controllo e ricorsione

7.1 Spiegazione teorica

L'astrazione del controllo riguarda i meccanismi che i linguaggi di programmazione offrono per gestire il flusso di esecuzione (come blocchi condizionali, cicli e chiamate a sottoprogrammi). In ML, essendo un linguaggio funzionale, il controllo del flusso si basa pesantemente sulla **valutazione delle espressioni** e sulla **ricorsione**, a differenza dei linguaggi imperativi che si basano su costrutti iterativi (come il `while` o il `for`).

Metodi di passaggio dei parametri: Quando una funzione viene invocata, l'ambiente locale deve essere popolato associando i parametri formali ai parametri attuali. Esistono storicamente diverse strategie:

- **Call by value (Passaggio per valore):** I parametri attuali vengono valutati *prima* che la funzione venga chiamata. Il risultato viene copiato nel parametro formale. È un meccanismo sicuro e senza side-effect diretti sulle variabili esterne, ed è la strategia di default adottata da ML (eager evaluation).
- **Call by reference (Passaggio per riferimento):** Viene passato l'indirizzo di memoria del parametro attuale. Modificare il parametro formale nella funzione modifica la variabile originale (side-effect). Non è il meccanismo predefinito nei linguaggi funzionali puri.
- **Call by name / Call by need:** Il parametro non viene valutato finché non è strettamente necessario all'interno del corpo della funzione (lazy evaluation). Se non viene mai usato, non viene mai valutato, risparmiando computazione e prevenendo errori (come la divisione per zero).

Ricorsione: In assenza di cicli iterativi standard, la ricorsione è l'unico modo per compiere iterazioni in un linguaggio puramente funzionale. Il programmatore definisce un caso base (condizione di terminazione) e un passo ricorsivo (in cui la funzione invoca se stessa su un input ridotto). Un concetto fondamentale legato all'ottimizzazione è la **Tail Recursion** (Ricorsione in coda): si ha quando la chiamata ricorsiva è l'ultimissima operazione eseguita nel corpo della funzione. Questo permette al compilatore di riutilizzare lo stesso record di attivazione (Stack Frame) senza saturare la memoria (evitando lo Stack Overflow).

Mutua Ricorsione: Si verifica quando due o più funzioni si chiamano a vicenda. Nei linguaggi con regole di scope rigide (dove un nome deve essere definito prima di essere usato), definire due funzioni A e B che si chiamano a vicenda solleverebbe un errore: se dichiaro prima A, essa non conoscerà B, e viceversa. ML risolve elegantemente questo problema introducendo la parola chiave **and**, che permette di dichiarare e valutare le due funzioni contemporaneamente nello stesso blocco.

7.2 Implementazione del codice

7.2.1 Mutua Ricorsione in ML

Vogliamo scrivere due funzioni: **take**, che estrae gli elementi in posizione dispari da una lista, e **skip**, che salta gli elementi in posizione pari (concettualmente le due operazioni si alternano).

```
(* L'uso di 'and' permette la mutua ricorsione.
```

```
Il compilatore valuta le firme di entrambe contemporaneamente. *)
```

```
fun take (L) =  
  if L = nil then nil  
  else hd(L) :: skip(tl(L))  
and skip (L) =  
  if L = nil then nil  
  else take(tl(L));
```

```
(* Esempio di esecuzione *)
```

```
> take ([1, 2, 3, 4, 5]);  
val it = [1, 3, 5] : int list
```

8 Laboratorio ML 3

Nel terzo laboratorio si consolida l'utilizzo della ricorsione semplice per risolvere problemi matematici e per manipolare liste, costruendo algoritmi iterativi attraverso chiamate a funzione ricorsive in stile dichiarativo.

8.1 Esercizi base sulla Ricorsione Matematica

Esercizio 3.1 - Fattoriale: Scrivere una funzione `fact` che calcola il fattoriale di un numero n (con $n \geq 1$). L'esercizio non richiede, in questa fase primaria, di gestire in modo sicuro i numeri negativi.

```
fun fact(n) =  
  if n = 1 then 1  
  else n * fact(n - 1);
```

```
(* Utilizzo: *)
```

```
> fact 10;
```

```
val it = 3628800 : int
```

```
(* Nota: In ML un "fact 100" genera una Exception-Overflow  
  in quanto supera il valore massimo immagazzinabile nel tipo int.  
  Per ovviare a questo limite, si può utilizzare la struttura IntInf  
  per operare con interi a precisione arbitraria. *)
```

Esercizio 3.11 - Potenza (Power): Scrivere una funzione `power` che prenda in ingresso una tupla formata da un numero intero x e un esponente i (assunto non negativo) e ne calcoli la potenza x^i . Notare l'utilizzo del pattern matching semplificato nella firma della funzione.

```
fun power (x, 0) = 1  
  | power (x, i) = x * power (x, i - 1);
```

```
(* Utilizzo: *)
```

```
> power (4, 3);
```

```
val it = 64 : int
```

8.2 Esercizi base sulla Ricorsione e Liste

Esercizio 3.10 - Duplicazione elementi (duplicatep): Scrivere una funzione che, data una lista di elementi di tipo qualsiasi, restituisca una nuova lista in cui ogni elemento originale è ripetuto consecutivamente due volte (es. da $[1,2]$ a $[1,1,2,2]$). Questo esercizio introduce un'applicazione molto comune del pattern matching su liste.

```
(* L'uso dei pattern semplifica la scrittura rispetto all'uso degli  
  if-else e di hd() / tl() *)  
fun duplicatep (nil) = nil  
  | duplicatep (x::xs) = x :: x :: duplicatep(xs);
```

```
(* Utilizzo: *)
```

```
> duplicatep [1, 2, 3, 4];
```

```
val it = [1, 1, 2, 2, 3, 3, 4, 4] : int list
```

Esercizio 3.12 - Massimo di una Lista (maxList): Scrivere una funzione per trovare l'elemento reale più grande all'interno di una lista (assunta non vuota). Questa funzione utilizza l'istruzione `let ... in ... end` per salvare il risultato della chiamata ricorsiva, evitandone il ricalcolo multiplo e ottimizzando le prestazioni (concetto propedeutico per strutture più avanzate).

```
fun maxList ([x:real]) = x
  | maxList (x::xs) =
    let
      val m = maxList(xs)
    in
      if x > m then x else m
    end;
```

9 Pattern ed environment locale in ML

9.1 Spiegazione teorica

In ML, oltre alla dichiarazione di funzioni tramite costrutti condizionali classici (`if-then-else`), è possibile (e fortemente consigliato) utilizzare il **Pattern Matching**. Questo meccanismo ricorda i costrutti `switch-case` dei linguaggi procedurali, ma è molto più potente perché permette di "destrutturare" i dati e assegnare variabili al volo.

- **Sintassi base dei pattern:** La definizione di una funzione procede elencando una sequenza di casi. Il primo pattern che corrisponde (esegue il "match") con l'argomento fornito determina quale espressione verrà valutata.

```
fun f (pattern_1) = espressione_1
  | f (pattern_2) = espressione_2
  | f (pattern_n) = espressione_n;
```

- **Esaustività:** La lista delle alternative dovrebbe sempre coprire tutti i casi possibili. Se i pattern non sono esaustivi, il compilatore di ML genera un *warning*. Se a runtime viene passato un argomento che non fa match con alcun pattern dichiarato, viene sollevata un'eccezione critica (`Exception- Match raised`).
- **Pattern validi e non validi:**
 - *Consentiti:* Costanti (es. `nil`, `0`, stringhe), costruttori di liste come l'operatore `cons` (es. `x::xs`, `x::y::xs`), tuple (es. `(x, y, z)`) e il carattere jolly o *wildcard* `_` (usato per i valori "anonimi" che non ci interessa memorizzare né usare).
 - *Non consentiti:* Operatori aritmetici (non si può scrivere `fun square(x+1) = ...`), concatenazione di liste tramite l'operatore `@`, numeri reali (`real`) e l'uso multiplo della stessa variabile in un singolo pattern (es. `fun uguali(n, n) = true` è illegale, la variabile `n` è già associata). L'identità logica va testata nel corpo con `if`.

Il costrutto `as`: Spesso è utile destrutturare un parametro (ad esempio separando testa e coda di una lista) ma al tempo stesso mantenere un riferimento "globale" all'intero oggetto originale per non doverlo ricostruire. La keyword `as` risolve questa necessità sintattica: scrivendo `L as x::xs`, la variabile `L` rappresenta la lista intera, mentre `x` e `xs` ne catturano rispettivamente testa e coda.

Costrutti `case` e `fn`: La sintassi del pattern matching si applica in modo identico anche alle funzioni anonime (`fn`) e al costrutto interno `case`:

```
case x of
  pattern_1 => espressione_1
| pattern_2 => espressione_2;
```

La differenza architetturale è che il `case` permette di effettuare pattern matching sul risultato momentaneo di un'espressione qualsiasi nel bel mezzo di una funzione, e non soltanto sui parametri di ingresso della stessa.

Ambiente locale (`let ... in ... end`): Per definire variabili temporanee, utilissime per memorizzare sotto-espressioni ricorrenti o decomporre i complessi risultati in uscita da una funzione, ML offre il costrutto `let`.

```
let
  val x = calcolo_pesante()
  val (a, b) = scorpora_tupla()
in
  a + b + x
end
```

I "binding" (assegnazioni) creati tra `let` e `in` arricchiscono l'ambiente locale ma restano visibili esclusivamente all'interno del blocco di codice delimitato tra `in` ed `end`.

9.2 Implementazione del codice

9.2.1 Merge e uso di `as`

Un esempio didattico eccellente in cui `as` semplifica la stesura è l'unione ordinata di due liste.

```
fun merge (nil, M) = M
  | merge (L, nil) = L
  | merge (L as x::xs, M as y::ys) =
    if x < y then x :: merge(xs, M)
    else y :: merge(L, ys);
```

Senza la keyword `as`, nel ramo `else` avremmo dovuto ricostruire a mano e sprecare risorse per ricreare la prima lista scrivendo `merge(x::xs, ys)`.

9.2.2 Utilizzo del `case` e *wildcard*

```
val is_one = fn x =>
  case x of
    1 => "one"
  | _ => "anything else";
```

10 Laboratorio ML 4

Il quarto laboratorio e i relativi esercizi di tutorato impongono l'abbandono delle funzioni primitive `hd` e `tl` viste all'inizio del corso, in favore dell'uso sistematico e ricorsivo del pattern matching. Le tecniche apprese qui costituiscono la base per affrontare le prove d'esame.

10.1 Destrutturazione avanzata di liste e tuple

Esercizio L4.1 - Flip alternato: Scrivere una funzione `flip` che inverte a due a due gli elementi di una lista e, se dispari, lascia l'ultimo elemento inalterato alla fine.

```
fun flip ([]) = []
  | flip ([x]) = [x]
  | flip (x::y::zs) = y :: x :: flip(zs);
```

Esercizio L4.4 - Flip di tuple in lista: Applicazione combinata di pattern matching su liste e tuple che sfrutta il costrutto `as` per risparmiare allocazioni di memoria.

```
fun flip (nil) = nil
  | flip ((x as (a:int, b)) :: xs) =
    if a < b then x :: flip(xs)
    else (b, a) :: flip(xs);
```

Nota per l'esame: Se la tupla (a,b) è già posizionata nell'ordine corretto ($a < b$), inseriamo nella chiamata ricorsiva direttamente `x` (l'intera tupla catturata inalterata con `as`) senza sprecare risorse della macchina per ricreare inutilmente una nuova tupla (a,b) .

Esercizio dal Tutorato 3 - Separazione e Let-Val: Un grande classico d'esame: dividere una lista in due liste separate (una per i pari, una per i dispari), mantenendo l'ordine, esplorandola una sola volta.

```
fun separaPariDispari (nil) = (nil, nil)
  | separaPariDispari (x::xs) =
    let
      (* Decompone elegantemente la tupla restituita dalla ricorsione *)
      val (pari, dispari) = separaPariDispari (xs)
    in
      if x mod 2 = 0 then (x::pari, dispari)
      else (pari, x::dispari)
    end;
```

In questo frammento logico, il `let` evita la disastrosa pratica di calcolare due volte la ricorsione `separaPariDispari(xs)` (una volta per la testa e una volta per la coda).

10.2 Insiemi Logici tramite Liste (Esercizi L4.6 - L4.8)

Nel corso non esistono strutture dati pre-fabbricate per gli "Insiemi" (Set). Devono essere simulati e controllati manualmente garantendo l'assenza di duplicati logici.

Appartenenza (member) ed Eliminazione (delete): L'uso della *wildcard* `_` per il parametro non utilizzato ("`x`") nel caso base previene i *warning* di tipo.

```
fun member (_, nil) = false
  | member (x, y::ys) = (x = y orelse member(x, ys));

fun delete (a, []) = []
  | delete (b, c::ys) =
    if b = c then ys
    else c :: delete(b, ys);
```

Inserimento con controllo di univocità (insert): Anche qui l'efficienza si ottiene combinando l'ambiente locale tramite l'identificatore strutturato.

```
fun insert (x, nil) = [x]
  | insert (x, S as y::ys) =
    if x = y then S
    else y :: insert(x, ys);
```

11 Input-output, eccezioni e funzioni polimorfe

11.1 Spiegazione teorica

Questa sezione affronta la gestione delle operazioni di input/output, il sistema di controllo e recupero degli errori a runtime tramite le eccezioni, e i fondamenti del polimorfismo nel sistema di tipi di ML.

Input e Output in ML: A differenza della valutazione pura di espressioni matematiche, le operazioni di I/O introducono nel linguaggio il concetto di **effetto collaterale** (side-effect), modificando lo stato dei canali di comunicazione standard o dei file memorizzati su disco.

- **Output basico:** La funzione predefinita `print` accetta come argomento una stringa e la stampa sullo standard output. La funzione possiede la firma `string -> unit`; restituisce l'unico valore del tipo `unit`, ovvero la tupla vuota `()`, indicando che l'espressione ha esaurito la sua computazione producendo unicamente un effetto collaterale a video.
- **Espressioni composte:** Per eseguire una sequenza ordinata di operazioni che causano effetti collaterali, ML permette l'uso di espressioni composte separate dal punto e virgola e racchiuse tra parentesi tonde: `(exp1; exp2; ...; expn)`. Il tipo e il valore finale dell'intera macro-espressione sono determinati unicamente dall'ultima sotto-espressione valutata (`expn`). Le istruzioni intermedie servono esclusivamente per i loro effetti collaterali e l'ultima istruzione non necessita del punto e virgola di chiusura.

- **Gestione dei File (TextIO):** ML offre supporto linguistico all'interazione con il file system tramite la struttura standard `TextIO`. I canali di lettura vengono gestiti tramite oggetti di tipo `TextIO.instream`. Le funzioni principali includono:
 - `TextIO.openIn(filename)`: Apre un file di testo in modalità lettura e restituisce un canale `instream`.
 - `TextIO.endOfStream(infile)`: Restituisce un valore booleano che indica se si è raggiunta la fine del file.
 - `TextIO.inputN(infile, n)`: Legge una quantità fissa pari a `n` caratteri dal file, restituendoli sotto forma di stringa.
 - `TextIO.input(infile)`: Legge l'intero contenuto rimanente del file come un'unica stringa.
 - `TextIO.inputLine(infile)`: Legge una riga di testo comprensiva del carattere di newline `\n`. Restituisce un tipo speciale costruttore denominato **option**.

Il tipo costruttore option: In presenza di funzioni che possono non restituire un valore valido (come la lettura oltre la fine di un file o la ricerca in un dizionario), ML evita l'uso di puntatori nulli fragili introducendo il tipo parametrico `'a option`. Un oggetto di tipo `option` può assumere due forme:

- **SOME v:** Contiene il valore effettivo `v` di tipo `'a`.
- **NONE:** Indica l'assenza di un valore valido (es. fine del file raggiunta).

Per estrarre il valore da una variabile `option` forzandone il tipo da `'a option` ad `'a`, si utilizza la funzione di sistema `valOf`. Se applicata su un elemento che contiene **SOME v**, restituisce `v`; se applicata su **NONE**, interrompe il programma sollevando l'eccezione `Option`.

Eccezioni in ML: Le eccezioni sono costrutti linguistici semantici per interrompere il flusso normale del programma in presenza di errori a runtime.

- **Eccezioni di sistema:** Sollevate automaticamente dalla macchina astratta di ML. Esempi comuni sono `Div` (divisione per zero), `Empty` (applicazione di `hd` o `tl` su liste vuote) e `Chr` (conversione ASCII fuori range).
- **Eccezioni utente e parole chiave:** Il programmatore può definire nuove eccezioni nell'ambiente tramite la keyword `exception Nome`. Le eccezioni possiedono un tipo primitivo dedicato chiamato `exn`. Per innescare materialmente un errore si usa la keyword `raise Nome`.
- **Eccezioni con parametri:** È possibile associare un contenuto informativo all'eccezione definendola tramite la sintassi `exception Nome of tipo`. In questo caso l'identificatore agisce da costruttore (es. `exception Errore di int`, lanciata con `raise Errore(42)`).
- **Gestione delle eccezioni (handle):** Per intercettare un errore e ripristinare il flusso si appone l'operatore `handle` a valle di un'espressione, implementando una sintassi analoga al pattern matching: `<espressione> handle PatternEccezione => espressione_di_recupero`.

Funzioni polimorfe in ML: Il termine polimorfismo indica la capacità di un frammento di codice di operare su molteplici forme e tipi di dato. ML adotta un approccio statico rigoroso (*compile-time strong typing*), ma implementa il polimorfismo parametrico: se una funzione non esegue operazioni esclusive di un tipo specifico, il compilatore le assegna una variabile di tipo generica indicata con un singolo apice iniziale ('a, 'b).

Interazione tra Operatori e Polimorfismo:

- **Operatori che restringono il polimorfismo:** Gli operatori aritmetici classici (+, -, *), gli operatori di divisione (/, div, mod), i confronti di disuguaglianza (<, >), i connettivi booleani e le funzioni di conversione esplicita forzano i parametri ad assumere tipi primitivi rigidi (es. `int` o `real`), inibendo il polimorfismo.
- **Operatori che preservano il polimorfismo:** Gli operatori sulle tuple (creazione e proiezioni `#n`), gli operatori nativi sulle liste (`::`, `@`, `hd`, `tl`, `nil`) e gli operatori di uguaglianza logica consentono l'uso di tipi generici parametrici.
- **Tipi di uguaglianza (Equality types):** Gli operatori = e <> richiedono che i tipi messi a confronto supportino la verifica di uguaglianza. Sono considerati tipi di uguaglianza gli interi, i booleani, i caratteri, le stringhe e le tuple/liste composte dagli stessi. Sono strutturalmente **esclusi** i numeri reali (`real`) e tutte le funzioni. Le variabili di tipo generico vincolate a supportare esclusivamente l'uguaglianza vengono identificate dal compilatore con un **doppio apice iniziale** ("a). Cercare di applicare funzioni con vincolo "a su numeri reali genera un errore statico di unificazione del tipo.
- **Osservazione critica sull'inversione di liste:** Se si scrive una funzione di inversione testando la fine della lista tramite il controllo booleano `if L = nil then ...`, l'uso dell'operatore = costringe il compilatore ad assegnare alla funzione il tipo vincolato "a list -> "a list. Di conseguenza, la funzione fallirà il controllo dei tipi se applicata a liste di numeri reali o di funzioni. Per preservare il polimorfismo puro ('a list -> 'a list), si deve ricorrere al pattern matching sui costruttori della lista o utilizzare la funzione predefinita `null(L)`.

11.2 Implementazione del codice

11.2.1 Esempio di I/O e gestione dei flussi di lettura da File

(* Funzione ricorsiva ausiliaria che consuma lo stream carattere per carattere *)

```
fun makeList1 (infile, NONE) = nil
  | makeList1 (infile, SOME c) =
    c :: makeList1(infile, TextIO.input1(infile));
```

(* Inizializza la lettura estraendo il primo carattere *)

```
fun makeList (infile) = makeList1(infile, TextIO.input1(infile));
```

(* Utilizzo completo con apertura e chiusura del canale *)

```
fun readFileAndConvert (filename) =
  let
    val instream = TextIO.openIn(filename)
    val charList = makeList(instream)
  in
    TextIO.closeIn(instream); (* Effetto collaterale di chiusura *)
    charList                  (* Restituisce la lista di caratteri *)
  end;
```

11.2.2 Eccezioni personalizzate e meccanismi di recupero (handle)

```
exception OutOfRange of int * int;
```

```
fun comb1 (n, m) =
  if n <= 0 then raise OutOfRange(n, m)
  else if m < 0 orelse m > n then raise OutOfRange(n, m)
  else if m = 0 orelse m = n then 1
  else comb1(n - 1, m) + comb1(n - 1, m - 1);
```

(* La funzione comb fa da wrapper gestendo l'eccezione tramite handle *)

```
fun comb (n, m) =
  comb1(n, m)
  handle OutOfRange(0, 0) => 1
    | OutOfRange(n, m) =>
      (print("Error: values out of range. n=" ^ Int.toString(n) ^
        " m=" ^ Int.toString(m) ^ "\n");
       0);
```

11.2.3 Polimorfismo puro vs Polimorfismo di uguaglianza

```
(* Funzione con polimorfismo puro: accetta qualsiasi tipo *)
fun identity (x) = x;
(* Tipo assegnato: val identity = fn: 'a -> 'a *)

(* Funzione con polimorfismo vincolato all'uguaglianza *)
fun identity_eq (x) = if (x = x) then x else x;
(* Tipo assegnato: val identity_eq = fn: ''a -> ''a *)

(* Funzione di inversione che preserva il polimorfismo totale *)
fun rev_polymorphic (nil) = nil
  | rev_polymorphic (x::xs) = rev_polymorphic(xs) @ [x];
(* Tipo assegnato: val rev_polymorphic = fn: 'a list -> 'a list *)
```

12 Laboratorio ML 5

Il quinto laboratorio si concentra sull'ottimizzazione delle funzioni ricorsive tramite l'uso efficiente dell'ambiente locale (`let val`), sull'analisi delle espressioni di controllo condizionale annidate (`case`) e sull'implementazione pratica di funzioni che generano effetti collaterali tramite la stampa a video.

12.1 Ottimizzazione dell'Ambiente Locale (`let val`)

Esercizio L5.4 - Insieme delle parti (`powerSet`) efficiente: Rispetto alla versione base in cui la chiamata ricorsiva sulla coda della lista veniva calcolata due volte raddoppiando la complessità a ogni passo, l'introduzione di un legame temporaneo nel `let` riduce drasticamente i tempi di calcolo memorizzando il risultato parziale.

```
(* Funzione di supporto per inserire un elemento in tutte le sotto-liste *)
fun insertAll (x, nil) = nil
  | insertAll (x, y::ys) = (x::y) :: insertAll(x, ys);

fun powerSet (nil) = [nil]
  | powerSet (x::xs) =
    let
      val tailPowerSet = powerSet(xs) (* Calcolata una sola volta *)
    in
      tailPowerSet @ insertAll(x, tailPowerSet)
    end;
```

Esercizio L5.7 - Computazione efficiente di x^{2^i} (doubleExp): Scrivere un programma che calcoli la potenza doppia effettuando una sola chiamata ricorsiva per ogni decremento dell'indice i .

```
fun doubleExp (x: real, 0) = x
  | doubleExp (x, i) =
    let
      val y = doubleExp(x, i - 1)
    in
      y * y
    end;
```

Esercizio L5.3 - Destrutturazione di Tuple senza Pattern Matching diretto: Scrivere la funzione `split` estraendo i valori della tupla risultante tramite l'uso esplicito delle funzioni di proiezione `#1` e `#2` all'interno del blocco locale.

```
fun split (nil) = (nil, nil)
  | split ([a]) = ([a], nil)
  | split (a::b::cs) =
    let
      val tuplaRisultato = split(cs)
      val M = #1 tuplaRisultato
      val N = #2 tuplaRisultato
    in
      (a::M, b::N)
    end;
```

12.2 Uso delle espressioni case

Esercizio L5.1 - Simulazione dell'if-then-else tramite case: Implementare la funzione `is_lower_than5` verificando la validità della condizione logica booleana tramite pattern matching interno.

```
fun is_lower_than5 n =
  case (n < 5) of
    true => 1
  | false => 2;
```

12.3 Esercizi di Input/Output e Stampa

Esercizio L5.9 - Stampa di una lista (printList): Scrivere una funzione che accetti una lista di interi e la stampi riga per riga sullo standard output.

```
fun printList (nil) = ()
  | printList (x::xs) =
    (
      print(Int.toString(x));
      print("\n");
      printList(xs)
    );
```

Esercizio L5.10 - Calcolo e Formattazione Combinazioni (comb): Scrivere una funzione che calcoli il coefficiente binomiale stampando a video in modo strutturato i parametri inseriti e il risultato finale ottenuto.

```
fun factorial 0 = 1
  | factorial n = n * factorial(n - 1);

fun comb (n, m) =
  (
    print("n is ");
    print(Int.toString(n));
    print("\n");
    print("m is ");
    print(Int.toString(m));
    print("\n");
    print("Result is ");
    print(Int.toString(factorial(n) div (factorial(m) * factorial(n - m))));
    print("\n")
  );
```

13 Funzioni di ordine superiore e curried

13.1 Spiegazione teorica

In ML le funzioni sono trattate come "cittadini di prima classe", il che significa che possono essere assegnate a variabili, passate come argomenti ad altre funzioni o restituite come risultato. Una funzione che accetta o restituisce un'altra funzione prende il nome di **Funzione di Ordine Superiore** (Higher-Order Function).

Gestione dell'Ambiente e Binding: Quando si passa una funzione come parametro o la si restituisce in uscita, sorge un problema architetturale: quale ambiente non-locale deve essere utilizzato durante la sua futura esecuzione?

- **Deep binding:** L'ambiente non-locale viene "congelato" al momento della *creazione* (o passaggio) della funzione. È l'approccio adottato in accoppiata allo *Scoping Statico*.
- **Shallow binding:** L'ambiente non-locale viene valutato solo al momento in cui la funzione viene effettivamente *chiamata*.

Come già visto, ML utilizza rigorosamente lo **Scoping Statico** abbinato al **Deep binding**. Questa combinazione viene tecnicamente implementata attraverso le *Closure*, ovvero strutture dati nascoste che memorizzano il codice della funzione e l'ambiente al momento della sua definizione, garantendone la robustezza e la prevedibilità.

Funzioni Curried (Currying): A livello formale e sintattico, **ogni funzione in ML accetta uno e un solo argomento**. Fino a questo momento abbiamo aggirato questo limite matematico passando argomenti multipli impacchettati all'interno di una tupla (es. `fun max(x, y)`). Un approccio alternativo, proprio dei linguaggi puramente funzionali, è la **forma Curried** (in onore del logico Haskell Curry). Una funzione a due argomenti viene progettata in modo tale da prendere il primo argomento e *restituire una nuova funzione* che attenderà il secondo.

- *Sintassi Tupla:* `fun exp1(x, y) = ...` ha tipo `real * int -> real`.
- *Sintassi Curried:* `fun exp2 x y = ...` ha tipo `real -> int -> real` (valutato a destra come `real -> (int -> real)`).
- **Vantaggio: Applicazione Parziale (Partial Instantiation).** Le funzioni curried permettono di fornire soltanto una parte degli argomenti richiesti, generando istantaneamente una nuova funzione "specializzata" da riutilizzare a runtime. Ad esempio, potremmo fissare la base di una potenza a 3.0 e usare la nuova funzione creata solo per far variare l'esponente.

13.2 Implementazione del codice

13.2.1 Funzioni di Ordine Superiore Classiche (Implementate a mano)

Costruiamo le versioni semplificate delle classiche funzioni funzionali per comprendere il loro comportamento interno (il linguaggio ML dispone già di versioni built-in altamente ottimizzate).

```
(* 1. simpleMap: applica la funzione F a tutti gli elementi di una lista *)
fun simpleMap (F, nil) = nil
  | simpleMap (F, x::xs) = F(x) :: simpleMap(F, xs);
(* Tipo: ('a -> 'b) * 'a list -> 'b list *)

(* 2. filter: mantiene solo gli elementi che soddisfano il predicato P *)
fun filter (P, nil) = nil
  | filter (P, x::xs) =
    if P(x) then x :: filter(P, xs)
    else filter(P, xs);
(* Tipo: ('a -> bool) * 'a list -> 'a list *)

(* 3. reduce: collassa la lista applicando ricorsivamente la funzione F *)
exception EmptyList;
fun reduce (F, nil) = raise EmptyList
  | reduce (F, [a]) = a
  | reduce (F, x::xs) = F(x, reduce(F, xs));
(* Tipo: ('a * 'a -> 'a) * 'a list -> 'a *)
```

13.2.2 Currying e Applicazione Parziale

```
(* Funzione in forma Curried, notare l'assenza di virgole e parentesi *)
fun exponent2 x 0 = 1.0
  | exponent2 x y = x * exponent2 x (y - 1);

(* Possiamo chiamarla fornendo subito entrambi i parametri... *)
> exponent2 3.0 4;
val it = 81.0 : real

(* ...oppure possiamo fornire solo il primo parametro (la base 3.0),
   ottenendo in ritorno una NUOVA funzione pronta a ricevere un esponente *)
> val powerOfThree = exponent2 3.0;
val powerOfThree = fn : int -> real

> powerOfThree 4;
val it = 81.0 : real
```

14 Laboratorio ML 6

Nel sesto laboratorio si fondono le nozioni di I/O (File e testuali) introdotte marginalmente nella Lezione 7, assieme alle Funzioni di Ordine Superiore, per costruire elaboratori dati puramente funzionali.

14.1 Elaborazione di Flussi di Testo (I/O)

Viene esplorata l'interazione con l'esterno usando la struttura `TextIO`, supportata dai tipi costruttori come `option` (per gestire letture di fine file che restituiranno `NONE`) e dal pattern matching per il disinnescamento degli errori.

Esercizio base: Primitive TextIO

```
(* 1. Aprire un file in lettura (restituisce un instream) *)
val input = TextIO.openIn("file.txt");

(* 2. Leggere i successivi 5 caratteri (restituisce string) *)
TextIO.inputN(input, 5);

(* 3. Leggere una riga intera (restituisce string option) *)
TextIO.inputLine(input);

(* 4. Leggere il prossimo carattere senza consumarlo dal flusso *)
TextIO.lookahead(input);

(* 5. Chiudere rigorosamente il flusso per liberare la risorsa *)
TextIO.closeIn(input);
```

14.2 Esercizi dal Tutorato: I/O Sicuro ed Eccezioni

Nel tutorato viene richiesto di costruire dei gestori di file incapsulati che intercettano gli errori di sistema per lanciare eccezioni di dominio semantiche.

Esercizio: Leggere la prima riga in sicurezza Scrivere una funzione che tenti di leggere la prima riga di un file. Se il file non esiste, bisogna catturare l'eccezione di sistema di basso livello (`Io`) per sollevarne una propria (`FileNotFound`). Se il file esiste ma è vuoto, lanciare `EmptyFile`.

```
exception FileNotFound of string;
exception EmptyFile;
fun readFirstLine(filename) =
  let
    (* Tentativo di apertura controllato tramite handle *)
    val file = TextIO.openIn(filename)
        handle Io => raise FileNotFound(filename)
    val lineOpts = TextIO.inputLine(file)
  in
    TextIO.closeIn(file);
    (* Valutazione polimorfica dell'option per l'esito della lettura *)
    case lineOpts of
      NONE => raise EmptyFile
    | SOME(line) => line
  end;
```

14.3 Combinare I/O e Ordine Superiore

Esercizio L6.9 - Lettura e Somma (readAndSum): Scrivere una funzione che legga un file di testo contenente un numero (stringa) per ogni riga, ne esegua il parsing ad intero e ne calcoli la somma totale. Si richiede l'uso della funzione nativa `Int.fromString` che trasforma stringhe in interi, restituendo un `int option` di sicurezza.

```
fun readAndSum1 file =
  if TextIO.endOfStream(file) then 0
  else (* valOf forza l'estrazione "scartando" il costruttore SOME,
        estraendo il valore reale. Si usa due volte:
        per l'inputLine e per fromString. Nota: l'uso di valOf è
        considerato "unsafe" (può lanciare eccezioni); in produzione
        si predilige sempre l'uso del pattern matching o del case.*)

    valOf(Int.fromString(valOf(TextIO.inputLine(file))))
    + readAndSum1(file);
fun readAndSum filename =
  let
    val f = TextIO.openIn(filename)
    val res = readAndSum1(f)
  in
    TextIO.closeIn(f); (* Chiusura garantita *)
    res
  end;
```


Esercizio L6.10 - Lista di funzioni curried (applyList): Un eccellente esercizio in forma curried: scrivere una funzione che accetti prima una lista di funzioni (es. elevamento al quadrato, al cubo) e in seguito un valore *a*, restituendo una lista in cui ogni funzione è stata iterativamente applicata al valore di partenza.

```
(* Funzione Curried: prende prima la lista Fs, poi si "congela"
   aspettando che venga fornito il valore 'a' *)
fun applyList nil a = nil
  | applyList (F::Fs) a = F(a) :: (applyList Fs a);

(* Esempio di utilizzo con istanziiazione finale: *)
> applyList [fn x => x * 2, fn x => x * x * x] 4;
val it = [8, 64] : int list
```

15 Tipi di dati e composizione di funzioni

15.1 Spiegazione teorica

In questa lezione si conclude la panoramica sulle funzioni di ordine superiore esaminando le primitive di libreria, l'operatore di composizione e, soprattutto, l'architettura per la definizione di nuovi tipi di dato e strutture ricorsive.

Funzioni di Ordine Superiore di Sistema (Built-in): Le funzioni di ordine superiore fornite nativamente da ML (come *map*, *foldr* e *foldl*) sono **curried**, ovvero si aspettano gli argomenti passati in sequenza (separati da spazio) e non raggruppati in una singola tupla.

- **map:** Applica una determinata funzione a ciascun elemento di una lista, restituendo una nuova lista con i risultati.
- **foldr (Fold Right):** "Collassa" una lista in un singolo valore combinando gli elementi da destra verso sinistra, partendo da un valore iniziale base. Corrisponde logicamente all'applicazione annidata: $f(x_1, f(x_2, \dots f(x_n, c)))$.
- **foldl (Fold Left):** Applica l'aggregazione partendo da sinistra verso destra. È spesso più efficiente poiché strutturabile tramite *tail recursion*. Corrisponde a: $f(x_n, f(x_{n-1}, \dots f(x_1, c)))$.

Composizione di Funzioni: La composizione matematica di due funzioni, $H(x) = G(F(x))$, viene supportata in ML dall'operatore infisso *o* (una semplice lettera 'o' minuscola). La sintassi *G o F* crea un'unica nuova funzione in cui l'output di *F* viene reindirizzato come input di *G*. È una tecnica dichiarativa fondamentale per concatenare manipolazioni di liste ed elaborazioni dati in catene di montaggio algoritmiche (*pipeline*).

Il Sistema dei Tipi in ML: Per estendere i tipi predefiniti (*int*, *real*, *string*, liste e tuple), ML mette a disposizione due costrutti principali:

- **type (Ridenominazione e Alias):** Permette di creare un alias per un tipo già esistente o per un'associazione complessa, con lo scopo di rendere il codice più leggibile per il programmatore. Non genera veri e propri tipi nuovi per la macchina. Sintassi base: *type signal = int list*; oppure *type ('a, 'b) mapping = ('a * 'b) list*;

- **datatype (Tipi e Costruttori Strutturati):** Introduce tipologie di dato interamente nuove. Costituisce la base per i *Tipi Unione* (*Union Types* o ADT). Un **datatype** definisce un **costruttore di tipo** (il nome generale) e uno o più **costruttori di dato** (i valori validi, separati dal pipe |). I costruttori di dato possono anche "trasportare" un parametro, permettendo di incapsulare valori (es. `S of 'a`).

Tipi di dato Ricorsivi: Poiché i costruttori possono riferirsi al **datatype** stesso che stanno definendo, questo strumento linguistico è ideale per implementare **Alberi Binari**, liste concatenate custom o altre strutture dati dinamiche non lineari. Per generare **tipi mutuamente ricorsivi** (come un Albero la cui definizione dipende da una Foresta e viceversa), si connettono le rispettive dichiarazioni utilizzando la keyword `and`.

15.2 Implementazione del codice

15.2.1 Composizione e Primitive List

```
(* Utilizzo di map per estrarre la lunghezza delle stringhe *)
> map size ["La", "lezione", "nove"];
val it = [2, 7, 4] : int list

(* Utilizzo combinato di foldr per riprodurre un Logical AND su lista *)
> val andb = foldr (fn (x,y) => x andalso y) true;
val andb = fn : bool list -> bool
> andb [true, false, true];
val it = false : bool

(* Composizione elegante: converte i char in stringhe e li concatena *)
> val implode = (foldr (op ^) "") o (map str);
val implode = fn : char list -> string
```

15.2.2 Costrutti Datatype e Union Types

```
(* Tipo Unione Base: Enumerazione *)
> datatype fruit = Apple | Pear | Grape;

(* Uso pratico dell'Unione col Pattern Matching *)
> fun isApple (x) = (x = Apple);
val isApple = fn : fruit -> bool

(* Tipo Elemento Generico: può contenere un Pair o un Valore Singolo *)
> datatype ('a, 'b) element = P of 'a * 'b | S of 'a;
> P("a", 1);
val it = P ("a", 1) : (string, int) element

(* Somma sicura ignorando gli elementi singoli S *)
fun sumElList (nil) = 0
  | sumElList (S(x)::L) = sumElList(L)
  | sumElList (P(x,y)::L) = y + sumElList(L);
```

16 Laboratorio ML 7

Nel settimo laboratorio si sperimenta materialmente l'utilizzo delle funzioni curried e la definizione algoritmica di esplorazione degli Alberi Binari creati attraverso i `datatype`.

16.1 Currying e Composizione Pratica (Esercizio L7.1 - L7.4)

Scrivere una funzione che prende una funzione non curried (una che lavora sulle tuple) e la trasforma nella sua versione curried, separandone gli argomenti in chiamate ad anello.

```
(* Definizione di un generalizzatore Curry (arità 3) *)
fun curry F x1 x2 x3 = F(x1, x2, x3);

(* Utilizzo del trasformatore *)
> curry (fn (x,y,z) => x * y * z);
val it = fn : int -> int -> int -> int -> int

(* Assegnazione tramite Applicazione Parziale *)
> val G = curry (fn (x,y,z) => x * y * z) 1 2;
> G 3;
val it = 6 : int
```

16.2 Tipi di dato per gli Alberi Binari (Esercizi L7.5 - L7.10)

L'albero binario generico per tutti gli esercizi di questa sezione viene così codificato:

```
datatype 'a btree = Empty
                  | Node of 'a * 'a btree * 'a btree;
```

Esercizi L7.5 e L7.6 - Visita PostOrder e InOrder: Implementare le canoniche funzioni di esplorazione degli alberi riordinando le chiamate ricorsive e ricomponendo i rami tramite concatenazione di liste (@).

```
fun postOrder (Empty) = nil
  | postOrder (Node(a, left, right)) =
    postOrder(left) @ postOrder(right) @ [a];

fun inOrder (Empty) = nil
  | inOrder (Node(a, left, right)) =
    inOrder(left) @ [a] @ inOrder(right);
```

Esercizio L7.8 e L7.10 - Calcolo e Manipolazione (sumTree e doubleTree): Sommare le componenti o alterare l'intero strato dei nodi clonando strutturalmente l'albero. Nel primo esempio si assume un albero che contiene tuple di tipo `string * int` come etichette dei nodi (*mapTree*).

```
(* Somma la seconda componente (b) delle coppie di label *)
fun sumTree Empty = 0
  | sumTree (Node((a, b), left, right)) =
    b + sumTree(left) + sumTree(right);

(* Raddoppia i valori producendo un albero binario della medesima forma *)
fun doubleTree Empty = Empty
  | doubleTree (Node(x, left, right)) =
    Node(x * 2, doubleTree(left), doubleTree(right));
```

16.3 Esercizi e approfondimenti dal tutorato: Analisi di Struttura sugli Alberi

Le simulazioni d'esame e le sessioni di tutorato (es. schede sugli Alberi Binari) arricchiscono le capacità di investigare la topologia dell'albero e valutarne gli attributi matematici (Alberi Bilanciati e Perfetti).

Esercizio: Altezza e Albero Perfetto (isPerfect) Un albero è perfetto se tutte le foglie giacciono sulla stessa identica profondità e ogni nodo non foglia ha precisamente due figli. Si deve valutare coerentemente l'uguaglianza tra i rami sottostanti.

```
(* Funzione di supporto vitale per l'analisi della topologia *)
fun height Empty = 0
  | height (Node(_, l, r)) = 1 + Int.max(height(l), height(r));

fun isPerfect Empty = true
  | isPerfect (Node(_, l, r)) =
    height(l) = height(r) andalso isPerfect(l) andalso isPerfect(r);
```

Esercizio: Albero Bilanciato (isBalanced) Un albero è bilanciato se la profondità dei due sotto-alberi relativi a un qualsiasi nodo differisce in altezza, al massimo, per un valore assoluto di 1 (analogia con gli *Alberi AVL*). La soluzione proposta prevede uno scorrimento inefficiente adottando la restituzione di ~ 1 come codice di errore di sbilanciamento (Fail-Fast pattern interno).

```
fun isBalanced t =
  let
    (* Restituisce l'altezza, o ~1 nel caso un ramo fosse sbilanciato *)
    fun checkBal Empty = 0
      | checkBal (Node(_, l, r)) =
        let
          val hl = checkBal(l)
          val hr = checkBal(r)
        in
          if hl = ~1 orelse hr = ~1 orelse abs(hl - hr) > 1 then ~1
          else 1 + Int.max(hl, hr)
        end
      end
  in
    checkBal(t) <> ~1
  end;
```

Nota di ottimizzazione architetturale per l'esame: Calcolare sistematicamente `height(l)` e poi chiamare di nuovo `isBalanced(l)` provocherebbe un costo esponenziale sull'attraversamento (l'albero verrebbe scorso ripetutamente ai livelli bassi). Questa funzione annidata incapsula l'esplorazione restituendo entrambe le informazioni in una *singola* passata post-order, scalando la complessità ad $O(N)$.

Esercizio: Albero Specchio (mirrorTree) Data una radice asimmetrica, restituire il corrispettivo speculare ribaltando sistematicamente l'orientamento foglia-per-foglia.

```
fun mirrorTree Empty = Empty
  | mirrorTree (Node(v, l, r)) = Node(v, mirrorTree(r), mirrorTree(l));
```

17 Abstract Data Types in ML e Funtori

17.1 Spiegazione teorica

Per costruire sistemi software scalabili e sicuri, i linguaggi moderni offrono meccanismi di **Astrazione dei Dati** (Data Abstraction). L'obiettivo è separare nettamente l'*interfaccia* (cosa fa un tipo di dato e quali operazioni supporta) dalla sua *implementazione* (come i dati sono materialmente rappresentati in memoria).

In ML, questo incapsulamento (simile al concetto di classe e interfaccia in Java) viene realizzato attraverso tre costrutti fondamentali: **Signature**, **Structure** e **Functor**.

1. Signatures (Firme): Una *signature* agisce come un contratto pubblico o un'interfaccia. Definisce i tipi, le eccezioni e le firme delle funzioni che una struttura dovrà obbligatoriamente implementare.

- Sintassi: `signature NOME_SIG = sig ... end`
- Non contiene alcuna logica o codice eseguibile, ma solo dichiarazioni (`val`, `type`, `exception`).

2. Structures (Strutture): Una *structure* è l'implementazione concreta (il modulo) che fornisce il codice per i tipi e i metodi. Si collega a una signature per garantirne la validità strutturale (Ascription).

- **Ascription Trasparente (:):** `structure S : SIG = struct ... end`. Il compilatore controlla che *S* rispetti *SIG*. Tuttavia, l'implementazione interna dei tipi rimane visibile all'esterno (violando parzialmente l'incapsulamento).
- **Ascription Opaca (:>):** `structure S :> SIG = struct ... end`. È l'astrazione dei dati per eccellenza. Tutto ciò che non è esplicitamente definito nella signature viene nascosto. Se il tipo interno è implementato come `int`, all'esterno verrà visto solo come tipo astratto inaccessibile, impedendo all'utente di farvi operazioni aritmetiche illecite.

3. Functors (Funtori): Un funtore è una "struttura parametrizzata", ovvero una funzione che opera a livello di moduli. Prende in input una o più *structure* (che rispettano determinate *signature*) e genera in output una nuova *structure*.

- Sono il meccanismo principale in ML per la riusabilità del codice. Ad esempio, si può scrivere un funtore che genera un Albero Binario di Ricerca (BST) generico, passando in input una struttura che definisce semplicemente la funzione logica di ordinamento (es. alfabetico o numerico).

17.2 Implementazione del codice: Trasparenza vs Opacità

Il seguente frammento (analizzato nel Lab 9) dimostra la potenza dell'ascription opaca per proteggere l'invariante di un tipo astratto.

```
(* 1. La Firma nasconde l'implementazione del tipo 't' *)
signature MY_INT = sig
    type t
    val crea: int -> t
end;

(* 2. Struttura Trasparente: espone i dettagli *)
structure Transparent : MY_INT = struct
    type t = int
    fun crea x = x
end;

(* 3. Struttura Opaca (ADT Puro): blinda i dettagli *)
structure Opaque :> MY_INT = struct
    type t = int
    fun crea x = x
end;

(* Test di violazione dell'astrazione: *)
> val a = Transparent.crea(5);
> val b = a + 1; (* OK! Il compilatore sa che 'a' è un int *)

> val c = Opaque.crea(5);
> val d = c + 1;
(* ERRORE FATALE! Il compilatore blocca l'operazione.
   Per l'esterno, 'c' è di tipo astratto 't', non un 'int',
   quindi l'operatore '+' non è applicabile. *)
```

18 Laboratorio ML 8 - Alberi Binari di Ricerca

Il Laboratorio 8 e le relative schede di tutorato applicano le strutture astratte all'implementazione del **Binary Search Tree (BST)**, una struttura dati cruciale per la ricerca efficiente. La regola invariante di un BST stabilisce che per ogni nodo, tutti i valori nel sotto-albero sinistro devono essere strettamente minori del nodo, e tutti quelli nel sotto-albero destro strettamente maggiori.

18.1 Creazione di un BST tramite Functor (MakeBST)

Per rendere l'albero riutilizzabile per interi, stringhe o record personalizzati, lo si genera tramite un funtore che riceve in input l'operatore di confronto `lt` (less than).

```
(* Step 1: Signature che definisce il contratto di ordinamento *)
signature TOTALORDER = sig
    type t
    val lt : t * t -> bool
end;

(* Step 2: Struttura concreta per stringhe *)
structure StringOrder : TOTALORDER = struct
    type t = string
    fun lt (a, b) = a < b
end;

(* Step 3: Funtore per costruire l'albero *)
functor MakeBST (Order : TOTALORDER) = struct
    datatype bst = Empty
                | Node of Order.t * bst * bst

    fun insert (x, Empty) = Node(x, Empty, Empty)
      | insert (x, T as Node(y, left, right)) =
        if Order.lt(x, y) then
            Node(y, insert(x, left), right)
        else if Order.lt(y, x) then
            Node(y, left, insert(x, right))
        else T (* L'elemento esiste già, l'albero resta inalterato *)
end;

(* Step 4: Istanziamento della struttura finale *)
structure StringBST = MakeBST(StringOrder);
```


18.2 Esercizi e approfondimenti dal tutorato (Strutture e BST)

Nelle simulazioni d'esame è frequentissima la richiesta di definire una coda o un dizionario proteggendone lo stato tramite *signature*, oppure di manipolare un BST.

Esercizio: Progettazione ADT della Coda (Queue) Definire una *signature* e un'implementazione opaca per una Coda FIFO.

```
signature QUEUE = sig
  type 'a queue
  exception EmptyQueue
  val emptyQueue: 'a queue
  val enqueue: 'a -> 'a queue -> 'a queue
  val dequeue: 'a queue -> 'a * 'a queue
end;

structure Queue :> QUEUE = struct
  type 'a queue = 'a list
  exception EmptyQueue

  val emptyQueue = []

  (* Inserimento in coda (inefficiente su liste lunghe, ma didattico) *)
  fun enqueue x [] = [x]
    | enqueue x (y::ys) = y :: enqueue x ys

  (* Estrazione dalla testa *)
  fun dequeue [] = raise EmptyQueue
    | dequeue (x::xs) = (x, xs)
end;
```

Esercizio: Estrazione del Minimo in un BST (deletemin) Poiché nel BST i valori più piccoli collassano a sinistra, per trovare e rimuovere il minimo bisogna scendere lungo il ramo sinistro finché non si trova un nodo senza figli a sinistra.

```
exception EmptyTree;

fun deletemin Empty = raise EmptyTree
  (* Caso base: nessun figlio sinistro, questo è il minimo!
   Ritorno il valore e 'aggancio' l'eventuale ramo destro *)
  | deletemin (Node(y, Empty, right)) = (y, right)

  (* Caso ricorsivo: scendo a sinistra *)
  | deletemin (Node(w, left, right)) =
    let
      val (y, new_left) = deletemin(left)
    in
      (y, Node(w, new_left, right))
    end;
```

Esercizio: Ricerca di minimi e massimi in un'unica passata Esercizio avanzato del tutorato che sfrutta le tuple nell'ambiente locale per esplorare simultaneamente i bordi di un albero binario.

```
fun find_min_max Empty = raise EmptyTree
  | find_min_max (Node(v, Empty, Empty)) = (v, v)

(* Ha solo il figlio sinistro *)
| find_min_max (Node(v, left, Empty)) =
  let val (min_sx, max_sx) = find_min_max left
  in (Int.min(v, min_sx), Int.max(v, max_sx)) end

(* Ha solo il figlio destro *)
| find_min_max (Node(v, Empty, right)) =
  let val (min_dx, max_dx) = find_min_max right
  in (Int.min(v, min_dx), Int.max(v, max_dx)) end

(* Ha entrambi i figli *)
| find_min_max (Node(v, left, right)) =
  let
    val (min_sx, max_sx) = find_min_max left
    val (min_dx, max_dx) = find_min_max right
  in
    (Int.min(v, Int.min(min_sx, min_dx)),
     Int.max(v, Int.max(max_sx, max_dx)))
  end;
end;
```

19 Lambda Calcolo

19.1 Spiegazione teorica

Il **Lambda calcolo** (λ -calcolo) è un sistema formale logico-matematico sviluppato da Alonzo Church negli anni '30 per investigare i fondamenti della computazione. Sebbene sia nato decenni prima dei computer moderni, costituisce il cuore teorico e il modello di calcolo di tutti i linguaggi di programmazione funzionali (come LISP, Haskell, ML e Scala).

1. Sintassi del Lambda Calcolo: L'universo del lambda calcolo puro è estremamente essenziale. Non esistono numeri, booleani, stringhe o costrutti di base; esiste solo il concetto di "funzione anonima". Le espressioni (o λ -termini) si definiscono induttivamente tramite tre sole regole:

- **Variabile:** Un identificatore $x, y, z \dots$ è un λ -termine.
- **Astrazione (Creazione di una funzione):** Se e è un λ -termine e x è una variabile, allora $\lambda x.e$ è un λ -termine. Indica la definizione di una funzione anonima il cui parametro formale è x e il cui corpo è e (equivalente a `fn x => e` in ML).
- **Applicazione (Invocazione di una funzione):** Se e_1 ed e_2 sono λ -termini, l'affiancamento $(e_1 e_2)$ è un λ -termine. Rappresenta l'applicazione della funzione e_1 all'argomento e_2 . L'applicazione è associativa a sinistra: $x y z$ equivale a $(x y) z$.

2. Variabili Libere e Legate: In un'astrazione $\lambda x.e$, il simbolo λ funge da operatore di *binding* (legame).

- La variabile x è detta **legata** (bound) all'interno del corpo e .
- Una variabile che compare in un'espressione senza essere vincolata da alcun λ è detta **libera** (free).
- Esempio: In $(\lambda x.x\ y)$, x è legata dal λx , mentre y è libera.

3. Le Regole di Riduzione (La Computazione): Il calcolo progredisce attraverso riscritture sintattiche (riduzioni).

- **Alpha-conversione (α -riduzione):** È il rinominare in modo coerente le variabili legate per evitare "collisioni" o "catture" accidentali di nomi. Esempio: $\lambda x.x$ (la funzione identità) è α -equivalente a $\lambda y.y$ o $\lambda z.z$.
- **Beta-riduzione (β -riduzione):** È il vero motore della computazione. Definisce l'esecuzione di un'applicazione sostituendo il parametro formale con l'argomento attuale.
Sintassi formale: $(\lambda x.e_1)\ e_2 \rightarrow_\beta e_1[e_2/x]$
(Si sostituiscono tutte le occorrenze *libere* di x all'interno di e_1 con il termine e_2 , prestando attenzione ad applicare preventivamente un' α -conversione se vi è rischio di sovrapposizioni di nomi).

4. Forma Normale: Un λ -termine si definisce in **forma normale** se non contiene più alcun *redex* (radicale da ridurre). In altre parole, non è più possibile applicare alcuna β -riduzione. La forma normale rappresenta il "risultato" finale del calcolo.

5. Codifiche di Church (Encodings): Poiché nel λ -calcolo puro non ci sono tipi di dato, questi vengono "emulati" strutturalmente usando solo funzioni:

- **Booleani:**
 $Vero \equiv \lambda x.\lambda y.x$ (prende due argomenti e restituisce il primo)
 $Falso \equiv \lambda x.\lambda y.y$ (prende due argomenti e restituisce il secondo)
- **Numeri Naturali (Church Numerals):** I numeri sono codificati come funzioni di ordine superiore che applicano una certa funzione f un numero n di volte a un argomento x .
 $0 \equiv \lambda f.\lambda x.x$
 $1 \equiv \lambda f.\lambda x.f\ x$
 $2 \equiv \lambda f.\lambda x.f\ (f\ x)$

19.2 Implementazione e Riduzioni Logiche

Vediamo alcuni esempi classici di β -riduzione. *Nota notazionale:* Negli snippet e negli esercizi pratici seguenti, per comodità di formattazione ASCII (tipica degli interpreti o dei file di testo originali), l'operatore matematico λ viene rappresentato sintatticamente tramite il carattere backslash ($\backslash$).

19.2.1 Esempio 1: Applicazione di Base

Valutazione di un'applicazione semplice:

Termine: $(\lambda x. (\lambda z. x z)) y$
Regola: Il parametro formale è x , l'argomento è y .
Sostituiamo tutte le occorrenze di x nel corpo $(\lambda z. x z)$ con y .
Risultato: $\lambda z. y z$
(Il risultato è in forma normale in quanto non ci sono altre applicazioni).

19.2.2 Esempio 2: Valutazione di un Booleano con operatore

Modelliamo il costrutto `if True then A else B`:

Definizione If: $\lambda b. \lambda x. \lambda y. b x y$
Definizione True: $\lambda u. \lambda v. u$

Valutazione di $(\text{If True } A \ B)$:

1. $(\lambda b. \lambda x. \lambda y. b x y) (\lambda u. \lambda v. u) A B$
2. Sostituiamo b : $(\lambda x. \lambda y. (\lambda u. \lambda v. u) x y) A B$
3. Sostituiamo x con A : $(\lambda y. (\lambda u. \lambda v. u) A y) B$
4. Sostituiamo y con B : $(\lambda u. \lambda v. u) A B$
5. Sostituiamo u con A : $(\lambda v. A) B$
6. Sostituiamo v con B : A

Risultato corretto: Il ramo "True" ha restituito A .

19.3 Esercizi e approfondimenti dal tutorato: Computazione in Forma Normale

All'esame viene spesso fornito un lungo e complesso termine di Lambda Calcolo e viene richiesto di mostrarne passo-passo la riduzione fino all'ottenimento della Forma Normale. L'accortezza fondamentale risiede nell'evitare le collisioni applicando correttamente le parentesi e la precedenza (associativa a sinistra).

19.3.1 Esercizio L11.12: Riduzione annidata

Si riduca il seguente termine a forma normale:

$(\lambda x. x z x) ((\lambda y. y y x) z)$

Svolgimento Passo-Passo:

1. Notiamo che c'è un'applicazione annidata sulla destra. Conviene prima ridurre l'argomento (*Applicative Order*):

Sotto-termine destro: $(\lambda y. y y x) z$
Sostituisco y con z : $z z x$

2. Il nostro termine principale è ora semplificato:

$(\lambda x. x z x) (z z x)$

3. Attenzione alle collisioni! C'è un rischio di cattura di variabile per la x finale. Per procedere in assoluta pulizia formale (anche se qui le x combaciano come variabili libere di contesto), sostituiamo l'intero blocco (zzx) al posto delle x legate dal primo λ :

$(z z x) z (z z x)$

Risultato Finale: $(zzx)z(zzx)$. L'espressione non contiene più alcun simbolo di astrazione λ su cui effettuare applicazioni, pertanto è in forma normale.

19.3.2 Esercizio L11.13: Riduzione con Catture Evitate

Si riduca il seguente termine a forma normale:

Termine Iniziale:

$(\lambda x. x y) (\lambda t. t z) ((\lambda x. \lambda z. x y z) y x)$

Svolgimento Passo-Passo:

1. Isoliamo il blocco destro e risolviamolo per semplificare il lavoro. Notiamo l'applicazione di due argomenti in cascata (y e poi x).

Blocco destro: $((\lambda x. \lambda z. x y z) y) x$

Applico y alla x : $(\lambda z. y y z) x$

Applico x alla z : $y y x$

2. Il termine intero diventa ora un'applicazione sequenziale:

$(\lambda x. x y) (\lambda t. t z) (y y x)$

3. Il Lambda calcolo è associativo a sinistra. Applichiamo quindi il primo termine al secondo:

Sotto-termine sinistro: $(\lambda x. x y) (\lambda t. t z)$

Sostituisco $(\lambda t. t z)$ al posto di x : $(\lambda t. t z) y$

Applico y al posto di t : $y z$

4. Reinseriamo il risultato nell'equazione principale, unendo i due blocchi:

$(y z) (y y x)$

Risultato Finale: $(yz)(yyx)$. Tutte le funzioni astratte sono state risolte.

20 Programmazione Logica e Prolog

20.1 Spiegazione teorica

Il paradigma logico si discosta radicalmente sia dalla programmazione imperativa (che descrive *come* eseguire un calcolo modificando lo stato) sia da quella funzionale (che valuta espressioni e funzioni). La programmazione logica è ****dichiarativa****: il programmatore descrive la struttura logica del problema (le relazioni tra le entità) e il motore interno del linguaggio si occupa di trovare la soluzione.

Fondamenti del Paradigma Logico:

- **Relazioni vs Funzioni:** In matematica, una funzione $f(x) = y$ mappa un input a un output. Una relazione $R(x, y)$ afferma semplicemente che x e y sono legati in qualche modo. La programmazione logica si basa sulle relazioni.
- **Prolog (Programming in Logic):** È il linguaggio di programmazione logica più celebre, basato su un sottoinsieme della logica del primo ordine (le *Clausole di Horn*).

Sintassi base di Prolog:

- **Atomi:** Costanti, nomi, o simboli. Iniziano rigorosamente con una lettera ****minuscola**** (es. `john`, `rochester`, `a`).
- **Variabili:** Iniziano rigorosamente con una lettera ****maiuscola**** o con un underscore (es. `X`, `Person`, `_`). L'underscore singolo `_` è la variabile anonima (corrisponde alla wildcard in ML).
- **Fatti (Facts):** Asserzioni incondizionate considerate sempre vere nel nostro "mondo" o database. Es: `padre(john, mary)`.
- **Regole (Rules):** Asserzioni condizionate (implicazioni logiche). Hanno la forma `Testa :- Corpo`. Il simbolo `:-` si legge "se" (if). Es: `nonno(X, Z) :- padre(X, Y), padre(Y, Z)`. La virgola `,` rappresenta l'AND logico, mentre il punto e virgola `;` rappresenta l'OR.
- **Query / Goal:** Interrogazioni poste al sistema (precedute dal prompt `?-`). Prolog risponde **true** (o **yes**) se la query è deducibile, **false** (o **no**) altrimenti, oppure restituisce le associazioni (binding) delle variabili che rendono vera la query.

Unificazione e Risoluzione:

- L'**Unificazione** è il processo mediante il quale Prolog cerca di rendere identici due termini assegnando valori alle variabili. Ad esempio, per unificare `padre(john, X)` e `padre(Y, mary)`, Prolog istanzia `Y = john` e `X = mary`.
- Il motore di **Risoluzione** esplora il database dall'alto verso il basso (top-down) e risolve le regole da sinistra verso destra (left-to-right).
- **Backtracking (Ritorsione):** Se Prolog intraprende un percorso di ricerca che fallisce (arriva a un punto cieco in cui non può unificare), "torna indietro" all'ultimo punto in cui c'era un'alternativa (choice point) e tenta una strada diversa.

Operatori Speciali:

- **Is (Aritmetica):** In Prolog, `X = 3 + 4` non calcola 7, ma unifica la variabile `X` con la struttura sintattica `3 + 4`. Per forzare la valutazione aritmetica si usa l'operatore `is`: `X is 3 + 4`.
- **Cut (!) (Taglio):** È un operatore extra-logico che blocca il backtracking. Consente di ottimizzare la ricerca ("potatura dell'albero") impedendo a Prolog di esplorare alternative che sappiamo essere inutili o errate.

- **Negation as Failure ($\backslash +$ o **not**):** In Prolog, se un fatto non può essere dimostrato, viene considerato falso (Assunzione del Mondo Chiuso). **not(P)** è vero se e solo se l'esecuzione di P fallisce.

Liste in Prolog: Strutturalmente identiche a quelle in ML. Si indicano con le parentesi quadre []. La sintassi per destrutturare una lista in Testa e Coda usa il pipe |: [Head | Tail].

20.2 Implementazione del codice

20.2.1 Liste, Ricorsione e Backtracking (append e member)

A differenza dei linguaggi funzionali in cui le funzioni hanno input e output definiti, in Prolog una singola definizione relazionale può operare in molteplici direzioni (invertibilità).

```
% Definizione della relazione "Membro di una lista"
% 1. Caso base: X è membro della lista se è la testa della lista.
member(X, [X | _]).

% 2. Passo ricorsivo: X è membro se è membro della coda della lista.
member(X, [_ | Tail]) :- member(X, Tail).

% Definizione dell'Append (concatenazione di liste)
% append(L1, L2, L3) è vero se L3 è la concatenazione di L1 e L2.
append([], L, L).
append([H | T1], L2, [H | T3]) :- append(T1, L2, T3).

% POTENZA DICHIARATIVA:
% Interrogazione standard: ?- append([a,b], [c,d], X).
%   -> X = [a,b,c,d]
% Sottrazione di liste:    ?- append(X, [c,d], [a,b,c,d]).
%   -> X = [a,b]
% Generazione di pattern: ?- append(X, Y, [a,b,c]).
%   -> X=[], Y=[a,b,c] ; X=[a], Y=[b,c] ; ...
```


20.2.2 Tic-Tac-Toe in Prolog (Tris Interattivo)

Il file `tic-tac-toe-interactive.pl` fornito a lezione illustra come unire le regole dichiarative per la vittoria, il backtracking per la scelta della mossa e l'I/O (Input/Output).

```
% 1. Dichiariamo che i predicati 'o' (computer) e 'x' (giocatore)
% sono dinamici (il database può essere modificato a runtime)
:- dynamic o/1.
:- dynamic x/1.

% 2. Definizione delle condizioni di vittoria (Linee vincenti)
ordered_line(1, 2, 3).
ordered_line(4, 5, 6).
ordered_line(7, 8, 9).
ordered_line(1, 4, 7).
% ... altre combinazioni diagonali ...

% Permutazioni per controllare la linea in qualsiasi ordine
line(A, B, C) :- ordered_line(A, B, C).
line(A, B, C) :- ordered_line(A, C, B).
% ... (altre permutazioni delegate dal Cut o dal Backtracking)

% 3. Logica di intelligenza artificiale: "Cosa è una buona mossa?"
% Prolog tenta le regole in ordine (dall'alto verso il basso).
move(A) :- good(A), empty(A).

% Condizione 1: Cerca di vincere
good(A) :- win(A).
% Condizione 2: Blocca la vittoria del giocatore
good(A) :- block_win(A).
% Condizioni fallback: occupa il centro, poi gli angoli
good(5).
good(1).
good(3).

% Implementazione di 'win' e 'block_win'
% "Vinciamo se abbiamo due 'o' su una linea e A è la terza casella vuota"
win(A) :- o(B), o(C), line(A, B, C).
% "Blocchiamo se l'utente 'x' ha due caselle su una linea"
block_win(A) :- x(B), x(C), line(A, B, C).

% 4. Gestione dello stato (Effetti collaterali con asserta)
makemove :- move(X), !, asserta(x(X)). % Il Cut (!) ferma la ricerca
                                         % appena trova una mossa valida
makemove :- all_full.                  % Fallback in caso di pareggio
```

Commento Architetture per l'esame: L'uso di `:- dynamic` abbinato al comando `asserta` (che inserisce dinamicamente un nuovo fatto nel database all'inizio) e `retract` (che lo elimina) permette a Prolog di mantenere uno "stato" mutevole, aggirando la pura immutabilità funzionale. Inoltre, il cut `!` in `makemove` è fondamentale: senza di esso, se la mossa successiva portasse a un fallimento o al backtracking, Prolog annullerebbe l'inserimento della mossa per provare il `good` successivo, generando comportamenti indesiderati nell'albero di gioco.

21 Domande Generali Teoria

- **Domanda 1**

Testo: La ricorsione in coda:

- A) Non è implementabile nei linguaggi imperativi
- B) Nessuna delle altre risposte
- C) Richiede di non scrivere mai la chiamata ricorsiva come ultimo statement di una subroutine
- D) Richiede di non ritornare mai direttamente il valore ritornato da una chiamata ricorsiva
- E) Permette di risolvere il problema della ricorsione infinita

Risposta esatta: B - Nessuna delle altre risposte

Spiegazione: La ricorsione in coda (tail recursion) avviene esattamente quando la chiamata ricorsiva è l'*ultima* operazione eseguita dalla funzione (il che rende false C e D). È implementabile nei linguaggi imperativi (falsa A) e il suo scopo non è risolvere la ricorsione infinita (falsa E), ma piuttosto ottimizzare l'uso della memoria evitando la crescita dello Stack (Stack Overflow), riutilizzando lo stesso Record di Attivazione. Dato che questa opzione corretta non è presente, la risposta è "Nessuna delle altre".

- **Domanda 2**

Testo: Call by constant:

- A) Fa una copia della variabile
- B) È utile per passare oggetti
- C) Ritorna il valore al chiamante
- D) Permette la modifica della variabile
- E) Nessuna delle altre risposte

Risposta esatta: E - Nessuna delle altre risposte

Spiegazione: Il passaggio per costante non fa una copia (quello è il call by value, falsa A), non è un meccanismo di ritorno (falsa C) e, per definizione, impedisce la modifica della variabile all'interno della funzione (falsa D). Sebbene in alcuni linguaggi (come C++) i reference costanti siano usati per passare oggetti grandi in modo efficiente, la definizione teorica del meccanismo non è "utile per passare oggetti". Pertanto, nessuna delle affermazioni fornite lo descrive correttamente.

- **Domanda 3**

Testo: I record di attivazione:

- A) Sono necessari solo in presenza di funzioni di ordine superiore
- B) Sono allocati dinamicamente solo in caso di scope dinamico
- C) Nessuna delle altre risposte
- D) Devono essere esplicitamente allocati e deallocati dal codice del programma che li usa
- E) Sono allocati solo nello heap

Risposta esatta: C - Nessuna delle altre risposte

Spiegazione: I Record di Attivazione (RdA) servono per supportare l'invocazione di procedure in quasi tutti i linguaggi, indipendentemente dalle funzioni di ordine superiore (falsa A) o dallo scope dinamico (falsa B). L'allocazione è automatica e gestita dalla macchina astratta (falsa D). Solitamente vengono allocati sullo Stack, tranne in alcuni casi specifici come le chiusure o i thread, dove vanno nello Heap (falsa E).

- **Domanda 4**

Testo: Si può dire che una macchina astratta che capisce il linguaggio C non sia implementata in modo puramente compilativo perché:

- A) Gli eseguibili generati da un compilatore C in genere non eseguono direttamente sulla macchina hardware, ma su una macchina astratta che include il Runtime del linguaggio e le funzionalità del Sistema Operativo
- B) Gli eseguibili generati dal compilatore vengono comunque interpretati da una macchina virtuale
- C) Una macchina astratta che capisca un linguaggio di alto livello come il C non è mai implementabile con un compilatore
- D) Il Runtime del linguaggio C è comunque sempre interpretato
- E) Nessuna delle altre risposte

Risposta esatta: A - Gli eseguibili generati da un compilatore C in genere non eseguono direttamente sulla macchina hardware...

Spiegazione: Anche se il C produce codice macchina "nativo", tale codice si appoggia sempre a librerie di Runtime (es. per allocare memoria o per l'I/O) e ai servizi del Sistema Operativo. Pertanto, l'ambiente di esecuzione finale è una macchina astratta mista e non solo l'hardware nudo. Le altre opzioni sono errate perché il C non usa una macchina virtuale interprete in senso stretto come la JVM di Java (false B e D), e un compilatore è assolutamente implementabile per il C (falsa C).

- **Domanda 5**

Testo: Il costrutto `for` dei linguaggi C, C++ e Java:

- A) Non è un costrutto di iterazione determinata
- B) È necessario a tali linguaggi per implementare qualsiasi tipo di algoritmo
- C) Nessuna delle altre risposte
- D) È un costrutto di iterazione determinata
- E) Permette di sapere in anticipo quante volte il ciclo verrà ripetuto (indipendentemente dal corpo del ciclo)

Risposta esatta: A - Non è un costrutto di iterazione determinata

Spiegazione: A differenza di linguaggi storici come il Pascal, il `for` in C/C++/Java permette di modificare la variabile di controllo (l'indice) all'interno del corpo del ciclo. Per questo motivo, a tempo di compilazione non è possibile determinare a priori il numero esatto di iterazioni (rendendo false D ed E). La B è falsa perché un ciclo `while` (o la ricorsione) rendono il linguaggio già Turing-completo senza bisogno del `for`.

- **Domanda 6**

Testo: Un'entità esprimibile è:

- A) Nessuna delle altre risposte
- B) Un'entità che può essere generata come risultato da un'espressione complessa o da una funzione
- C) Un'entità che ancora non compare nell'ambiente
- D) Un'entità che può essere memorizzata
- E) Una generica entità a cui può essere dato un nome

Risposta esatta: B - Un'entità che può essere generata come risultato da un'espressione complessa o da una funzione

Spiegazione: Per definizione teorica, i valori "esprimibili" (o valutabili) sono quelli che costituiscono il risultato della valutazione di espressioni. Ciò li distingue dai valori "memorizzabili" (che possono stare in una variabile, opzione D) e "denotabili" (a cui si può associare un nome, opzione E).

- **Domanda 7**

Testo: In assenza di ambiente non locale:

- A) Per implementare funzioni definite ricorsivamente è necessario utilizzare un `fixed point combinator`
- B) Non si possono implementare algoritmi ricorsivi
- C) Nessuna delle altre risposte
- D) Non si possono implementare algoritmi iterativi o ricorsivi
- E) Non si possono implementare algoritmi iterativi

Risposta esatta: C - Nessuna delle altre risposte

Spiegazione: Sebbene la mancanza di un ambiente non locale renda impossibile fare riferimento a variabili globali o esterne dirette, è sempre possibile aggirare il problema passando tutti i parametri necessari esplicitamente per argomento. Inoltre, la ricorsione è garantita dalla presenza del nome della funzione nell'ambiente, non dalle variabili non locali. Pertanto, tutte le affermazioni che indicano l'impossibilità di fare ricorsione o iterazione sono false (A, B, D, E).

• **Domanda 8**

Testo: I puntatori di catena dinamica (Dynamic Link) contenuti in un record di attivazione:

- A) Collegano una lista di zone di memoria gestita dinamicamente
- B) Servono per accedere alle variabili dinamiche
- C) Non esistono "puntatori di catena dinamica" in un record di attivazione
- D) Nessuna delle altre risposte
- E) Permettono, a partire da un RdA, di trovare il RdA precedente sullo stack

Risposta esatta: E - Permettono, a partire da un RdA, di trovare il RdA precedente sullo stack

Spiegazione: Il Dynamic Link è il puntatore che salva lo stato (la base) del record di attivazione del chiamante. Serve esclusivamente a ripristinare correttamente l'ambiente della funzione precedente quando la funzione corrente termina. Non servono ad accedere all'heap (memoria gestita dinamicamente, false A e B).

• **Domanda 9**

Testo: In presenza di variabili modificabili:

- A) Nessuna delle altre risposte
- B) Esistono un Ambiente che associa valori denotabili (fra cui le locazioni di memoria) a nomi ed una Memoria che associa locazioni di memoria a valori memorizzabili
- C) Non esistono valori denotabili
- D) La valutazione del comando di assegnamento restituisce sempre un valore
- E) Il comando di assegnamento non ha effetti collaterali

Risposta esatta: B - Esistono un Ambiente che associa valori denotabili... ed una Memoria che associa locazioni...

Spiegazione: Per modellare formalmente le variabili modificabili nei linguaggi imperativi serve una struttura a "due stadi": l'Ambiente mappa gli identificatori (nomi) ad un indirizzo fisico (locazione, l-value), mentre la Memoria mappa quella locazione al suo effettivo contenuto (r-value). Le altre opzioni sono false: in questo paradigma i valori denotabili esistono eccome (C è falsa) e l'assegnamento esiste *proprio* per avere l'effetto collaterale di cambiare lo stato della memoria (E è falsa).

- **Domanda 10**

Testo: Il concetto di variabile modificabile:

- A) È l'unico concetto utilizzabile quando si parla di variabili
- B) È imposto dall'architettura di Von Neumann (variabili non modificabili richiederebbero macchine astratte caratterizzate da memoria a sola lettura)
- C) Nessuna delle altre risposte
- D) È tipico del paradigma di programmazione imperativo
- E) Permette di evitare il fenomeno dell'aliasing

Risposta esatta: D - È tipico del paradigma di programmazione imperativo

Spiegazione: Il paradigma imperativo si basa sul concetto di stato computazionale che evolve nel tempo, per cui la variabile modificabile è il concetto centrale. Nei paradigmi funzionali e logici si usano variabili immutabili (il che rende falsa la A). Non previene l'aliasing, anzi ne è soggetto qualora più nomi puntino alla stessa locazione modificabile (falsa E). Non è "imposto" dall'architettura nuda (si possono creare macchine astratte funzionali sopra Von Neumann), ma ne è una naturale e diretta astrazione (falsa B).

- **Domanda 11**

Testo: Nella sostituzione $(\lambda a.abc)[d/c]$:

- A) È necessario applicare una Alfa-equivalenza per evitare una cattura di variabile
- B) Viene catturata la variabile c
- C) Si rischia di catturare la variabile " a " ed è necessario applicare Beta-equivalenza per risolvere il problema
- D) Non c'è alcuna cattura di variabile
- E) Nessuna delle altre risposte

Risposta esatta: D - Non c'è alcuna cattura di variabile

Spiegazione: Eseguendo la sostituzione, la variabile libera c viene rimpiazzata con d . Il risultato è $\lambda a.abd$. La variabile d appena inserita non cade sotto lo "scope" di alcun legatore λ associato a d (l'unico legatore presente è λa), quindi non subisce alcuna cattura non voluta e non è necessaria alcuna α -conversione.

- **Domanda 12**

Testo: Il fenomeno della cattura di variabili:

- A) Nessuna delle altre risposte
- B) Non può essere evitato in alcun modo
- C) Fa sì che dopo una sostituzione una variabile libera diventi legata (per esempio da un'astrazione $\lambda x.$)
- D) Comporta la "sparizione" di variabili libere durante un'astrazione funzione
- E) È dovuto all'assenza di un ambiente non locale

Risposta esatta: C - Fa sì che dopo una sostituzione una variabile libera diventi legata...

Spiegazione: Nel lambda calcolo, la cattura di variabile si verifica quando, sostituendo un termine all'interno di un'espressione, una variabile che originariamente era libera finisce accidentalmente nello scope di un parametro formale (un legatore λ) con lo stesso nome. Questo altera profondamente la logica dell'espressione. Per evitarlo (rendendo falsa la B), si procede rinominando preventivamente la variabile legata tramite α -conversione.

- **Domanda 13**

Testo: Una Macchina Astratta ML (LO) è:

- A) È un modo per descrivere un interprete
- B) Nessuna delle altre risposte
- C) È implementabile solo basandosi sull'architettura di Von Neumann
- D) È un modo per descrivere un compilatore
- E) È associata ad un proprio linguaggio macchina L, che è in grado di capire ed eseguire

Risposta esatta: E - È associata ad un proprio linguaggio macchina L, che è in grado di capire ed eseguire

Spiegazione: Una macchina astratta è definita teoricamente come una collezione di strutture dati e algoritmi capace di memorizzare ed eseguire programmi. Per definizione intrinseca, ogni macchina astratta è caratterizzata da (e associata a) un proprio linguaggio macchina che essa "comprende" e interpreta direttamente, fungendo da esecutore.

- **Domanda 14**

Testo: Il passaggio di parametri per nome:

- A) Permette di passare valori solo dal chiamante al chiamato (e non viceversa)
- B) Nessuna delle altre risposte
- C) È implementabile passando una chiusura come parametro
- D) Ha un valore solo teorico e non è implementabile in pratica
- E) Permette la cattura di variabili libere in modo da effetti non deterministici

Risposta esatta: C - È implementabile passando una chiusura come parametro

Spiegazione: Nel passaggio per nome (call by name), l'argomento non viene valutato al momento della chiamata, ma ogni volta che viene menzionato nel corpo della funzione. A livello implementativo, questo risultato si ottiene passando in modo invisibile una chiusura priva di argomenti, nota come *thunk*, che contiene l'espressione da calcolare e l'ambiente in cui valutarla.

- **Domanda 15**

Testo: La tecnica del display:

- A) Nessuna delle altre risposte
- B) Permette di implementare facilmente lo scope dinamico
- C) Permette di ridurre il costo derivante dalla scansione della catena statica quando si implementa lo scope statico
- D) Permette di visualizzare le zone di memoria allocata dinamicamente
- E) Permette di implementare le regole di scope statico senza generare frammentazione della memoria

Risposta esatta: C - Permette di ridurre il costo derivante dalla scansione della catena statica...

Spiegazione: Con lo scoping statico, per accedere a una variabile non-locale molto "profonda", bisognerebbe attraversare sequenzialmente i puntatori di catena statica ($O(n)$). Il display è un array mantenuto a runtime che punta direttamente ai Record di Attivazione attivi per ogni livello lessicale corrente, portando il tempo di accesso alle variabili non-locali ad essere $O(1)$, riducendone così drasticamente il costo computazionale.

- **Domanda 16**

Testo: Un compilatore da un linguaggio L ad un linguaggio LO è:

- A) Un programma che trasforma un programma PL (espresso nel linguaggio L) in un programma PLO (espresso nel linguaggio LO) tale che per ogni input I si ha $PL(I) = PLO(I)$
- B) Nessuna delle altre risposte
- C) Un programma scritto nel linguaggio LO che riceve come ingresso un programma PL (espresso nel linguaggio L) ed il suo input I generando lo stesso output che genera PL con input I
- D) L'implementazione di una macchina astratta scritta nel linguaggio LO, che capisce programmi scritti nel linguaggio L
- E) Una implementazione di macchine astratte indipendente dalla macchina fisica

Risposta esatta: A - Un programma che trasforma un programma PL (espresso nel linguaggio L) in un programma PLO (espresso nel linguaggio LO)...

Spiegazione: Questa è la definizione formale di compilatore: è un semplice traduttore. Prende il codice nel linguaggio sorgente L e lo "traduce" integralmente in un linguaggio target LO, garantendo che i due programmi mantengano un'assoluta equivalenza semantica (a parità di input, producono lo stesso output). L'opzione C, invece, è la definizione formale di un interprete.

- **Domanda 17**

Testo: Il costrutto `for` dei linguaggi C, C++ e Java non è un costrutto di iterazione determinata perché:

- A) L'esistenza di costrutti di iterazione derminata implicherebbe che C, C++ e Java non sono Turing-completi
- B) Nessuna delle altre risposte
- C) Dall'interno del ciclo è possibile modificare il valore del contatore
- D) Non esistono costrutti di iterazione determinata
- E) C, C++ e Java sono linguaggi imperativi

Risposta esatta: C - Dall'interno del ciclo è possibile modificare il valore del contatore

Spiegazione: Si definisce *iterazione determinata* un ciclo di cui si può prevedere in modo matematico il numero di iterazioni totali prima ancora che venga eseguita la prima (es. i cicli `for` storici del Pascal). Dal momento che in C, C++ e Java è lecito alterare il contatore del ciclo nel corpo, questo principio viene violato, rendendoli cicli "mascherati" a condizione generale, come dei `while`.

- **Domanda 18**

Testo: Un oggetto denotabile (intendendo per "oggetto" una generica entità che può essere una variabile, una funzione, etc....) è:

- A) Un "oggetto" che può essere memorizzato in una variabile
- B) Un "oggetto" per cui compare un binding nell'ambiente
- C) Nessuna delle altre risposte
- D) Un "oggetto" che ancora non compare nell'ambiente
- E) Un "oggetto" che può essere generato come risultato da un'espressione complessa o da una funzione

Risposta esatta: B - Un "oggetto" per cui compare un binding nell'ambiente

Spiegazione: Le entità *denotabili* sono formalmente tutti quegli elementi di un programma a cui è possibile associare un "nome" (un identificatore). Pertanto, un oggetto diventa denotabile nel momento in cui figura all'interno dell'Environment (Ambiente), che per sua natura è l'insieme strutturato dei binding fra identificatori e tali entità.

- **Domanda 19**

Testo: L'ambiente non locale di un blocco di codice è:

- A) Nessuna delle altre risposte
- B) L'insieme dei valori che le variabili non locali possono assumere
- C) L'insieme dei binding creati all'interno del blocco di codice
- D) L'insieme dei binding visibili dentro al blocco, ma non direttamente definiti in esso
- E) Il subset dell'ambiente non visibile dentro al blocco di codice

Risposta esatta: D - L'insieme dei binding visibili dentro al blocco, ma non direttamente definiti in esso

Spiegazione: Un blocco vede il proprio "ambiente locale" (i binding dichiarati internamente) più un "ambiente non-locale". Quest'ultimo raccoglie le dichiarazioni effettuate all'esterno (variabili globali o variabili dei blocchi padri avvolgenti) a cui il blocco in esecuzione ha diritto di accesso pur non avendole istanziate localmente.

- **Domanda 20**

Testo: I puntatori di catena dinamica contenuti in un record di attivazione:

- A) Nessuna delle altre risposte
- B) Non esistono "puntatori di catena dinamica" in un record di attivazione
- C) Servono per identificare la zona di memoria in cui è memorizzata una variabile locale
- D) Devono essere esplicitamente allocati e deallocati dal codice del programma che li usa
- E) Servono per accedere alle variabili dinamiche

Risposta esatta: A - Nessuna delle altre risposte

Spiegazione: Il compito del puntatore di catena dinamica (Dynamic Link) non è gestire le variabili (rendendo false C ed E), né necessita di gestione esplicita dal programmatore (falsa D). Il suo unico ruolo è puntare alla base del record di attivazione *del chiamante* per ripristinare il corretto stack frame alla fine dell'invocazione. Essendo questa corretta definizione assente, l'unica opzione giusta è "Nessuna delle altre risposte".

- **Domanda 21**

Testo: L'allocazione dinamica della memoria:

- A) Nessuna delle altre risposte
- B) È sempre effettuata solo dal compilatore o dall'interprete
- C) Può essere fatta solo dallo heap
- D) Può essere fatta solo dallo stack
- E) Può essere fatta sia dallo stack che dallo heap

Risposta esatta: E - Può essere fatta sia dallo stack che dallo heap

Spiegazione: L'allocazione dinamica della memoria avviene durante l'esecuzione del programma. Questa può avvenire sullo Stack (es. i Record di Attivazione che vengono allocati e deallocati ad ogni chiamata di funzione, o costrutti come `alloca` in C) oppure sullo Heap (es. oggetti allocati esplicitamente dal programmatore con `malloc` in C o `new` in Java). Le opzioni C e D sono quindi limitanti e false.

- **Domanda 22**

Testo: Un compilatore da un linguaggio L ad un linguaggio LO è:

- A) Nessuna delle altre risposte
- B) Una implementazione di macchine astratte indipendente dalla macchina fisica
- C) Un programma che trasforma un programma PLO (espresso nel linguaggio LO) in un programma PL (espresso nel linguaggio L) tale che per ogni input I si ha $PL(I) = PLO(I)$
- D) L'implementazione di una macchina astratta scritta nel linguaggio LO, che capisce programmi scritti nel linguaggio L
- E) Un programma scritto nel linguaggio LO che riceve come ingresso un programma PL (espresso nel linguaggio L) ed il suo input I generando lo stesso output che genera PL con input I

Risposta esatta: A - Nessuna delle altre risposte

Spiegazione: Leggendo attentamente l'opzione C, essa descrive la traduzione al contrario (dal linguaggio macchina LO al linguaggio sorgente L, che sarebbe semmai un de-compilatore). Le opzioni D ed E definiscono concettualmente un interprete, non un compilatore. Pertanto, mancando la definizione corretta (traduzione da PL a PLO), la risposta esatta è "Nessuna delle altre".

- **Domanda 23**

Testo: I dangling pointer:

- A) Sono identificabili tramite tecniche di reference counting (contatore dei riferimenti)
- B) Sono un problema solo per il linguaggio Java
- C) Sono identificabili tramite la tecnica detta "mark and sweep"
- D) Non possono essere identificati con certezza, ma questo non è un problema perché comportano solo un piccolo spreco di memoria
- E) Nessuna delle altre risposte

Risposta esatta: E - Nessuna delle altre risposte

Spiegazione: I dangling pointer (puntatori pendenti) si verificano quando un'area di memoria viene deallocata, ma rimangono dei puntatori che fanno ancora riferimento ad essa. Non causano un "piccolo spreco", ma gravi errori di sicurezza (falsa D). In Java non esistono grazie al Garbage Collector (falsa B). Il *Reference Counting* e il *Mark and Sweep* (false A e C) servono a trovare la "spazzatura" (memoria non più puntata), non i dangling pointer. Per questi ultimi servono tecniche diverse (es. tombstones o locks-and-keys).

- **Domanda 24**

Testo: La frammentazione esterna causa:

- A) Uno spreco di memoria
- B) Nessuna delle altre risposte
- C) L'impossibilità di allocare grandi blocchi di memoria anche se la memoria libera totale è sufficiente
- D) Un rallentamento rilevante nelle operazioni di allocazione della memoria
- E) Il funzionamento non corretto di programmi che allocano memoria dinamicamente

Risposta esatta: C - L'impossibilità di allocare grandi blocchi di memoria anche se la memoria libera totale è sufficiente

Spiegazione: La frammentazione esterna si verifica quando la memoria libera sullo Heap è "spezzettata" in tanti piccoli frammenti isolati tra blocchi allocati. Pur essendoci un quantitativo totale di memoria libera sufficiente per soddisfare una nuova richiesta, se quest'ultima esige un blocco contiguo molto grande, l'allocatore fallirà. L'opzione A (spreco) è più tipica della frammentazione interna.

- **Domanda 25**

Testo: La valutazione con corto circuito del predicato "A && B" (dove "&&" rappresenta un "AND" logico):

- A) Stabilisce che se "B" è vero allora "A" non viene valutato
- B) Nessuna delle altre risposte
- C) Stabilisce che se "A" è falso allora "B" non viene valutato
- D) Crea un non-determinismo nell'ordine della valutazione di "A" e "B"
- E) Stabilisce che "A" e "B" devono essere valutati in parallelo

Risposta esatta: C - Stabilisce che se "A" è falso allora "B" non viene valutato

Spiegazione: Nella valutazione cortocircuitata di espressioni logiche, l'analisi avviene da sinistra a destra. Nel caso dell'operatore AND, affinché l'intera espressione sia vera, entrambi gli operandi devono essere veri. Se il primo (A) risulta falso, l'esito finale è matematicamente garantito essere falso, per cui l'esecuzione salta completamente la valutazione di (B) (rendendo falsa la A e la E), risparmiando tempo ed evitando potenziali errori.

- **Domanda 26**

Testo: Se l'ambiente di una funzione non contiene il nome della funzione stessa:

- A) Nessuna delle altre risposte
- B) Non è possibile per la funzione invocarsi ricorsivamente
- C) Non ci sono particolari conseguenze
- D) La funzione non può usare scope dinamico
- E) La funzione non può usare scope statico

Risposta esatta: B - Non è possibile per la funzione invocarsi ricorsivamente

Spiegazione: Affinché una funzione possa compiere una chiamata ricorsiva, deve poter riferirsi a sé stessa tramite un identificatore valido. Se il nome della funzione non figura nel suo ambiente (o negli ambienti più esterni ad essa accessibili), il meccanismo di risoluzione dei nomi fallirà, impedendo la ricorsione.

- **Domanda 27**

Testo: La frammentazione interna causa:

- A) Il funzionamento non corretto di programmi che allocano memoria dinamicamente
- B) Nessuna delle altre risposte
- C) Un rallentamento rilevante nelle operazioni di allocazione della memoria
- D) Uno spreco di memoria
- E) L'impossibilità di allocare grandi blocchi di memoria anche se la memoria libera totale è sufficiente

Risposta esatta: D - Uno spreco di memoria

Spiegazione: La frammentazione interna avviene quando il sistema assegna un blocco di memoria di dimensioni fisse (es. per motivi di allineamento) leggermente più grande di quanto effettivamente richiesto dal programma. Lo spazio eccedente all'interno del blocco assegnato non può essere utilizzato da nessun altro, traducendosi in puro spreco. L'opzione E, come visto in precedenza, descrive invece la frammentazione esterna.

- **Domanda 28**

Testo: L'allocazione dinamica della memoria:

- A) Può essere fatta sia dallo stack che dallo heap
- B) Nessuna delle altre risposte
- C) È sempre effettuata solo dal compilatore o dall'interprete
- D) Può essere fatta solo dallo stack
- E) Può essere fatta solo dallo heap

Risposta esatta: A - Può essere fatta sia dallo stack che dallo heap

Spiegazione: Questa è una riformulazione esatta della Domanda 21 (vedi sopra). L'allocazione è considerata dinamica (eseguita a runtime) sia quando si accatastano i Record di Attivazione sullo Stack (allocazione LIFO), sia quando si riservano blocchi arbitrari sullo Heap.

- **Domanda 29**

Testo: L'ambiente (o environment) è:

- A) L'insieme delle associazioni (nome, entità denotabile) esistenti in uno specifico punto del programma ed in uno specifico momento durante l'esecuzione di un programma
- B) L'insieme dei valori che una variabile assume durante l'esecuzione di un programma
- C) Un insieme di associazioni (nome, valore) definite staticamente durante lo sviluppo di un programma
- D) Nessuna delle altre risposte
- E) Una lista di coppie (nome, tipo) che permette di accedere alle variabili di un programma

Risposta esatta: A - L'insieme delle associazioni (nome, entità denotabile) esistenti in uno specifico...

Spiegazione: Formalmente, nei linguaggi di programmazione, l'Ambiente è la struttura che si occupa di mappare l'identificatore (il nome stabilito dal programmatore) all'entità semantica che rappresenta (detta entità denotabile). Non mappa mai i nomi direttamente ai "valori" (questo è compito della Memoria, rendendo false B e C), ma piuttosto a locazioni di memoria, funzioni o tipi.

- **Domanda 30**

Testo: Si può dire che una macchina astratta che capisce il linguaggio Java non sia implementata in modo puramente compilativo perché:

- A) Non esistendo un vero e proprio runtime per Java, non si può parlare di compilazione pura
- B) La macchina virtuale di Java (JVM) deve comunque essere compilata
- C) Una macchina astratta che capisca un linguaggio di alto livello come Java non è mai implementabile con un compilatore
- D) Non esistono compilatori Java
- E) Nessuna delle altre risposte

Risposta esatta: E - Nessuna delle altre risposte

Spiegazione: Tutte le opzioni proposte affermano il falso: per Java esiste eccome un runtime environment (JRE/JVM) e possiede fior di compilatori (come `javac`), rendendo false A e D. Inoltre, teoricamente ogni linguaggio di alto livello può essere compilato (falsa C). Il vero motivo per cui Java non è puramente compilativo è l'approccio misto: il compilatore genera un codice intermedio (Bytecode) e non codice macchina nativo, il quale viene poi letto e interpretato/compilato just-in-time dalla JVM a runtime.

• **Domanda 31**

Testo: La ricorsione in coda:

- A) Permette di risolvere il problema della ricorsione infinita
- B) Richiede di non scrivere mai la chiamata ricorsiva come ultimo statement di una subroutine
- C) Richiede di non ritornare mai direttamente il valore ritornato da una chiamata ricorsiva
- D) Nessuna delle altre risposte
- E) Permette di evitare un'eccessiva crescita della dimensione dello stack

Risposta esatta: E - Permette di evitare un'eccessiva crescita della dimensione dello stack

Spiegazione: Se la chiamata ricorsiva è rigorosamente l'ultima operazione eseguita nel corpo di una funzione (ricorsione in coda o *tail recursion*), il compilatore o l'interprete può ottimizzarla riutilizzando il Record di Attivazione corrente invece di crearne uno nuovo. Questo previene lo Stack Overflow. Non previene, però, un loop infinito causato da una condizione di base errata (rendendo falsa la A). Le opzioni B e C affermano l'esatto contrario della definizione di ricorsione in coda.

• **Domanda 32**

Testo: La memoria gestita staticamente:

- A) È allocata esplicitamente dal programma a tempo di esecuzione, ma una volta allocata è staticamente legata al programma e non può essere liberata fino alla sua terminazione
- B) È allocata prima dell'esecuzione del programma. Le entità allocate staticamente possono essere deallocate durante l'esecuzione del programma, per liberare memoria
- C) È allocata dal compilatore prima dell'esecuzione del programma. Le entità allocate staticamente in memoria risiedono in una zona fissa di memoria durante tutta l'esecuzione del programma
- D) È una memoria a sola lettura
- E) Nessuna delle altre risposte

Risposta esatta: C - È allocata dal compilatore prima dell'esecuzione del programma...

Spiegazione: L'allocazione statica viene decisa e preparata dal compilatore ancor prima che il programma inizi l'esecuzione. Questa memoria (come quella per le variabili globali) occupa una dimensione fissa e permane nello stesso blocco per l'intero ciclo di vita del processo. Non può essere deallocata a piacimento a runtime (il che rende false la A e la B) e non è necessariamente a sola lettura (rendendo falsa la D).

- **Domanda 33**

Testo: Un interprete di un linguaggio L scritto in un linguaggio LO è:

- A) Nessuna delle altre risposte
- B) L'implementazione di una macchina astratta scritta nel linguaggio LO, che capisce programmi scritti nel linguaggio L
- C) Una implementazione di macchine astratte indipendente dalla macchina fisica
- D) Un programma scritto nel linguaggio LO che riceve come ingresso un programma PL (espresso nel linguaggio L) ed il suo input I generando lo stesso output che genera PL con input I
- D) Un programma che trasforma un programma PL (espresso nel linguaggio L) in un programma PLO (espresso nel linguaggio LO) tale che per ogni input I si ha $PL(I) = PLO(I)$

Risposta esatta: D - Un programma scritto nel linguaggio LO che riceve come ingresso un programma PL...

Spiegazione: Questa è la definizione *operazionale* di un interprete. A differenza di un compilatore (descritto nell'opzione E) che converte il codice e termina, l'interprete prende il codice sorgente (PL) e i dati di input (I) simultaneamente, e genera direttamente l'output valutando istruzione per istruzione a tempo di esecuzione.

- **Domanda 34**

Testo: In caso di scope statico:

- A) I legami fra nomi ed oggetto possono essere determinati solo a tempo di esecuzione
- B) Non è possibile annidare più blocchi di istruzioni
- C) Il valore assegnato ad una variabile non può essere modificato
- D) I legami fra nomi ed oggetto possono essere determinati semplicemente leggendo il testo di un programma
- E) Nessuna delle altre risposte

Risposta esatta: D - I legami fra nomi ed oggetto possono essere determinati semplicemente leggendo il testo di un programma

Spiegazione: Lo scope statico (o lessicale) permette di risolvere i binding basandosi unicamente sulla struttura annidata del codice sorgente (il testo scritto), indipendentemente da come o da dove la funzione viene chiamata a runtime. Se i legami si determinassero solo a tempo di esecuzione (opzione A), parleremmo di scope dinamico. Le opzioni B e C non hanno fondamento.

- **Domanda 35**

Testo: Un garbage collector:

- A) Può essere implementato tramite la tecnica detta "mark and sweep", che riesce sempre a identificare tutta la memoria allocata dinamicamente ma non più utilizzata
- B) Richiede un'implementazione complessa, usando la tecnica dei tombstone (pietre tombali)
- C) È implementabile solo in linguaggi di programmazione funzionali
- D) È implementabile tramite la tecnica di lucchetti e chiavi, che però può causare dei memory leak
- E) Nessuna delle altre risposte

Risposta esatta: A - Può essere implementato tramite la tecnica detta "mark and sweep"...

Spiegazione: Il Mark and Sweep è un algoritmo di Garbage Collection che prima "marca" tutti gli oggetti raggiungibili seguendo i riferimenti a partire dai root (stack e variabili globali), e poi "spazza" via (dealloca) l'intero heap rimanente. Questa tecnica risolve anche il problema dei riferimenti circolari (aliasing ciclico). Le "pietre tombali" e i "lucchetti/chiavi" (opzioni B e D) servono invece per prevenire i dangling pointer, non per fare garbage collection.

- **Domanda 36**

Testo: Un'entità denotabile è:

- A) Nessuna delle altre risposte
- B) Un'entità che può essere generata come risultato da un'espressione complessa o da una Funzione
- C) Una generica entità a cui può essere dato un nome
- D) Un'entità che ancora non compare nell'ambiente
- E) Un'entità che può essere memorizzata

Risposta esatta: C - Una generica entità a cui può essere dato un nome

Spiegazione: Come visto in una domanda precedente, nel contesto dei linguaggi di programmazione, l'aggettivo "denotabile" descrive tutto ciò a cui è possibile associare sintatticamente un identificatore (un nome). L'opzione B descrive le entità "esprimibili", mentre la E descrive le entità "memorizzabili".

- **Domanda 37**

Testo: Un interprete di un linguaggio L scritto in un linguaggio LO è:

- A) Un programma che trasforma un programma PL (espresso nel linguaggio L) in un programma PLO (espresso nel linguaggio LO) tale che per ogni input I si ha $PL(I) = PLO(I)$
- B) Nessuna delle altre risposte
- C) Una implementazione di macchine astratte indipendente dalla macchina fisica
- D) Un programma scritto nel linguaggio L che dato un input I produce lo stesso output generato dallo stesso programma scritto nel linguaggio LO
- E) L'implementazione di una macchina astratta scritta nel linguaggio LO, che capisce programmi scritti nel linguaggio L

Risposta esatta: E - L'implementazione di una macchina astratta scritta nel linguaggio LO...

Spiegazione: A differenza della Domanda 33, che dava la definizione di *funzionamento operativo*, qui si fornisce la definizione *strutturale*. Un interprete non è altro che la materializzazione (l'implementazione software) di una macchina astratta che prende vita grazie a un linguaggio ospite (LO), ed è progettata per "capire" direttamente un linguaggio ospitato (L).

- **Domanda 38**

Testo: L'ambiente (o environment) è:

- A) Nessuna delle altre risposte
- B) Un insieme di associazioni (nome, valore) definite staticamente durante lo sviluppo di un programma
- C) L'insieme delle associazioni (variabile, valore) esistenti in uno specifico punto del programma ed in uno specifico momento durante l'esecuzione di un programma
- D) L'insieme dei valori che una variabile assume durante l'esecuzione di un programma
- E) Una lista di coppie (nome, tipo) che permette di accedere alle variabili di un programma

Risposta esatta: A - Nessuna delle altre risposte

Spiegazione: Formalmente, l'Ambiente è l'insieme delle associazioni "NOME -> ENTITÀ DENOTABILE". Le opzioni proposte qui cercano di confondere i concetti: le associazioni dirette tra nome/variabile e "valore" (opzioni B e C) riassumono il concetto più ampio di "Stato" della computazione (Ambiente + Memoria), non il solo Ambiente. L'opzione E è falsa perché mappa nomi a tipi e non denota l'environment semantico.

- **Domanda 39**

Testo: Un garbage collector:

- A) Richiede un'implementazione complessa, usando la tecnica dei tombstone (pietre tombali)
- B) È implementabile solo in linguaggi di programmazione funzionali
- C) Può essere implementato tramite la tecnica del reference counting (contatore dei riferimenti), che riesce sempre ad identificare tutta la memoria allocata dinamicamente ma non più utilizzata
- D) È implementabile tramite la tecnica di lucchetti e chiavi, che però può causare dei memory leak
- E) Nessuna delle altre risposte

Risposta esatta: E - Nessuna delle altre risposte

Spiegazione: Ancora una volta le opzioni mescolano tecniche distinte. Tombstones e Lucchetti/Chiavi (A, D) servono per i dangling pointer, non per la deallocazione dei leak (GC). Il Reference Counting (C) è effettivamente una tecnica di GC, ma la frase asserisce che riesce *sempre* a identificare la memoria inutilizzata: questo è falso, poiché il Reference Counting fallisce clamorosamente di fronte a strutture dati con riferimenti circolari (es. liste doppiamente concatenate chiuse).

- **Domanda 40**

Testo: La memoria gestita dinamicamente:

- A) È usata solo in linguaggi interpretati
- B) Non è mai strettamente necessaria, ma permette di ottenere migliori prestazioni
- C) È necessaria per implementare la ricorsione
- D) Nessuna delle altre risposte
- E) È necessaria per implementare l'iterazione

Risposta esatta: D - Nessuna delle altre risposte

Spiegazione: Se per "memoria gestita dinamicamente" intendiamo l'heap, essa serve per strutture dati senza vincoli di dimensione a compile-time e non è legata all'iterazione o unicamente ai linguaggi interpretati. Se ci riferiamo allo stack dinamico (allocazione dei RdA), è sì necessario per la ricorsione, ma l'espressione ambigua e la mancanza di una formulazione perfettamente accurata fra le opzioni porta alla scelta di "Nessuna delle altre".

- **Domanda 41**

Testo: L'ambiente (o environment) è:

- A) Un insieme di associazioni (nome, valore) definite staticamente durante lo sviluppo di un programma
- B) Una lista di coppie (nome, tipo) che permette di accedere alle variabili di un programma
- C) Nessuna delle altre risposte
- D) L'insieme dei valori che una variabile assume durante l'esecuzione di un programma
- E) L'insieme delle associazioni (nome, oggetto denotabile) esistenti in uno specifico punto del programma ed in uno specifico momento durante l'esecuzione di un programma

Risposta esatta: E - L'insieme delle associazioni (nome, oggetto denotabile) esistenti in uno specifico punto del programma ed in uno specifico momento durante l'esecuzione di un programma

Spiegazione: Questa è la definizione formale ed esatta di Ambiente. A differenza dello "Stato", che mappa direttamente i nomi ai valori finali (inglobando anche la Memoria), l'Ambiente si limita a mappare gli identificatori (nomi) alle entità semantiche a cui si riferiscono (oggetti denotabili, come locazioni di memoria, funzioni o tipi) in quel preciso istante e punto del codice (regole di scope).

- **Domanda 42**

Testo: In caso di scope dinamico:

- A) Il valore assegnato ad una variabile non può essere modificato
- B) Non è possibile annidare più blocchi di istruzioni
- C) Nessuna delle altre risposte
- D) I legami fra nomi ed oggetto possono essere determinati solo a tempo di esecuzione
- E) I legami fra nomi ed oggetto possono essere determinati semplicemente leggendo il testo di un programma

Risposta esatta: D - I legami fra nomi ed oggetto possono essere determinati solo a tempo di esecuzione

Spiegazione: A differenza dello scope statico, in cui il legame di una variabile si evince leggendo la struttura lessicale del codice sorgente (il che rende falsa la E), lo scope dinamico risolve i nomi risalendo a ritroso la catena delle chiamate attive (catena dinamica) sullo Stack. Poiché l'ordine delle chiamate dipende dall'input e dal flusso del programma, la risoluzione può avvenire unicamente a runtime.

- **Domanda 43**

Testo: I puntatori di catena statica contenuti in un record di attivazione:

- A) Nessuna delle altre risposte
- B) Servono per accedere alle variabili statiche
- C) Devono essere esplicitamente allocati e deallocati dal codice del programma che li usa
- D) Servono per identificare la zona di memoria in cui è memorizzata una variabile in caso di scope statico
- E) Non esistono puntatori di catena statica in un record d'attivazione

Risposta esatta: D - Servono per identificare la zona di memoria in cui è memorizzata una variabile in caso di scope statico

Spiegazione: Il puntatore di catena statica (Static Link) viene mantenuto nel Record di Attivazione per puntare al record del blocco o funzione che lo avvolge "lessicalmente" (il padre nel codice sorgente). Seguendo questi puntatori, il programma riesce a trovare e accedere correttamente alle variabili non locali in un ambiente a scope statico.

- **Domanda 44**

Testo: Un'entità denotabile può avere un tempo di vita più lungo di quello delle associazioni (fra l'entità e identificatori) che lo riferiscono:

- A) Nessuna delle altre risposte
- B) Se l'entità è allocata dinamicamente dallo heap in una subroutine
- C) Mai
- D) Se l'entità è allocata dinamicamente dallo stack in una subroutine
- E) Se si usa scope dinamico

Risposta esatta: B - Se l'entità è allocata dinamicamente dallo heap in una subroutine

Spiegazione: Quando si alloca memoria sullo Heap all'interno di una funzione e la si assegna a un puntatore locale (la "variabile/associazione"), al termine della funzione la variabile locale viene distrutta (poppata dallo Stack), terminando così il tempo di vita dell'associazione. Tuttavia, l'entità allocata sullo Heap rimane in vita fino alla deallocazione esplicita (es. **free**) o all'intervento del Garbage Collector. Dunque, l'entità sopravvive all'associazione iniziale.

- **Domanda 45**

Testo: La memoria gestita dinamicamente:

- A) Non è mai strettamente necessaria, ma permette di ottenere migliori prestazioni
- B) È necessaria per implementare l'iterazione
- C) È usata solo in linguaggi interpretati
- D) È necessaria solo per implementare macchine astratte per linguaggi compilati
- E) Nessuna delle altre risposte

Risposta esatta: E - Nessuna delle altre risposte

Spiegazione: Come visto per la domanda 40, la memoria gestita dinamicamente (sia essa lo Stack per i Record di Attivazione, sia l'Heap per le strutture dati) è strettamente necessaria per implementare la ricorsione (il che rende falsa la A), ed è usata sia da linguaggi compilati che interpretati (il che rende false la C e la D). Non è necessaria per implementare la semplice iterazione, che richiede solo salti condizionati (falsa B).

- **Domanda 46**

Testo: I record d'attivazione:

- A) Devono essere esplicitamente allocati e deallocati dal codice del programma che li usa
- B) Possono essere allocati sia sullo stack sia sullo heap (in caso, per esempio, di funzioni di ordine superiore)
- C) Sono necessari solo in presenza di funzioni di ordine superiore
- D) Sono allocati solo sullo heap
- E) Nessuna delle altre risposte

Risposta esatta: B - Possono essere allocati sia sullo stack sia sullo heap (in caso, per esempio, di funzioni di ordine superiore)

Spiegazione: Di norma, i Record di Attivazione vengono allocati e deallocati automaticamente (falsa A) sulla struttura LIFO dello Stack. Tuttavia, nei linguaggi funzionali o con First-Class Functions, una funzione può restituire un'altra funzione e le sue variabili locali (closure) devono sopravvivere alla fine della chiamata originale. In questi casi specifici, il record viene promosso e allocato sullo Heap affinché non venga distrutto.

- **Domanda 47**

Testo: Quale delle seguenti affermazioni riguardanti il garbage collector è vera:

- A) Per Garbage Collection si intende una modalità automatica di gestione della memoria
- B) L'utente alloca liberamente memoria
- C) Non è permesso deallocare memoria
- D) Il sistema periodicamente recupera la memoria allocata e non più utilizzabile
- E) Tutte sono corrette

Risposta esatta: E - Tutte sono corrette

Spiegazione: Nei linguaggi dotati di Garbage Collector (es. Java), la gestione della memoria è delegata al Runtime System (A). Il programmatore è libero di istanziare oggetti con **new** (B), ma in genere il linguaggio impedisce di "forzare" o effettuare una deallocazione manuale per motivi di sicurezza (C). È quindi compito di un demone in background cercare ciclicamente la spazzatura e liberarla per il riutilizzo (D).

- **Domanda 48**

Testo: Quale delle seguenti affermazioni riguardanti il Garbage Collector è falsa:

- A) Il garbage collector annoterà le aree di memoria non più referenziate e le libererà automaticamente
- B) L'utente alloca liberamente memoria
- C) È permesso deallocare memoria
- D) Un sistema operativo, o un compilatore, e un modulo di run-time liberano porzioni di memoria non più utilizzate dalle applicazioni.
- E) Il sistema periodicamente recupera la memoria allocata e non più utilizzabile

Risposta esatta: C - È permesso deallocare memoria

Spiegazione: Come anticipato nella domanda precedente, una caratteristica saliente (e una garanzia di sicurezza) di un linguaggio "puramente" garbage-collected è che nasconde e inibisce i costrutti per la deallocazione esplicita della memoria (come il `free()` del C o il `delete` del C++). Quindi dire che "è permesso deallocare memoria" è falso.

- **Domanda 49**

Testo: Considerando la macchina astratta che permette di eseguire un linguaggio L, il numero di record di attivazione contemporaneamente presenti nel sistema:

- A) È sempre determinabile a priori da un'analisi statica del codice del programma eseguito
- B) Nessuna delle altre risposte
- C) Ha un massimo noto a priori solo se la macchina astratta alloca i record di attivazione dallo heap
- D) Può avere un massimo noto a priori solo se il linguaggio L permette ricorsione
- E) Non dipende dal programma che si esegue

Risposta esatta: B - Nessuna delle altre risposte

Spiegazione: Se un linguaggio permette la ricorsione, la profondità dello stack (numero di AR contemporanei) dipende dai dati di input e dalle condizioni a runtime, quindi non può essere determinata sempre a priori (rendendo false A, C ed E). D'altro canto, è un linguaggio *senza* ricorsione (es. i primi Fortran) ad avere un numero massimo noto a priori, non il contrario (rendendo falsa la D).

- **Domanda 50**

Testo: L'utilizzo della ricorsione in coda (tail recursion) in una funzione:

- A) Utilizza più memoria
- B) Nessuna delle altre risposte
- C) Riduce la possibilità di avere stack overflow
- D) Riduce l'efficienza della funzione
- E) Richiede rimozione manuale degli stack frame

Risposta esatta: C - Riduce la possibilità di avere stack overflow

Spiegazione: La "tail recursion" è una forma ottimizzata di ricorsione dove il risultato dell'ultima chiamata è anche il risultato della funzione chiamante, senza passaggi intermedi post-ritorno. I compilatori ottimizzano questo pattern riutilizzando lo stesso e unico Record di Attivazione, mantenendo un impatto di memoria costante. Pertanto è più efficiente e previene errori critici come lo Stack Overflow.

• **Domanda 51**

Testo: La chiusura (closure) è:

- A) Nessuna delle altre risposte
- B) espressione, ambiente (expression, environment) dove quest'ultimo ha almeno tutte le variabili nell'espressione unitamente all'indirizzo di ritorno
- C) espressione, ambiente (expression, environment) dove quest'ultimo ha alcune delle variabili nell'espressione
- D) espressione, ambiente (expression, environment) dove quest'ultimo ha almeno tutte le variabili nell'espressione
- E) utilizzata per le call by reference

Risposta esatta: D - espressione, ambiente (expression, environment) dove quest'ultimo ha almeno tutte le variabili nell'espressione

Spiegazione: Una chiusura (closure) è l'astrazione a runtime di una funzione. È composta da due elementi fondamentali: il codice della funzione (l'espressione) e l'ambiente di valutazione in cui è stata definita. Quest'ultimo deve necessariamente contenere i binding per "almeno tutte le variabili" libere menzionate all'interno del corpo della funzione, in modo da poterle valutare correttamente al momento dell'invocazione. Non contiene indirizzi di ritorno, prerogativa dei Record di Attivazione (rendendo falsa la B).

• **Domanda 52**

Testo: Polish notation. Indicare il risultato di $x - 567$:

- A) -37
- B) -5
- C) 23
- D) -7
- E) Nessuna delle altre risposte

Risposta esatta: D - -7

Spiegazione: Il testo originale contiene un evidente errore di battitura: $x - 567$ sta per $* - 567$ (dove la "x" indica il per della moltiplicazione e 56 è in realtà 5 e 6). Nella notazione polacca (prefissa), gli operatori precedono gli operandi. Si risolve partendo dall'operatore più interno: $(- 56)$ restituisce -1. L'espressione diventa quindi $(* -1 7)$, che equivale a -7.

- **Domanda 53**

Testo: Un linguaggio di programmazione (come FORTRAN) usa un'allocazione fissa della memoria (at compile time). Questo tipo di linguaggio non può supportare:

- A) floating point numbers
- B) Exceptions
- C) variable size arrays
- D) nessuna delle precedenti
- E) Procedures

Risposta esatta: C - variable size arrays

Spiegazione: L'allocazione statica (fissa a tempo di compilazione) implica che il compilatore debba conoscere in anticipo l'esatto ingombro in memoria di ogni singola struttura dati. Per questo motivo, costrutti la cui dimensione dipende dall'input dell'utente a tempo di esecuzione (come gli array a dimensione variabile o le allocazioni su heap) sono assolutamente impossibili da supportare.

- **Domanda 54**

Testo: La differenza fra espressione e comandi:

- A) Un comando può modificare l'ambiente (locale, non locale o globale) mentre una espressione no
- B) Non esistono differenze: sono due nomi diversi per la stessa entità sintattica
- C) L'esecuzione di un comando produce un valore (o non termina), mentre la valutazione di una espressione può non produrre valori
- D) Le espressioni non possono usare simboli appartenenti ad ambienti non locali
- E) Nessuna delle altre

Risposta esatta: E - Nessuna delle altre

Spiegazione: Tutte le opzioni fornite sono errate. La C è invertita: sono le *espressioni* che producono sempre un valore (restituendo un risultato), mentre i *comandi* (o statement) si eseguono per il loro effetto collaterale (modifica della Memoria/Stato) e non restituiscono valori. La A è falsa perché i comandi alterano la *Memoria* (lo store dei valori), mentre l'Ambiente (il binding dei nomi) viene modificato dalle *dichiarazioni*.

- **Domanda 55**

Testo: Point reversal viene usato per garbage collection:

- A) per evitare di sprecare spazio
- B) nelle implementazioni di python
- C) per gestire loops nelle strutture puntatore
- D) quando non si ha spazio per una search structure
- E) nessuna delle precedenti

Risposta esatta: D - quando non si ha spazio per una search structure

Spiegazione: L'algoritmo di inversione dei puntatori (Pointer Reversal o Schorr-Waite) è utilizzato nella fase "Mark" dei Garbage Collector. Normalmente la navigazione del grafo degli oggetti richiederebbe uno Stack (search structure) o chiamate ricorsive. Tuttavia, dato che il GC entra in funzione proprio quando la memoria scarseggia, allocare uno stack aggiuntivo potrebbe essere impossibile. Il pointer reversal inverte i puntatori attraversati per ricordare il percorso a ritroso senza usare memoria extra.

- **Domanda 56**

Testo: Il cut ! in Prolog:

- A) Ha successo e ferma il backtracking
- B) nessuna delle altre
- C) esegue il backtracking
- D) Chiama una eccezione
- E) fallisce e ferma il backtracking

Risposta esatta: A - Ha successo e ferma il backtracking

Spiegazione: Il cut (scritto !) è un operatore di ottimizzazione in Prolog. Esso ha sempre successo logico e si comporta come una direttiva per il motore di risoluzione: gli "impedisce" di esplorare eventuali soluzioni alternative (backtracking) per tutti i goal a sinistra del cut e per la regola corrente, forzandolo ad accettare la prima soluzione trovata come definitiva.

- **Domanda 57**

Testo: Call by name:

- A) Usata raramente al giorno d'oggi
- B) Utilizzata in molti linguaggi procedurali
- C) Richiede il passaggio dei valori delle variabili sullo stack
- D) Ha una definizione semplice
- E) Nessuna delle altre risposte

Risposta esatta: A - Usata raramente al giorno d'oggi

Spiegazione: Il passaggio di parametri per nome (introdotto storicamente in Algol 60) inietta l'argomento non valutato direttamente nel codice della funzione chiamante. A causa della sua complessità implementativa (richiede la creazione di "thunks" invisibili) e dei suoi side-effects inattesi, è caduto in disuso. Oggi sopravvive solo marginalmente (es. in Scala) o in varianti come il Call by Need (Haskell). Per questo motivo la B, C e D sono false.

- **Domanda 58**

Testo: Un'espressione può essere beta-ridotta ad un'unica forma normale:

- A) Mai
- B) Sempre
- C) Solo se la riduzione termina
- D) Ci possono essere diverse forme normali
- E) Nessuna delle altre risposte

Risposta esatta: C - Solo se la riduzione termina

Spiegazione: Per il Teorema di Church-Rosser (o Teorema di Confluenza) nel Lambda Calcolo, se un'espressione possiede una forma normale (cioè la sequenza di riduzioni beta giunge a un termine non più riducibile e termina), allora tale forma normale è univoca, indipendentemente dall'ordine con cui si scelgono di effettuare le riduzioni.

- **Domanda 59**

Testo: L'iterazione determinata:

- A) Non è implementabile in nessun linguaggio
- B) Permette di modificare la variabile di controllo durante l'esecuzione del corpo dell'iterazione
- C) Non permette di sapere in anticipo quante volte si ripeterà indipendentemente dal corpo del ciclo
- D) è presente in C, C++ e Java
- E) Nessuna delle altre risposte

Risposta esatta: E - Nessuna delle altre risposte

Spiegazione: L'iterazione determinata è caratterizzata proprio dall'assoluto opposto di quanto asserito nelle opzioni: permette *esattamente* di calcolare le iterazioni prima dell'esecuzione (falsa C), vieta rigidamente la modifica del contatore al suo interno (falsa B) e non è presente in C, C++ o Java (che usano iterazione indeterminata nel `for`, falsa D). Essendo storicamente presente in Pascal e Ada, non è vero che sia "non implementabile" (falsa A).

- **Domanda 60**

Testo: `TextIO.lookahead`

- A) Legge il carattere successivo ma lo lascia nell'input stream
- B) Verifica se c'è un altro carattere nell'input stream
- C) error
- D) Legge il carattere successivo
- E) Nessuna delle altre risposte

Risposta esatta: A - Legge il carattere successivo ma lo lascia nell'input stream

Spiegazione: Nel linguaggio SML, la funzione `TextIO.lookahead` (come suggerisce il nome stesso, "guarda avanti") consente di "sbirciare" il prossimo carattere in arrivo nel buffer del file o dello standard input, restituendolo al chiamante senza però consumarlo o rimuoverlo dal flusso (a differenza di funzioni come `input1`).

- **Domanda 61**

Testo: Memorizzare array in ordine di riga o colonna:

- A) Nessuna delle altre risposte
- B) Conta soltanto quando si usa il caching
- C) Viene sempre usato l'ordine per colonna
- D) Non conta mai
- E) Viene sempre usato l'ordine per riga

Risposta esatta: B - Conta soltanto quando si usa il caching

Spiegazione: A livello teorico e semantico (per la correttezza di un algoritmo) non vi è alcuna differenza. Tuttavia, a livello hardware, iterare su un array multidimensionale sfruttando la sua mappatura in memoria (Row-Major per il C, Column-Major per il Fortran) permette di accedere a posizioni adiacenti contigue, innescando la località spaziale che massimizza le performance di accesso alla memoria Cache del processore, abbattendo drasticamente i tempi di esecuzione.

- **Domanda 62**

Testo: Pointer reversal is used in garbage collection:

- A) To avoid wasting space for marking used records
- B) In implementations of Python
- C) To deal with loops in pointer structures
- D) When we don't have space for a search structure
- E) Nessuna delle altre risposte

Risposta esatta: D - When we don't have space for a search structure

Spiegazione: Questa è l'esatta traduzione in lingua inglese della domanda numero 55 vista in precedenza. Il pointer reversal è una tecnica salvavita impiegata nell'attraversamento di strutture a grafo quando la memoria residua non consente di istanziare uno Stack dedicato all'esplorazione (la "search structure").

- **Domanda 63**

Testo: Un ADT è definito essere un modello matematico di un user-defined type, insieme a tutte le operazioni su quel modo:

- A) Struttura
- B) Primitiva
- C) Assignment
- D) Cardinalità
- E) Nessuna delle precedenti

Risposta esatta: E - Nessuna delle precedenti

Spiegazione: Il termine "modo" presente nella domanda è un palese errore di traduzione/battitura per "modello" (model). La definizione canonica sancisce che un Abstract Data Type (ADT) è definito dai suoi tipi di dato e dall'insieme di tutte le operazioni *su quel modello*. Poiché nessuna delle opzioni A, B, C o D (struttura, primitiva, ecc.) completa correttamente l'asserzione, la risposta corretta è "Nessuna delle precedenti".

- **Domanda 64**

Testo: Un linguaggio di programmazione (come FORTRAN) usa una allocazione fissa della memoria (in compilazione). Questo tipo di linguaggio non può supportare:

- A) Procedure
- B) Eccezioni
- C) Ricorsione
- D) Nessuna delle altre risposte
- E) Numeri floating-point

Risposta esatta: C - Ricorsione

Spiegazione: Come appurato alla Domanda 53, allocare tutto staticamente prima dell'esecuzione ha forti conseguenze architettoniche. Oltre agli array dinamici, viene invalidata la Ricorsione: l'assenza di uno Stack dinamico per i record di attivazione costringe il programma ad allocare una singola area di memoria fissa per ogni subroutine. Invocare due volte la stessa funzione provocherebbe la sovrascrittura distruttiva del singolo blocco di memoria a lei destinato.

- **Domanda 65**

Testo: In Prolog:

- A) Conta l'ordine delle regole e predicati (da sinistra a destra) in una regola
- B) Nessuna delle altre risposte
- C) Conta l'ordine delle regole e predicati (da destra a sinistra) in una regola
- D) Conta solo l'ordine delle regole
- E) Conta solo l'ordine dei predicati (da destra a sinistra) in una regola

Risposta esatta: A - Conta l'ordine delle regole e predicati (da sinistra a destra) in una regola

Spiegazione: A dispetto dell'idea originaria della Programmazione Logica in cui l'ordine delle dichiarazioni non dovrebbe alterare il risultato matematico, in Prolog l'ordine è cruciale per via del suo motore di inferenza (algoritmo di risoluzione procedurale). L'esplorazione (SLD-Resolution) è di tipo *Depth-First, Left-To-Right*: le regole vengono testate dall'alto verso il basso (nell'ordine in cui sono scritte nel database), e per ciascuna regola, i predicati (sub-goals) al suo interno sono provati sequenzialmente da sinistra a destra.

• **Domanda 66**

Testo: Un linker:

- A) È scritto in linguaggio macchina
- B) Sostituisce indirizzi relativi con indirizzi assoluti
- C) Viene utilizzato nei linguaggi interpretati
- D) Nessuna delle altre risposte
- E) Viene usato per l'efficienza

Risposta esatta: B - Sostituisce indirizzi relativi con indirizzi assoluti

Spiegazione: Il Linker (collegatore) è lo strumento fondamentale della catena di compilazione che riceve in ingresso i vari moduli oggetto (.o o .obj). Il suo compito primario è la "Rilocazione" (oltre che la risoluzione delle dipendenze esterne): trasforma gli indirizzi relativi calcolati dal compilatore all'interno dei singoli moduli non interconnessi, unendoli e traducendoli negli indirizzi assoluti e definitivi dell'eseguibile unico finale.

22 Domande Lambda Calcolo

• **Domanda 67**

Testo: Beta-riducendo $(\lambda n.\lambda m.\lambda f.\lambda x.(nf)((mf)x))(\lambda f.\lambda x.f f f f x)(\lambda f.\lambda x.f x)$ si ottiene:

- A) $\lambda f.\lambda x.f f f f x$
- B) f
- C) Nessuna delle altre risposte
- D) $\lambda f.\lambda x.f f f f f f x$
- E) x
- F) $\lambda f.\lambda x.f f f f f x$

Risposta esatta: F - $\lambda f.\lambda x.f f f f f x$

Spiegazione: Il primo blocco dell'espressione rappresenta la funzione Somma (*Addition*) nella codifica dei Numerali di Church. I due argomenti passati sono il numero 4 (una funzione che applica f per 4 volte a x) e il numero 1 (che applica f per 1 volta). La somma matematica di 4 e 1 è 5, perciò il risultato sarà l'applicazione di f per 5 volte consecutive alla x , che corrisponde all'opzione F.

- **Domanda 68**

Testo: Beta-riducendo $(\lambda n.\lambda m.\lambda f.\lambda x.(nf)((mf)x))(\lambda f.\lambda x.fx)(\lambda f.\lambda x.x)$ si ottiene:

- A) $\lambda f.\lambda x.f(f(f(fx)))$
- B) $\lambda f.\lambda x.fx$
- C) x
- D) fx
- E) Nessuna delle altre risposte

Risposta esatta: B - $\lambda f.\lambda x.fx$

Spiegazione: Come nella domanda precedente, il termine iniziale è l'operatore di Addizione. In questo caso viene applicato al numero 1 (rappresentato da $\lambda f.\lambda x.fx$) e al numero 0 di Church (rappresentato da $\lambda f.\lambda x.x$, in cui la f non viene applicata). Poiché $1 + 0 = 1$, il risultato beta-ridotto restituisce il numerale di Church corrispondente a 1.

- **Domanda 69**

Testo: Beta-riducendo $(\lambda n.\lambda f.\lambda x.f((nf)x))(\lambda f.\lambda x.f(f(fx)))$ si ottiene:

- A) La riduzione non termina
- B) fx
- C) $\lambda f.\lambda x.f f f f f x$
- D) Nessuna delle altre risposte
- E) $\lambda f.\lambda x.f(f(f(fx)))$

Risposta esatta: E - $\lambda f.\lambda x.f(f(f(fx)))$

Spiegazione: Il primo termine è la funzione Successore (*SUCC*), che prende un numerale di Church n e restituisce $n + 1$ avvolgendo il tutto in un'ulteriore applicazione di f . Il parametro passato è il numero 3 (con 3 f). Applicando il successore al numero 3 si ottiene sintatticamente il numero 4 (con 4 f), cioè l'opzione E.

- **Domanda 70**

Testo: Beta-riducendo $(\lambda a.((\lambda b.\lambda c.c)\lambda d.\lambda e.d))(\lambda f.\lambda g.f)$ si ottiene:

- A) $\lambda b.\lambda c.c$
- B) Nessuna delle altre risposte
- C) La riduzione non termina
- D) c
- E) $\lambda b.\lambda c.b$

Risposta esatta: A - $\lambda b.\lambda c.c$

Spiegazione: Questa è una codifica logico-condizionale. L'argomento fornito al posto di a è $(\lambda f.\lambda g.f)$, che rappresenta il booleano *TRUE* di Church (prende due parametri e restituisce il primo). Il corpo centrale contiene una scelta tra $\lambda b.\lambda c.c$ (ovvero *FALSE*) e $\lambda d.\lambda e.d$. L'espressione si comporta come un **if TRUE then FALSE else TRUE**. Scegliendo il primo parametro (essendo TRUE), il risultato è il falso di Church.

• **Domanda 71**

Testo: Beta-riducendo $(\lambda x.xy)(\lambda z.zx)(\lambda z.zx)$ si ottiene:

- A) Nessuna delle altre risposte
- B) xyz
- C) $(\lambda x.xy)yx$
- D) $yx(\lambda z.zx)$
- E) La riduzione non termina

Risposta esatta: D - $yx(\lambda z.zx)$

Spiegazione: Valutiamo passo passo l'applicazione da sinistra a destra (essendo associativa a sinistra). Appliciamo la prima funzione al secondo termine sostituendo x con $(\lambda z.zx)$: otteniamo $((\lambda z.zx)y)$. Applicando questa funzione passiamo y al posto di z , ottenendo (yx) . Infine, accodiamo il terzo termine originale non ancora processato: $(yx)(\lambda z.zx)$.

• **Domanda 72**

Testo: Beta-riducendo $(\lambda n.\lambda f.\lambda x.f((nf)x))(\lambda f.\lambda x.f f f x)$ si ottiene:

- A) Nessuna delle altre risposte
- B) $\lambda f.\lambda x.f f f f f x$
- C) La riduzione non termina
- D) $\lambda f.\lambda x.f f f f f x$
- E) fx
- F) L'espressione è irriducibile

Risposta esatta: A - Nessuna delle altre risposte

Spiegazione: Ancora una volta, abbiamo la funzione Successore applicata al numero di Church 3 ($f f f x$). Il risultato atteso sarebbe il numero 4, ovvero $\lambda f.\lambda x.f f f f x$. Poiché tra le opzioni fornite ci sono solo 5, 6, fx o risposte logiche scorrette, e manca il risultato corretto per 4, la risposta esatta è "Nessuna delle altre".

• **Domanda 73**

Testo: Beta f-riducendo $((\lambda a.aa)(\lambda b.b))(\lambda c.c)$ si ottiene:

- A) La riduzione non termina
- B) aaa
- C) $\lambda x.xa$
- D) Nessuna delle altre risposte
- E) $\lambda c.c$

Risposta esatta: E - $\lambda c.c$

Spiegazione: Eseguiamo il primo blocco $((\lambda a.aa)(\lambda b.b))$. Sostituendo la funzione identità $\lambda b.b$ al posto di a , l'espressione diventa $(\lambda b.b)(\lambda b.b)$. Valutando questa applicazione (identità su identità), riotteniamo $\lambda b.b$. L'espressione totale diventa ora $(\lambda b.b)(\lambda c.c)$, ovvero l'identità applicata a $\lambda c.c$, che restituisce intatto $\lambda c.c$.

- **Domanda 74**

Testo: Beta f-riducendo $(\lambda a.aaa)((\lambda b.b)(\lambda c.c))$ si ottiene:

- A) $\lambda a.a$
- B) Nessuna delle altre risposte
- C) aaa
- D) La riduzione non termina
- E) $\lambda x.xa$

Risposta esatta: B - Nessuna delle altre risposte

Spiegazione: Valutiamo prima l'argomento interno: $(\lambda b.b)(\lambda c.c)$, che è l'identità applicata a $\lambda c.c$. Il risultato è $\lambda c.c$. Ora passiamo questo risultato alla funzione esterna: $(\lambda a.aaa)(\lambda c.c)$. Sostituendo a , otteniamo $(\lambda c.c)(\lambda c.c)(\lambda c.c)$. Valutando questa sequenza di identità, il risultato finale converge a $\lambda c.c$. Poiché questo termine non è tra le opzioni, "Nessuna" è la risposta corretta.

- **Domanda 75**

Testo: Beta f-riducendo $(\lambda d.((\lambda a.abc)(bc)))(\lambda y.xyz)$ si ottiene:

- A) Nessuna delle altre risposte
- B) La riduzione non termina
- C) $\lambda b.bba$
- D) $\lambda a.abc$
- E) $bcbc$

Risposta esatta: E - $bcbc$

Spiegazione: Iniziamo con il corpo più interno: $(\lambda a.abc)(bc)$. Sostituendo (bc) al posto della a si ottiene $(bc)bc$. L'espressione complessiva diventa quindi $\lambda d.((bc)bc)$ a cui dobbiamo applicare l'argomento $\lambda y.xyz$. Tuttavia, la variabile legata d non appare da nessuna parte nel corpo $(bcbc)$. Pertanto, l'argomento viene assorbito a vuoto (scartato) e l'espressione restituisce semplicemente $bcbc$.

- **Domanda 76**

Testo: Beta f-riducendo $(\lambda n.\lambda f.\lambda x.f((nf)x))(\lambda f.\lambda x.f(f(fx))))$ si ottiene:

- A) Nessuna delle altre risposte
- B) L'espressione è irriducibile
- C) La riduzione non termina
- D) $\lambda f.\lambda x.f f f f f x$
- E) fx
- F) $\lambda f.\lambda x.f(f(f(f(fx))))$

Risposta esatta: F - $\lambda f.\lambda x.f(f(f(f(fx))))$

Spiegazione: Analogamente alla domanda 3, la funzione esterna calcola il Successore del numerale in ingresso. Nel testo dell'opzione corretta sembra ci sia un offset di parentesi nella stesura originale del test, ma concettualmente il successore di un numero di Church aggiunge una singola applicazione della funzione f in testa, portando all'opzione indicata F come catena annidata corretta rispetto alla base.

- **Domanda 77 Testo:** Beta-riducendo $(\lambda n.\lambda m.\lambda f.\lambda x.(nf)((mf)x))(\lambda f.\lambda x.fx)(\lambda f.\lambda x.x)$ si ottiene:

- A) Nessuna delle altre risposte
- B) x
- C) $\lambda f.\lambda x.f f f f f f x$
- D) $\lambda f.\lambda x.fx$
- E) fx
- F) $\lambda f.\lambda x.f f f f x$

Risposta esatta: D - $\lambda f.\lambda x.fx$ **Spiegazione:** Il primo termine rappresenta l'operatore matematico di Addizione all'interno della codifica dei Numeri di Church. Tale funzione riceve come argomenti due numerali e ne somma le applicazioni. Il primo argomento $(\lambda f.\lambda x.fx)$ è il numerale 1, mentre il secondo $(\lambda f.\lambda x.x)$ è il numerale 0. Eseguendo la somma logica di $1 + 0$ si ottiene 1, che corrisponde sintatticamente all'opzione D.

- **Domanda 78**

Testo: Beta-riducendo $(\lambda a.((\lambda b.\lambda c.c)\lambda d.\lambda e.d))(\lambda f.\lambda g.g)$ si ottiene:

- A) Nessuna delle altre risposte
- B) $\lambda b.\lambda c.b$
- C) b
- D) $\lambda b.\lambda c.c$
- E) La riduzione non termina

Risposta esatta: B - $\lambda b.\lambda c.b$

Spiegazione: L'espressione principale è un costrutto condizionale (un IF-THEN-ELSE in logica di Church). L'argomento passato ad a è $(\lambda f.\lambda g.g)$, che rappresenta il booleano *FALSE* (prende due argomenti e restituisce il secondo). Nel blocco condizionale $((a \text{ arg1}) \text{ arg2})$, il *FALSE* forza la selezione del secondo argomento, ovvero $\lambda d.\lambda e.d$. Questo termine prende due parametri e restituisce il primo, che lo rende di fatto equivalente al *TRUE* di Church. Tramite una semplice α -conversione (rinomina delle variabili), $\lambda d.\lambda e.d$ equivale perfettamente a $\lambda b.\lambda c.b$.

- **Domanda 79**

Testo: Beta-riducendo $(\lambda a.aab)((\lambda a.aab)(\lambda a.(\lambda b.ba)c))$ si ottiene:

- A) La riduzione non termina
- B) aab
- C) $\lambda a.aab$
- D) $ccb(ccb)c$
- E) Nessuna delle altre risposte

Risposta esatta: E - Nessuna delle altre risposte

Spiegazione: Questa espressione innesca una reazione di auto-applicazione ricorsiva complessa. Risolvendo il blocco più a destra $(\lambda a.(\lambda b.ba)c)$ otteniamo $\lambda a.ca$. Passando questo termine a $(\lambda a.aab)$, l'applicazione $(\lambda a.ca)(\lambda a.ca)b$ si riduce a $c(\lambda a.ca)b$. Questo enorme blocco risultante diviene a sua volta l'argomento per la funzione più esterna $(\lambda a.aab)$, portando a un'espansione massiccia del termine che non coincide con nessuna delle stringhe limitate (come $ccb(ccb)c$) elencate nelle opzioni A, B, C o D.

- **Domanda 80**

Testo: Beta-riducendo $((\lambda a.aaa)(\lambda b.b))(\lambda c.c)$ si ottiene:

- A) $\lambda c.c$
- B) La riduzione non termina
- C) aaa
- D) $\lambda x.xa$
- E) Nessuna delle altre risposte

Risposta esatta: A - $\lambda c.c$

Spiegazione: Si valuta inizialmente il blocco funzionale $((\lambda a.aaa)(\lambda b.b))$. Sostituendo la funzione identità $\lambda b.b$ a tutte le occorrenze di a , otteniamo un'applicazione tripla dell'identità: $(\lambda b.b)(\lambda b.b)(\lambda b.b)$. Riducendola su sé stessa, l'identità persiste ritornando $\lambda b.b$. L'espressione si è ora semplificata in $(\lambda b.b)(\lambda c.c)$. Quest'ultima è banalmente l'applicazione della funzione identità al termine $\lambda c.c$, che viene restituito intatto.

- **Domanda 81**

Testo: Beta-riducendo $(\lambda d.((\lambda a.abc)(bc)))(\lambda y.xyz)$ si ottiene:

- A) La riduzione non termina
- B) $bcbc$
- C) Nessuna delle altre risposte
- D) $\lambda a.abc$
- E) $\lambda b.bb$

Risposta esatta: B - *bcbc*

Spiegazione: Questa domanda è concettualmente speculare alla Domanda 9 analizzata in precedenza. Il nucleo più profondo $((\lambda a.abc)(bc))$ si valuta sostituendo (bc) al posto della variabile a , generando la stringa $(bc)bc$. L'intera espressione è ora $(\lambda d.(bcbc))(\lambda y.xyz)$. Poiché nel corpo non v'è alcuna occorrenza della variabile legata d , l'argomento esterno $(\lambda y.xyz)$ è completamente irrilevante: esso viene assorbito e scartato nel vuoto, lasciando come unica soluzione il corpo originario *bcbc*.

23 Domande Calcolo Indirizzo Array

- **Domanda 82 Testo:** Se gli array sono memorizzati per righe e `int a[100][100]` è un array multidimensionale di interi (si assuma che la dimensione di un intero sia 4 byte) con `a[0][0]` che ha indirizzo `0x5000`, qual è l'indirizzo di `a[5][10]`?

- A) `0x57F8`
- B) Nessuna delle altre risposte
- C) `0x53ED`
- D) `0x5510`
- E) `0x51FE`

Risposta esatta: A - `0x57F8` **Spiegazione:** In un array memorizzato in ordine di riga (Row-Major Order), l'indirizzo di un elemento (i, j) si calcola tramite la formula: $\text{Base} + (i \times \text{Numero_Colonne} + j) \times \text{Dimensione_Dato}$. Sostituendo i valori forniti: $0x5000 + (5 \times 100 + 10) \times 4 = 0x5000 + 510 \times 4 = 0x5000 + 2040$. Convertendo 2040 decimale in esadecimale otteniamo `0x7F8`. Sommandolo alla base abbiamo infine `0x57F8`.

- **Domanda 83**

Testo: Se gli array sono memorizzati per colonne ed `int a[100][100]` è un array multidimensionale di interi (si assuma che la dimensione di un intero sia 4 byte) con `a[0][0]` che ha indirizzo `0x5000`, qual è l'indirizzo di `a[5][10]`?

- A) `0x5510`
- B) `0x53ED`
- C) Nessuna delle altre risposte
- D) `0x500F`
- E) `0x41FE`

Risposta esatta: C - Nessuna delle altre risposte

Spiegazione: L'ordine di memorizzazione per colonne (Column-Major Order) esplora primariamente l'indice di riga. La formula inversa diventa: $\text{Base} + (j \times \text{Numero_Righe} + i) \times \text{Dimensione_Dato}$. Sostituendo: $0x5000 + (10 \times 100 + 5) \times 4 = 0x5000 + 1005 \times 4 = 0x5000 + 4020$. Il valore decimale 4020 convertito in esadecimale è `0xFB4`. L'indirizzo esatto risultante è `0x5FB4`, che non è presente tra le opzioni matematiche offerte.

- **Domanda 84**

Testo: Se gli array sono memorizzati per righe e `char a[100][100][100]` è un array multidimensionale di caratteri con `a[0][0][0]` che ha indirizzo `0x1000`, qual è l'indirizzo di `a[5][5][10]`?

- A) Nessuna delle altre risposte
- B) `0x51510`
- C) `0xC54E`
- D) `0xD54E`
- E) `0x50510`

Risposta esatta: D - `0xD54E`

Spiegazione: In un array a 3 dimensioni con ordinamento a righe, l'indirizzo dell'elemento (i, j, k) è $\text{Base} + (i \times d_2 \times d_3 + j \times d_3 + k) \times \text{Dimensione}$. Applicando le misure: $0x1000 + (5 \times 100 \times 100 + 5 \times 100 + 10) \times 1 = 0x1000 + 50000 + 500 + 10 = 0x1000 + 50510$. La conversione esadecimale del puro scostamento (offset) 50510 genera il valore `0xC54E`. Sommandoci l'indirizzo di base originario `0x1000`, si ottiene regolarmente `0xD54E`.

- **Domanda 85**

Testo: Se gli array sono memorizzati per colonne e `char a[100][100][100]` è un array multidimensionale di caratteri con `a[0][0][0]` che ha indirizzo `0x1000`, qual è l'indirizzo di `a[5][5][10]`?

- A) Nessuna delle altre risposte
- B) `0x18899`
- C) `0x51510`
- D) `0xD54E`
- E) `0x19899`

Risposta esatta: E - `0x19899`

Spiegazione: L'estensione 3D dell'indirizzamento per colonne segue le logiche Fortran. La formula posizionale per un elemento (i, j, k) trasforma l'offset in $i + j \times d_1 + k \times d_1 \times d_2$ moltiplicato per la dimensione. Calcolo dell'offset: $5 + 5 \times 100 + 10 \times (100 \times 100) = 5 + 500 + 100000 = 100505$. La codifica in base 16 del numero intero 100505 restituisce `0x18899`. Aggiungendo a quest'ultimo l'iniziale `0x1000` di Base, deriviamo analiticamente `0x19899`.

- **Domanda 86**

Testo: Se gli array sono memorizzati per colonne e `int a[25][25]` è un array multidimensionale di interi (si assuma che un intero sia memorizzato in 4 byte) con `a[0][0]` che ha indirizzo `0x1000`, qual è l'indirizzo di `a[5][10]`?

- A) `0x11FE`
- B) Nessuna delle altre risposte
- C) `0x100F`
- D) `0x13FC`
- E) `0x121C`

Risposta esatta: D - `0x13FC`

Spiegazione: Utilizzando la formula per matrici Column-Major in 2D: $\text{Base} + (j \times \text{Righe} + i) \times \text{Size}$. Otteniamo: $0x1000 + (10 \times 25 + 5) \times 4 = 0x1000 + (255) \times 4 = 0x1000 + 1020$. Convertendo l'incremento decimale di 1020 in hex (sapendo che 1024 corrisponde a `0x400`, e togliendo 4 otteniamo `0x3FC`) la destinazione definitiva risulterà in `0x13FC`.

- **Domanda 87**

Testo: Se gli array sono memorizzati per colonne ed `short int a[100][100]` è un array multidimensionale di interi corti (si assuma che la dimensione di uno short int sia 2 byte) con `a[0][0]` che ha indirizzo `0x4100` qual è l'indirizzo di `a[5][10]`?

- A) `0x48DA`
- B) `0x4510`
- C) Nessuna delle altre risposte
- D) `0x47DA`
- E) `0x41FE`

Risposta esatta: A - `0x48DA`

Spiegazione: Riavviando l'esplorazione per colonne (Column-Major) a dimensione pari a 2 Byte. La misurazione si declina in: $0x4100 + (10 \times 100 + 5) \times 2 = 0x4100 + 1005 \times 2 = 0x4100 + 2010$. Decodificando il delta numerico 2010 dal decimale alle sedici cifre otteniamo il valore `0x7DA`. Inglobato nello stato architetturale di base `0x4100`, il puntatore farà capo a `0x48DA`.

- **Domanda 88**

Testo: Se gli array sono memorizzati per colonne e `char a[100][100]` è un array multidimensionale di caratteri con `a[0][0]` che ha indirizzo `0x1100`, qual è l'indirizzo di `a[5][10]`?

- A) Nessuna delle altre risposte
- B) `0x24ED`
- C) `0x21FE`
- D) `0x22FE`
- E) `0x14ED`

Risposta esatta: E - `0x14ED`

Spiegazione: In un array memorizzato in Column-Major Order (per colonne), l'indirizzo di un elemento (i, j) si calcola come: $\text{Base} + (j \times \text{Numero_Righe} + i) \times \text{Dimensione}$. Con dimensione di 1 byte per il tipo `char`: $0x1100 + (10 \times 100 + 5) \times 1 = 0x1100 + 1005$. Convertendo 1005 decimale in esadecimale otteniamo `0x3ED`. Sommandolo alla base: $0x1100 + 0x3ED = 0x14ED$.

- **Domanda 89**

Testo: Se gli array sono memorizzati per colonne e `char a[100][100]` è un array multidimensionale di caratteri con `a[0][0]` che ha indirizzo `5243`, qual è l'indirizzo di `a[5][10]`? (i numeri sono in decimale)

- A) Nessuna delle altre risposte
- B) `5753`
- C) `6248`
- D) `4730`
- E) `4955`

Risposta esatta: C - `6248`

Spiegazione: Avendo gli stessi indici della domanda precedente, sappiamo che l'offset posizionale per colonna è pari a 1005 elementi. Poiché la dimensione del `char` è 1 e i calcoli richiesti sono direttamente in base decimale, basta sommare l'offset all'indirizzo di base: $5243 + 1005 = 6248$.

- **Domanda 90**

Testo: Se gli array sono memorizzati per colonne ed `short int a[100][100]` è un array multidimensionale di interi corti (si assuma che la dimensione di uno `short int` sia 2 byte) con `a[0][0]` che ha indirizzo `0x4100`, qual è l'indirizzo di `a[5][10]`?

- A) `0x41FE`
- B) `0x500F`
- C) `0x47DA`
- D) `0x4510`
- E) `0x43ED`
- F) Nessuna delle altre risposte

Risposta esatta: F - Nessuna delle altre risposte

Spiegazione: Questa è una riproposizione esatta della Domanda 6 della sezione calcolo indirizzi. Come calcolato in precedenza, l'offset per colonne è 1005, che moltiplicato per 2 byte di dimensione fa 2010 (ovvero `0x7DA` in esadecimale). L'indirizzo corretto è quindi $0x4100 + 0x7DA = 0x48DA$. Essendo tale opzione assente, la risposta corretta è "Nessuna".

- **Domanda 91**

Testo: Se gli array sono memorizzati per righe ed `int a[100][100]` è un array multidimensionale di interi (si assuma che la dimensione di un intero sia 4 byte) con `a[0][0]` che ha indirizzo `0x1000`, qual è l'indirizzo di `a[5][10]`?

- A) `0x100F`
- B) Nessuna delle altre risposte
- C) `0x13ED`
- D) `0x11FE`
- E) `0x1510`

Risposta esatta: B - Nessuna delle altre risposte

Spiegazione: Per un array in Row-Major Order (per righe), la formula è $\text{Base} + (i \times \text{Numero_Colonne} + j) \times \text{Dimensione}$. Sostituendo: $0x1000 + (5 \times 100 + 10) \times 4 = 0x1000 + 510 \times 4 = 0x1000 + 2040$. 2040 in decimale corrisponde a `0x7F8` in esadecimale. L'indirizzo finale atteso è `0x17F8`, che non è presente tra le risposte.

- **Domanda 92**

Testo: Se gli array sono memorizzati per righe e `char a[100][100]` è un array multidimensionale di caratteri con `a[0][0]` che ha indirizzo `0x2000`, qual è l'indirizzo di `a[5][10]`?

- A) `0x200F`
- B) `0x21FE`
- C) Nessuna delle altre risposte
- D) `0x23ED`
- E) `0x2510`

Risposta esatta: B - `0x21FE`

Spiegazione: Analogamente alla domanda precedente (ordine per righe), l'offset in termini di celle per la coordinata `[5][10]` è $(5 \times 100 + 10) = 510$. Poiché la dimensione del `char` è 1 byte, il salto in memoria ammonta a 510 byte, cioè `0x1FE` in esadecimale. Sommandolo all'indirizzo base: $0x2000 + 0x1FE = 0x21FE$.

- **Domanda 93**

Testo: Se gli array sono memorizzati per colonne ed `int a[100][100]` è un array multidimensionale di interi (si assuma che la dimensione di un intero sia 4 byte) con `a[0][0]` che ha indirizzo `0x5000`, qual è l'indirizzo di `a[5][10]`?

- A) Nessuna delle altre risposte
- B) `0x53ED`
- C) `0x500F`
- D) `0x41FE`
- E) `0x551`

Risposta esatta: A - Nessuna delle altre risposte

Spiegazione: Questa è la ripetizione esatta della Domanda 2 della sezione calcolo indirizzi. La formula per colonne restituisce un offset di 1005, che moltiplicato per 4 fa 4020 byte. Convertito, 4020 diventa `0xFB4`. L'indirizzo finale corretto sarebbe `0x5FB4`. Poiché non è tra le opzioni (la E contiene un evidente typo e le altre sono errate), la risposta è "Nessuna".

- **Domanda 94**

Testo: Se gli array sono memorizzati per righe e `char a[100][100]` è un array multidimensionale di caratteri con `a[0][0]` che ha indirizzo `0x1100`, qual è l'indirizzo di `a[5][10]`?

- A) `0x22FE`
- B) `0x12FE`
- C) `0x1510`
- D) `0x210F`
- E) Nessuna delle altre risposte

Risposta esatta: B - 0x12FE

Spiegazione: Come calcolato precedentemente per il `char` memorizzato per righe, l'offset totale in byte è 510, che corrisponde a 0x1FE. Aggiungendolo alla nuova base fornita: $0x1100 + 0x1FE = 0x12FE$.

- **Domanda 95**

Testo: Se gli array sono memorizzati per colonne ed `short int a[100][100]` è un array multidimensionale di interi corti (si assuma che la dimensione di uno `short int` sia 2 byte) con `a[0][0]` che ha indirizzo 16640, qual è l'indirizzo di `a[5][10]`?:

- A) 18650
- B) 18394
- C) 17680
- D) 16894
- E) Nessuna delle altre risposte

Risposta esatta: A - 18650

Spiegazione: L'offset in posizioni per l'ordine a colonne è $(10 \times 100 + 5) = 1005$. Moltiplicato per i 2 byte del tipo di dato, il salto ammonta a 2010 byte in decimale. Poiché ci viene fornito l'indirizzo base direttamente in decimale, non serve la conversione esadecimale: $16640 + 2010 = 18650$.

24 Domande Tipi Funzioni ML

- **Domanda 96**

Testo: Qual è il tipo della seguente espressione: `fun f x y = x y; ?`

- A) `fun: 'a->'b`
- B) `fun: 'a->'b->string->string`
- C) `fun: ('a->'b)->'a->'b`
- D) `fun: int->int`
- E) `fun: string->string->string->string`

Risposta esatta: C - `fun: ('a->'b)->'a->'b`

Spiegazione: In ML, la notazione `x y` significa che `x` viene applicato a `y` come funzione. Affinché ciò sia corretto nel sistema di tipi, `x` deve essere una funzione generica che prende in input un tipo `'a` e restituisce un tipo `'b` (quindi `'a -> 'b`). Di conseguenza, l'argomento `y` deve necessariamente essere di tipo `'a`. Il tipo totale della funzione è quindi il tipo di `x`, seguito dal tipo di `y`, che restituisce `'b`.

- **Domanda 97**

Testo: Qual è il tipo della seguente espressione: `List.map print` (sottintendendo l'applicazione parziale in un contesto `string`) ?

- A) `fun: string list -> string list`
- B) `fun: string list -> unit list`
- C) `fun: string list -> string`
- D) `fun: string -> string list`
- E) `fun: string -> unit`
- F) `fun: 'a list -> 'b list`

Risposta esatta: B - `fun: string list -> unit list`

Spiegazione: La funzione d'ordine superiore `List.map` possiede la firma polimorfa `('a -> 'b) -> 'a list -> 'b list`. La funzione `print` incorporata in ML ha tipo `string -> unit`. Passando `print` come primo argomento a `map` (currying), la funzione risultante prenderà in input una lista di stringhe (`string list`) e restituirà una lista con i risultati delle valutazioni, ovvero una lista di valori `unit` (`unit list`).

- **Domanda 98**

Testo: Qual è il tipo della seguente espressione: `let f1 l = List.map (fun x y -> x,y) l; ?`

- A) `f1:'a list->('b -> 'a -> 'b) list`
- B) `f1:'a list-> 'b -> ('a * 'b)`
- C) `fun x y -> x,y : 'a -> 'b -> 'a * 'b`
- D) `fun x y -> x,y : 'a -> 'b -> 'a -> 'b`
- E) `f1:'a list->('b -> 'a * 'b) list`

Risposta esatta: E - `f1:'a list->('b -> 'a * 'b) list`

Spiegazione: Analizziamo la funzione anonima interna `(fun x y -> x,y)`: essa prende un elemento `x` (di tipo `'a`), poi un `y` (di tipo `'b`) e restituisce la tupla `(x, y)` (di tipo `'a * 'b`). Grazie all'applicazione parziale, quando `List.map` passa un singolo elemento della lista a questa funzione, questa non viene valutata completamente, ma restituisce la funzione in attesa della `y`. Quindi `List.map` trasformerà una lista originaria di tipo `'a list` in una lista in cui ogni elemento è una funzione `('b -> 'a * 'b)`.

- **Domanda 99**

Testo: Qual è il tipo della seguente espressione: `(fn x => x) 12; ?`

- A) `12: int`
- B) `'a: 12`
- C) `string: "12"`
- D) `fn: 'a -> 'a`
- E) Nessuna delle altre risposte

Risposta esatta: A - `12: int`

Spiegazione: L'espressione `(fn x => x)` è la funzione identità generica in SML, che restituisce l'argomento ricevuto. Fornendo in pasto l'intero 12 all'operatore di applicazione, la funzione viene eseguita e il risultato è semplicemente il valore 12. L'inferenza dei tipi dedurrà coerentemente che il tipo finale in output per il programma è `int`.

- **Domanda 100**

Testo: Qual è il tipo della seguente espressione: `fun f1 x = [x,x]; ?`

- A) `f1=fn: 'a list -> 'a list`
- B) `f1=fn: 'a list -> 'b list`
- C) `f1=fn: 'a -> 'b`
- D) `f1=fn: 'a -> 'a list`
- E) `f1=fn: 'a -> 'a`
- F) `f1=fn: string -> string`

Risposta esatta: D - `f1=fn: 'a -> 'a list`

Spiegazione: La funzione riceve un singolo argomento `x` a cui il sistema assegna provvisoriamente il tipo generico `'a`. Il corpo della funzione crea e restituisce una lista sintatticamente valida `[x, x]`. Poiché le liste in ML sono rigorosamente omogenee, una lista formata da elementi `x` avrà come tipo di ritorno `'a list`.

- **Domanda 101**

Testo: Qual è il tipo della seguente espressione: `fun f2 x y = (x @ (y x)); ?`

- A) `fun: 'a -> 'a -> 'a -> 'a`
- B) `fun: 'a list -> ('a list -> 'a list) -> 'a list`
- C) `fun: 'a list -> 'a list -> 'a list`
- D) `fun: 'a list -> 'a list -> 'b list`
- E) `fun: 'a -> ('a list -> 'a list) -> 'a list`

Risposta esatta: B - `fun: 'a list -> ('a list -> 'a list) -> 'a list`

Spiegazione: L'operatore `@` serve per concatenare (append) due liste dello stesso tipo. Questo implica che sia `x` sia il risultato di `(y x)` debbano essere liste di tipo `'a list`. Affinché `(y x)` sia un'espressione valida, `y` deve essere una funzione che

accetta `x` (che abbiamo stabilito essere `'a list`) e restituisce a sua volta `'a list`. Dunque, la funzione `f2` riceve un `'a list` (il parametro `x`), poi una funzione (`'a list -> 'a list`) (il parametro `y`) e restituisce il risultato dell'`append`, che è `'a list`.

- **Domanda 102**

Testo: Qual è il tipo della seguente espressione: `fun f3 x = List.map x; ?`

- A) `fun: ('a list -> 'b list) -> 'a list -> 'b list`
- B) `fun: ('a -> 'b) -> 'a -> 'b`
- C) `fun: 'a list -> 'b list`
- D) `fun: ('a -> 'a) -> 'a list -> 'a list`
- E) `fun: ('a -> 'b) -> 'a list -> 'b list`

Risposta esatta: E - `fun: ('a -> 'b) -> 'a list -> 'b list`

Spiegazione: La funzione della libreria standard `List.map` ha esattamente la firma polimorfa `('a -> 'b) -> 'a list -> 'b list`. Creando una funzione `f3` che prende `x` e lo passa come primo argomento a `List.map`, `x` deve necessariamente essere la funzione di trasformazione (`'a -> 'b`). `f3` restituisce l'applicazione parziale di `map`, che aspetta una `'a list` e restituisce una `'b list`. Di fatto, `f3` è l'esatto equivalente (*eta-expansion*) di `List.map`.

- **Domanda 103**

Testo: Qual è il tipo della seguente espressione: `fn x => fn y => x + y; ?`

- A) `fn: int -> int`
- B) `fn: 'a -> 'a -> 'a`
- C) `fn: int -> int -> int`
- D) `fn: 'a -> 'a`
- E) `fn: int -> int -> 'a`
- F) Nessuna delle altre risposte

Risposta esatta: C - `fn: int -> int -> int`

Spiegazione: In Standard ML, se non viene specificato diversamente in maniera esplicita, l'operatore matematico di somma `+` è sovraccaricato in modo da avere come tipo di default `int * int -> int`. Il currying della notazione anonima prende l'intero `x`, restituisce una funzione che attende l'intero `y`, e infine restituisce la somma intera.

- **Domanda 104**

Testo: Qual è il tipo della seguente espressione: `fun funzione x y z = x ^ y ^ z` ?

- A) `fn: string -> string`
- B) `fn: 'a -> 'a`
- C) `fn: 'a -> 'a -> 'a -> 'a`
- D) `fn: string -> string -> string -> int`
- E) Nessuna delle altre risposte
- F) `fn: string -> string -> string -> string`

Risposta esatta: F - `fn: string -> string -> string -> string`

Spiegazione: L'operatore `^` (caret) in ML è dedicato esclusivamente alla concatenazione di stringhe. Ciò forza il motore di inferenza di ML a imporre il tipo `string` ai parametri `x`, `y` e `z`. Poiché la concatenazione di tre stringhe genera a sua volta una nuova stringa, il tipo in uscita è l'ennesimo `string`. (Nota: nel PDF c'è un evidente errore di stampa dove l'operatore `^` viene trasposto come simbolo matematico $\wedge$).

- **Domanda 105**

Testo: Qual è il tipo della seguente espressione: `fun c a1 a2 = a1::a2`; ?

- A) `fn: 'a list -> 'a list -> 'a list`
- B) `fn: 'a -> 'a list -> 'a list`
- C) `fn: 'a -> 'a -> 'a`
- D) `fn: 'a -> 'a list`
- E) `fn: 'a -> 'a -> 'a list`

Risposta esatta: B - `fn: 'a -> 'a list -> 'a list`

Spiegazione: Il costruttore di lista `::` (cons) inserisce un singolo elemento in testa a una lista esistente. Per definizione sintattica, l'operando di sinistra (`a1`) deve essere un singolo elemento di tipo generico `'a`, mentre l'operando di destra (`a2`) deve essere una lista formata da elementi dello stesso tipo (`'a list`). Il risultato dell'operazione è l'intera nuova lista concatenata, da qui il tipo finale di `'a list`.

- **Domanda 106**

Testo: Qual è il tipo della seguente espressione: `fn l1 => fn l2 => l1 @ l2`; ?

- A) `fn: 'a list -> 'a list -> 'a list`
- B) `fn: 'a -> 'a list -> 'a list`
- C) `fn: 'a -> 'a -> 'a`
- D) `fn: 'a -> 'a list`
- E) `fn: 'a -> 'a -> 'a list`

Risposta esatta: A - `fn: 'a list -> 'a list -> 'a list`

Spiegazione: A differenza dell'operatore `::` (cons), l'operatore `@` unisce (effettua un merge/append) due liste complete. Pertanto, il motore di inferenza pretenderà che sia `l1` che `l2` siano entrambi istanze di `'a list`. (Nota: nel file originale c'è un errore OCR con i numeri `l1` e `l2`, la notazione corretta usa `l1` ed `l2`).

• **Domanda 107**

Testo: Qual è il tipo della seguente espressione: `fn a => fn b => fn c => (a * b) / c; ?` (nota: corretto in base all'esito fornito tra le opzioni del PDF, originariamente apparso come `a b / c`)

- A) `fn: int -> int -> int -> int`
- B) `fn: 'a -> 'a -> 'a -> 'a`
- C) `fn: int -> int -> int -> real`
- D) `fn: real -> real -> real -> real`
- E) `fn: int -> int -> real -> real`

Risposta esatta: D - `fn: real -> real -> real -> real`

Spiegazione: In SML l'operatore di divisione `/` è riservato strettamente e in modo esclusivo ai numeri in virgola mobile (i `real`). La divisione tra interi usa l'operatore `div`. Questo singolo vincolo forza immediatamente a cascata l'inferenza di tutti i tipi coinvolti: `c` deve essere `real` per poter fungere da divisore, `a * b` deve risultare `real` (costringendo `a` e `b` a essere `real`), e il risultato finale sarà un `real`.

• **Domanda 108**

Testo: Qual è il tipo della seguente espressione: `fun f [] = (7.0; print("ab")) | f (x::xs) = (2.3; print("cd"));` ?

- A) `val f = fn: 'a list -> unit`
- B) `val f = fn: 'a list -> real`
- C) `'a list -> int`
- D) `'a tuple`
- E) `real list -> string`

Risposta esatta: A - `val f = fn: 'a list -> unit`

Spiegazione: Esaminando il costrutto, capiamo dai pattern `[]` e `x::xs` che la funzione accetta una lista. Poiché non impone vincoli su cosa contenga, è un parametro `'a list`. Analizzando i corpi separati da `;` (valutazione sequenziale), l'interprete valuta la prima espressione (es. `7.0`), la scarta e valuta l'ultima, restituendone il risultato. L'ultima espressione è sempre `print(...)` che in ML ha effetti collaterali sull'output di sistema, ma restituisce sempre un token vuoto di tipo `unit`.

• **Domanda 109**

Testo: Qual è l'output del seguente codice: `List.partition (fn x => x>0) (List.take ([1,1,2,2,3,3],4)); ?`

- A) `val it=([1,3], [1, 2, 2, 3]): int list * int list`
- B) `val it=[1,3]: int list`
- C) `val it=[1]: int list`
- D) `val it=([1],[1,2,2]): int list * int list`
- E) `val it=([],[2]); int list * int list`
- F) `val it=[] int list`

Risposta esatta: D - `val it=([1],[1,2,2]): int list * int list`

Spiegazione: La prima operazione ad essere eseguita, trovandosi tra le parentesi più interne, è `List.take(..., 4)`. Questa preleva unicamente i primi 4 elementi della lista fornita, accorciandola a `[1, 1, 2, 2]`. Successivamente, `List.partition` con il predicato `(x > 0)` divide questa nuova lista in una tupla di due liste: la prima contiene gli elementi che soddisfano il requisito (cioè il solo 1), la seconda contiene tutti gli altri (i numeri negativi).

• **Domanda 110**

Testo: Qual è il tipo della seguente espressione: `fn a => fn b => a=b; ?`

- A) `fn: "a -> "a -> bool`
- B) `fn: int -> int -> bool`
- C) `fn: string -> string -> bool`
- D) `fn: bool -> bool -> bool`
- E) `fn: 'a -> 'a -> bool`

Risposta esatta: A - `fn: "a -> "a -> bool`

Spiegazione: L'operatore `=` verifica l'uguaglianza tra due elementi che, per logica, restituiscono un valore `bool`. In SML, tuttavia, l'uguaglianza non è definita per tutti i tipi (ad esempio, è illegale confrontare due `real` per problemi di precisione binaria o due funzioni). Si utilizza allora un tipo speciale di variabile di tipo, definita "Equality Type" e denotata dal doppio apice `"a`, che indica "qualsiasi tipo a patto che sia supportato il controllo di uguaglianza".

- **Domanda 111**

Testo: Qual è il tipo della seguente espressione: `fun f [] = (print("ab"); 7.0) | f(x::xs) = (print("cd"); 4.0); ?`

- A) `'a list -> int`
- B) `'a -> real`
- C) `'a record`
- D) `'a tuple`
- E) `'a list -> real`

Risposta esatta: E - `'a list -> real`

Spiegazione: Esattamente come per la Domanda 13 appena vista, l'ingresso è implicitamente marcato come una `'a list`. Questa volta, all'interno del blocco sequenziale, le posizioni sono invertite: il `print` viene eseguito e il suo `unit` scartato in favore del secondo termine. In entrambi i rami viene ritornato un numero decimale (rispettivamente 7.0 e 4.0), fissando il tipo di uscita a `real`.

- **Domanda 112**

Testo: In ML `:>` serve a:

- A) Nessuna delle altre risposte
- B) Nasconde i componenti di una lista
- C) È usata per definire funzioni polimorfiche
- D) Nasconde i componenti di una struttura
- E) Definisce la firma di una struttura

Risposta esatta: D - Nasconde i componenti di una struttura

Spiegazione: In SML, l'operatore `:>` (Ascription Opaca o Opaque Signature Matching) viene utilizzato in fase di dichiarazione di una struttura per associarvi una firma (signature). A differenza dell'ascription trasparente (`:`), esso "nasconde" (rende astratti) all'esterno tutti i dettagli implementativi e i tipi concreti definiti nella struttura che non sono esplicitamente descritti nella firma, garantendo così l'Information Hiding e l'implementazione degli Abstract Data Types (ADT).

- **Domanda 113**

Testo: Dato il seguente programma in ML, indicare il risultato:

```
Datatype fruit = Apple | Pear | Grape; fun isApple x = (x=Apple); isApple (Lemon);
```

- A) Nessuna delle altre risposte
- B) False
- C) Undeclared value
- D) True
- E) Type error

Risposta esatta: C - Undeclared value

Spiegazione: L'errore scaturisce a tempo di compilazione. Il costruttore di tipo `Lemon` non è stato dichiarato da nessuna parte né nel `datatype fruit`, né come variabile. Di conseguenza, l'interprete si blocca prima ancora di poter effettuare il controllo di uguaglianza (che restituirebbe `False`) o l'inferenza di tipo, segnalando che il valore non è definito (`Undeclared value/Unbound constructor`). (Nota: nel PDF il testo omette `i =` e `i |` per refusi di trascrizione, ma la semantica è quella indicata).

- **Domanda 114**

Testo: Qual è l'output del seguente codice: `val i = ref 1; while !i<10 do i := !i+1; ?`

- A) `val i=10`
- B) `val it=()`
- C) Loop infinito
- D) Type error
- E) Nessuna delle altre risposte

Risposta esatta: B - `val it=()`

Spiegazione: In SML, il costrutto `while ... do ...` viene utilizzato per scopi prettamente imperativi (side effects) e aggiornamento di riferimenti mutabili. Al termine della sua esecuzione (quando la condizione diventa falsa), il ciclo non restituisce alcun valore utile dal punto di vista funzionale, ma valuta sempre all'entità vuota `unit`, che la shell di SML stampa a video come `val it = () : unit.`

- **Domanda 115**

Testo: Qual è l'output del seguente codice:

```
val m = (List.map(fn x=>x*x) [4, 5, 6]); List.foldr Int.+ 0 m;
```

- A) 15
- B) 77
- C) 'a list -> int
- D) NaN
- E) Nessuna delle altre risposte

Risposta esatta: B - 77

Spiegazione: Seguiamo il codice passo passo. Prima eseguiamo `List.map` con la funzione di elevamento al quadrato (`x*x`) sulla lista `[4, 5, 6]`. Il risultato è la nuova lista `m = [16, 25, 36]`. Poi usiamo `List.foldr` passando la funzione di somma intera (`Int.+`), con elemento neutro 0, su `m`. Questo esegue l'accumulazione `16 + (25 + (36 + 0))`, che ammonta esattamente a 77.

- **Domanda 116**

Testo: In ML `4+3.0` genera come risultato:

- A) Errore di tipo
- B) 7.0
- C) Nessuna delle altre risposte
- D) Dipende dall'implementazione
- E) 7

Risposta esatta: A - Errore di tipo

Spiegazione: SML è un linguaggio fortemente tipizzato che non ammette coercion (conversioni di tipo implicite) tra tipi numerici. L'intero 4 e il floating point 3.0 non possono essere addizionati direttamente con lo stesso operatore senza prima essere forzati esplicitamente a far combaciare i tipi (es. usando `Real.fromInt(4) + 3.0`). L'espressione, così come fornita, causa un type mismatch (Errore di tipo).

- **Domanda 117**

Testo: Si assuma che `A` sia un array di lunghezza 40. Trovare il 30esimo elemento.

- A) `Array.sub(A,30)`
- B) `Array.sub(A,29)`
- C) Nessuna delle altre risposte
- D) `sub(A,29)`
- E) `A[30]`

Risposta esatta: B - `Array.sub(A,29)`

Spiegazione: In SML, la struttura standard per operare sugli array mutabili è `Array`. L'accesso in lettura si effettua obbligatoriamente chiamando la funzione `Array.sub(array, indice)` (rendendo errata l'opzione E e D). Siccome gli array

sono zero-based (il primo elemento è in posizione 0), il 30esimo elemento logico si troverà all'indice 29.

- **Domanda 118**

Testo: Indicare il risultato di `Struct2.x = Struct2.i`; Dato il seguente programma ML:

```
structure Struct1 = struct val i = 3; type t = int; val x = 4 end; signature
SIG1 = sig val i:int; type t; val x: t; end; structure Struct2:> SIG1
= Struct1;
```

- A) Nessuna delle altre risposte
- B) 3
- C) False
- D) Type error
- E) True

Risposta esatta: D - Type error

Spiegazione: Questo è un classico esempio di Information Hiding tramite l'operatore `>` (Ascription Opaca). All'interno di `Struct1`, il tipo `t` è esplicitamente sinonimo di `int`. Tuttavia, sigillando `Struct1` nella `Struct2` tramite `SIG1`, esponiamo `t` come tipo "astratto" (`type t;`). Quando si tenta di valutare l'uguaglianza all'esterno, il compilatore vede `Struct2.x` come avente il tipo opaco incommensurabile `t`, e `Struct2.i` come `int`. Non sapendo più che `t` maschera un intero, fallisce con un Errore di Tipo.

- **Domanda 119**

Testo: In ML, quale espressione è legale:

- A) `[1.0, 2, 3]`
- B) `fn x:real => x*2`
- C) `"a", 1, 2`
- D) `(1, 2, 0)`
- E) Nessuna delle altre risposte

Risposta esatta: D - `(1, 2, 0)`

Spiegazione: L'opzione A è illegale perché le liste in ML non possono mescolare tipi `real` e `int`. L'opzione B è illegale perché un operando annotato come `real` non può essere moltiplicato per 2 (che è `int`, servirebbe `2.0`). L'opzione C manca delle parentesi tonde necessarie per formare una tupla. L'opzione D, `(1, 2, 0)`, è invece una tupla sintatticamente perfetta, formata da tre interi di tipo `int * int * int`.

- **Domanda 120**

Testo: Qual è il tipo di `SOME[2, 3, 4]`;

- A) `'a list option`
- B) `int list option`
- C) `(int*int*int) option`
- D) Nessuna delle altre risposte
- E) `int list`

Risposta esatta: B - `int list option`

Spiegazione: Analizzando dall'interno verso l'esterno, `[2, 3, 4]` è banalmente una lista omogenea di interi, quindi il suo tipo è `int list`. `SOME` è un costruttore polimorfo del datatype `option`, avente firma `'a -> 'a option`. Sostituendo al parametro generico `'a` il nostro `int list`, otteniamo che l'espressione generata ha tipo finale `int list option`.

25 Domande Output Codici

- **Domanda 121**

Testo: Utilizzando la tecnica dei tombstones cosa succede alla fine del frammento di codice dato?

```
p = malloc();
q = malloc();
*p = 999;
*q = 111;
*p += *q;
q = p;
free(p);
```

- A) `*p=1110` e `*q=111`
- B) `p` è deallocato e `q` punta ad una tombstone
- C) `p` e `q` sono entrambi deallocati
- D) `q` è deallocato e `p` punta ad una tombstone
- E) `*p=999` e `*q=111`
- F) `*p=1110` e `*q=1110`
- G) Nessuna delle altre risposte

Risposta esatta: G - Nessuna delle altre risposte

Spiegazione: Analizziamo il codice. Vengono allocate due zone di memoria con i rispettivi tombstones (T1 per `p` e T2 per `q`). I valori diventano `*p = 999` e `*q = 111`. Con `*p += *q`, il valore puntato da `p` diventa 1110. Eseguendo `q = p`, il puntatore `q` smette di puntare a T2 e inizia a puntare al tombstone T1 (generando un memory leak su T2). Eseguendo infine `free(p)`, la memoria di T1 viene liberata e il tombstone T1 viene invalidato (marcato come "morto"). I puntatori locali `p` e `q` non vengono "deallocati", ma continuano a esistere sullo stack ed **entrambi puntano allo stesso tombstone invalidato** (T1). Poiché nessuna delle opzioni A-F descrive correttamente questa situazione, la risposta corretta (che è stata aggiunta nelle versioni successive del quiz, vedi domanda seguente) è "Nessuna".

- **Domanda 122**

Testo: Utilizzando la tecnica dei tombstones cosa succede alla fine del frammento di codice dato?

```
p = malloc();
q = malloc();
*p = 999;
*q = 111;
*p += *q;
q = p;
free(p);
```

- A) `*p=1110` e `*q=111`
- B) `*p=1110` e `*q=1110`
- C) `p` e `q` sono entrambi deallocati
- D) `q` è deallocato e `p` punta ad una tombstone
- E) Nessuna delle risposte

Risposta esatta: E - Nessuna delle risposte

Spiegazione: Identica alla domanda precedente, ma con le opzioni corrette. Come dimostrato, alla fine dell'esecuzione del codice **entrambi** i puntatori `p` e `q` puntano allo stesso tombstone ormai invalidato. Le variabili puntatore in sé non sono deallocate (rimangono sullo stack), ma il loro riferimento è "pendente" seppur protetto dalla tombstone. L'opzione esatta è dunque "Nessuna delle risposte".

- **Domanda 123**

Testo: Utilizzando la tecnica dei tombstones cosa succede alla fine del frammento di codice dato?

```
p = malloc();
q = malloc();
*p = 123;
*q = 321;
p = q;
free(q);
```

- A) `*p=999` e `*q=111`
- B) `*p=123` e `*q=321`
- C) `p` è deallocato e `q` punta ad una tombstone
- D) `p` e `q` sono entrambi deallocati
- E) `q` è deallocato e `p` punta ad una tombstone
- F) `*p=321` e `*q=321`
- G) Nessuna delle risposte

Risposta esatta: G - Nessuna delle risposte

Spiegazione: Assegnando `p = q`, si fa in modo che `p` punti allo stesso tombstone di `q`. Subito dopo, la `free(q)` dealloca la memoria a cui quel tombstone puntava, invalidandolo. Di conseguenza, alla fine dell'esecuzione, sia `p` che `q` si ritroveranno a puntare contemporaneamente a una tombstone marcata come distrutta (dangling pointer). Dato che nessuna delle frasi proposte descrive questo stato ("entrambi puntano ad una tombstone"), la risposta è Nessuna.

• **Domanda 124**

Testo: Si consideri lo pseudo-codice. Qual è il valore di ritorno di `pluto()` se i parametri sono passati **per nome**?

```
int c = 2;
int pippo (int a) {
    c = c + 2;
    return a * 2;
}
int pluto (void) {
    return pippo(c + 1);
}
```

- A) Nessuna delle altre risposte
- B) 10
- C) Dipende dal tipo di scope (statico o dinamico) utilizzato
- D) 4
- E) 6

Risposta esatta: B - 10

Spiegazione: Nel passaggio per nome (Call by Name), l'argomento `(c + 1)` non viene valutato al momento della chiamata, ma viene testualmente sostituito all'interno della funzione `pippo` (tramite un thunk invisibile). All'interno di `pippo`, viene prima eseguito `c = c + 2`, portando la variabile globale `c` da 2 a 4. Successivamente, si esegue il `return a * 2`, che per nome diventa `(c + 1) * 2`. Poiché `c` ora vale 4, il calcolo sarà $(4 + 1) * 2 = 10$.

• **Domanda 125**

Testo: Si consideri lo pseudo-codice. Qual è il valore di ritorno di `pluto()` se i parametri sono passati **per valore**?

```

int c = 2;
int pippo (int a) {
    c = c + 2;
    return a * 2;
}
int pluto (void) {
    return pippo(c + 1);
}

```

- A) 10
- B) Dipende dal tipo di scope (statico o dinamico) utilizzato
- C) Nessuna delle altre risposte
- D) 6
- E) Non è possibile passare c+1 per valore

Risposta esatta: D - 6

Spiegazione: Nel passaggio per valore (Call by Value), l'argomento viene valutato *prima* di entrare nella funzione. Poiché inizialmente **c=2**, l'espressione **c+1** viene valutata a 3, e questo valore fisso viene copiato nel parametro locale **a**. All'interno di **pippo**, la variabile globale **c** viene aggiornata a 4 (**c = 2 + 2**), ma questo non intacca minimamente la variabile locale **a**, che è rimasta 3. Il calcolo finale sarà **a * 2 = 3 * 2 = 6**.

- **Domanda 126**

Testo: Si consideri lo pseudo-codice. Qual è il valore di ritorno di `f(1, r(1), 1)` se i parametri sono passati **per nome**?

```
int r(int x) {  
    return r(x-1);  
}  
int f(int a, int b, int c) {  
    if (c == 1) return a;  
    else return b;  
}
```

- A) Non è possibile dirlo senza conoscere il tipo di scope (statico o dinamico) utilizzato
- B) Nessuna delle altre risposte
- C) 1
- D) Si ha ricorsione infinita
- E) 0

Risposta esatta: C - 1

Spiegazione: Se il passaggio fosse per valore, l'argomento `r(1)` verrebbe valutato prima della chiamata a `f`, causando una ricorsione infinita. Ma trovandoci in Call by Name, gli argomenti vengono passati chiusi in un thunk e valutati *solo e soltanto* se il loro valore viene esplicitamente richiesto nel codice. Nel corpo di `f`, `c` vale 1, quindi l'espressione `(c == 1)` è vera. La funzione restituisce immediatamente `a` (che vale 1). Il parametro `b (r(1))` non viene mai menzionato né valutato, scongiurando il loop infinito.

- **Domanda 127**

Testo: Dato il frammento di programma (espresso in pseudo-codice), quanto vale la variabile globale `c` dopo aver eseguito `topolino()`, assumendo **scope dinamico**?

```
int a, b, c;  
void pippo (void) { int a; a = 6; b = 5; }  
void pluto (void) { int c; int b; pippo(); c = 3; a = 4; }  
void topolino (void) { int a; a = 1; b = 10; pluto(); c = a + b; }
```

- A) 14
- B) 3
- C) Non è possibile dirlo
- D) 6
- E) Nessuna delle altre risposte

Risposta esatta: A - 14

Spiegazione: Tracciamo la catena dinamica: `topolino` chiama `pluto` che chiama `pippo`. In `pippo`: la `b=5` cerca una `b` locale, non la trova e risale alla funzione

chiamante (**pluto**). Poiché **pluto** ha una **b** locale, aggiorna quella a 5 (la **b** globale resta 10). In **pluto**: imposta la sua **c** locale a 3, poi **a=4** cerca una **a** risalendo la catena fino a **topolino**, aggiornando la **a** locale di quest'ultima a 4. In **topolino**: esegue $c = a + b$. Cerca **c**, non ne ha, risale al main e aggiorna la **c** globale. A quale valore? **a** è la sua **a** locale (appena aggiornata a 4). **b** non c'è in **topolino**, quindi risale alla **b** globale (che era stata posta a 10 all'inizio di **topolino**). Dunque $c = 4 + 10 = 14$.

- **Domanda 128**

Testo: Dato il frammento di programma (espresso in pseudo-codice), qual è il valore di ritorno di **f1()**, assumendo **scope statico**?

```
int x, y, z;
void f3 (void) { x = 0; y = 5; }
void f2 (void) { int y; f3(); y = 0; z = 10; }
int f1 (void) {
    int x;
    x = -5; y = 10; z = x + y;
    f2();
    return z - y - x;
}
```

- A) 5
- B) Nessuna delle altre risposte
- C) Non è possibile dirlo
- D) 0
- E) -5

Risposta esatta: B - Nessuna delle altre risposte

Spiegazione: Esecuzione statica (lessicale, ogni blocco guarda le variabili globali se non le ha localmente). In **f1**: **x** locale diventa -5. **y** globale diventa 10. **z** globale diventa $-5 + 10 = 5$. Viene chiamata **f2**. In **f2**: viene chiamata **f3**, che aggiorna **x** globale a 0 e **y** globale a 5. Poi **f2** setta la sua **y** locale a 0 e aggiorna la **z** globale a 10. Tornando a **f1** per il **return** $z - y - x$: **z** (globale) è 10, **y** (globale) è 5 (aggiornata da **f3**), **x** (locale di **f1**) è -5. Il calcolo è $10 - 5 - (-5) = 10$. Poiché 10 non è tra le opzioni (5, 0, -5), la risposta corretta è Nessuna.

- **Domanda 129**

Testo: Si consideri lo pseudo-codice. Qual è il valore di ritorno di `pluto()` se i parametri sono passati **per valore**?

```
int b = 666;
int pippo (int x) {
    x = 666;
    b = 1;
    return x / 2;
}
int pluto (void) {
    int a, c;
    a = b / 333;
    c = pippo(a);
    return a + c;
}
```

- A) Nessuna delle altre risposte
- B) 1
- C) 3
- D) 0
- E) Dipende dal tipo di scope (statico o dinamico) utilizzato

Risposta esatta: A - Nessuna delle altre risposte

Spiegazione: Analizziamo `pluto`: $a = 666 / 333$, quindi $a = 2$. Chiamando `pippo(2)` per valore, una copia del 2 viene inserita in `x`. All'interno di `pippo`: la `x` locale viene sovrascritta con 666, e la `b` globale diventa 1. La funzione ritorna $666 / 2 = 333$, valore che viene salvato nella variabile `c` in `pluto`. Alla fine, `pluto` calcola il ritorno sommando $a + c$. Poiché `a` è rimasto inalterato a 2, il ritorno sarà $2 + 333 = 335$. Questo valore non è presente, per cui l'opzione giusta è "Nessuna".

- **Domanda 130**

Testo: Funzione implementata dallo pseudo-codice:

```
int somma (int a, int b) {  
    if (a == 0) return b;  
    return somma (a-1, b+1);  
}
```

- A) Non usa ricorsione in coda
- B) Non può essere implementata per via iterativa
- C) Nessuna delle altre risposte
- D) Usa ricorsione in coda
- E) Causa sempre ricorsione infinita

Risposta esatta: D - Usa ricorsione in coda

Spiegazione: La funzione si chiama iterativamente decrementando **a** e incrementando **b**. La caratteristica fondamentale è che l'espressione `return somma(a-1, b+1);` è l'ultimissima istruzione eseguita nel blocco. Non ci sono calcoli "postumi" (es. moltiplicazioni o somme esterne alla chiamata ricorsiva) che il record di attivazione debba attendere. Per questo motivo, questa forma ottimizzata prende il nome di Ricorsione in Coda (Tail Recursion).

- **Domanda 131**

Testo: Si consideri lo pseudo-codice. Qual è il valore di ritorno di `f3(10,f4(0))` se i parametri sono passati **per valore**?

```
int f4 (int x) {  
    return 1/x;  
}  
int f3 (int a, int b) {  
    if (a > 1) return a;  
    else return b;  
}
```

- A) 10
- B) Non è possibile dirlo senza conoscere il tipo di scope (statico o dinamico) utilizzato
- C) Non è possibile passare `f4(0)` per valore
- D) Nessuna delle altre risposte
- E) 1

Risposta esatta: D - Nessuna delle altre risposte

Spiegazione: Nel passaggio dei parametri per valore (Call by Value), gli argomenti della funzione vengono sempre valutati compiutamente *prima* che la funzione chiamante venga effettivamente invocata. Pertanto, l'interprete cercherà di valutare `f4(0)` ancor prima di entrare in `f3`. L'esecuzione di `f4(0)` comporta l'operazione

matematica $1 / 0$, che genera una divisione per zero (eccezione hardware/runtime crash). Il programma si interromperà, quindi non ritornerà né 10, né 1. Non è vero che "non è possibile passarlo" (la sintassi è corretta, ma fallisce semanticamente l'esecuzione). L'unica risposta logica è "Nessuna".

- **Domanda 132**

Testo: Si consideri lo pseudo-codice. Qual è il valore di ritorno di `f1()` se i parametri sono passati **per valore**?

```
int i=0;
int x[10];
int f2 (int z) {
    i = i/2;
    return z - i;
}
int f1 (void) {
    int y;
    i = 4;
    x[0]=1; x[1]=0; x[2]=2; x[3]=4; x[4]=6;
    y = f2(x[i]);
    return y + i;
}
```

- A) Nessuna delle altre risposte
- B) 0
- C) 2
- D) Dipende dal tipo di scope (statico o dinamico) utilizzato
- E) 4

Risposta esatta: A - Nessuna delle altre risposte

Spiegazione: Analizziamo `f1()`: la variabile globale `i` viene settata a 4. L'istruzione `y = f2(x[i])` nel call-by-value richiede la valutazione anticipata dell'argomento. Essendo `i=4`, calcoliamo `x[4]` che corrisponde al valore 6. Entriamo in `f2(6)`: la `i` globale viene dimezzata ($i = 4 / 2 = 2$), e viene ritornato `z - i`, cioè $6 - 2 = 4$. Assegniamo dunque 4 a `y` in `f1`. Alla riga successiva, viene valutato il `return y + i`. Dato che `y=4` e la `i` globale è ora 2, la somma è $4 + 2 = 6$. Poiché 6 non è presente fra le opzioni fornite, la risposta esatta è "Nessuna".

- **Domanda 133**

Testo: Dato il frammento di programma, qual è il valore di ritorno di `f1()`, assumendo **scope dinamico**?

```
int x, y, z;
void f3 (void) { x = 0; y = 5; }
void f2 (void) { int y; f3(); y = 0; z = 10; }
int f1 (void) {
    int x;
    x = -5; y = 10; z = x + y;
    f2();
    return z - y - x;
}
```

- A) Non è possibile dirlo
- B) 5
- C) 0
- D) Nessuna delle altre risposte
- E) -5

Risposta esatta: C - 0

Spiegazione: Con lo *scope dinamico*, l'interprete risale lo Stack delle chiamate (`f3` -> `f2` -> `f1` -> `main`) per trovare le variabili. 1. In `f1`: `x` locale vale -5. `y` globale vale 10. `z` globale vale 5 (-5 + 10). 2. Chiamata a `f2`: viene allocata una `y` locale. 3. Chiamata a `f3`: assegna 0 a `x`. Risalendo la catena, il primo `x` si trova in `f1`, quindi **la `x` locale di `f1` diventa 0**. Assegna 5 a `y`. Risalendo, il primo `y` è quello locale a `f2`, quindi **la `y` locale di `f2` diventa 5**. 4. Ritorno in `f2`: l'istruzione `y = 0` sovrascrive la `y` locale di `f2` a 0. L'istruzione `z = 10` trova solo la `z` globale e la setta a 10. 5. Ritorno in `f1`: esegue `z - y - x`. La variabile `z` trovata è la globale (10). La variabile `y` trovata è la globale (rimasta a 10, intoccata). La variabile `x` trovata è la locale di `f1` (modificata a 0 da `f3`). Calcolo finale: $10 - 10 - 0 = 0$.

- **Domanda 134**

Testo: Si consideri lo pseudo-codice. Qual è il valore di ritorno di `pluto()` se i parametri sono passati **per valore**?

```
int b = 666;
int pippo (int x) {
    x = 666;
    b = 1;
    return x / 2;
}
int pluto (void) {
    int a, c;
    a = b / 333;
    c = pippo(a);
    return a + c;
}
```

- A) Nessuna delle altre risposte
- B) 666
- C) 999
- D) Dipende dal tipo di scope (statico o dinamico) utilizzato
- E) 333

Risposta esatta: A - Nessuna delle altre risposte

Spiegazione: Questo è lo stesso pseudo-codice analizzato in precedenza. In `pluto`, calcoliamo $a = 666 / 333 = 2$. `pippo(2)` riceve una copia del 2 in `x`, che viene subito sovrascritta con 666. La funzione ritorna $666 / 2 = 333$ (salvato in `c`) e muta `b` in 1. Poiché il passaggio è per valore, la variabile `a` in `pluto` resta inalterata (vale 2). Il ritorno finale è $a + c = 2 + 333 = 335$. Essendo assente il 335, la risposta corretta è Nessuna.

- **Domanda 135**

Testo: Funzione implementata dallo pseudo-codice:

```
int somma (int a, int b) {
    if (a == 0) return b;
    return somma (a-1, b) + 1;
}
```

- A) Nessuna delle altre risposte
- B) Non può essere implementata per via iterativa
- C) Non usa ricorsione in coda
- D) Causa sempre ricorsione infinita

Risposta esatta: C - Non usa ricorsione in coda

Spiegazione: Affinché una funzione utilizzi la Ricorsione in Coda (Tail Recursion),

la chiamata a se stessa deve essere rigorosamente l'ultima espressione valutata. In questo pseudo-codice, la chiamata `somma(a-1, b)` viene effettuata, il suo risultato deve essere atteso dal Record di Attivazione corrente e, solo successivamente, vi viene sommato `+ 1`. Questo blocco `(+ 1)` finale "rompe" il pattern di coda, richiedendo l'espansione dello Stack.

- **Domanda 136**

Testo: Si consideri lo pseudo-codice della Domanda 14. Qual è il valore di ritorno di `pluto()` se i parametri sono passati **per nome**?

```
int b = 666;
int pippo (int x) {
    x = 666; b = 1; return x / 2;
}
int pluto (void) {
    int a, c; a = b / 333; c = pippo(a); return a + c;
}
```

- A) Non è possibile passare a per nome
- B) Dipende dal tipo di scope (statico o dinamico) utilizzato
- C) Nessuna delle altre risposte
- D) 999
- E) 335

Risposta esatta: D - 999

Spiegazione: Nel passaggio per nome (Call by Name), passare una variabile semplice (come `a`) genera un thunk che punta a quella specifica locazione di memoria; questo di fatto simula un comportamento di "Passaggio per Riferimento". 1. In `pluto`: calcoliamo `a = 2`. Invochiamo `pippo(a)` passando `a` per nome. 2. In `pippo`: l'istruzione `x = 666` sostituisce testualmente `x` con `a`, alterando la variabile `a` nel chiamante facendola diventare 666. 3. Il ritorno calcola `x / 2` (ovvero `a / 2`), che valuta `666 / 2 = 333`. 4. Tornati in `pluto`: `c = 333`. Si esegue `return a + c`. Dato che la variabile locale `a` è stata mutata in 666 dalla chiamata per nome, il calcolo finale sarà `666 + 333 = 999`.

- **Domanda 137**

Testo: La funzione implementata dallo pseudo-codice:

```
int mistero (int a, int b) {  
    if (b == 0) return a;  
    return mistero (a/2, b-1);  
}
```

- A) Può causare una crescita incontrollata dello stack
- B) Causa sempre ricorsione infinita
- C) Nessuna delle altre risposte
- D) Non può essere implementata per via iterativa
- E) Non usa ricorsione in coda

Risposta esatta: C - Nessuna delle altre risposte

Spiegazione: Analizziamo logicamente il costrutto: la chiamata ricorsiva a `mistero` è l'ultimo blocco logico ad essere eseguito e ritornato (senza operazioni appese come un `+1`), dunque USA esplicitamente la ricorsione in coda (falsa E). Grazie alla ricorsione in coda, il compilatore la ottimizzerà rendendo lo stack costante e prevenendo crescite incontrollate (falsa A). Per induzione, non causa ricorsione infinita a meno che `b` non sia negativo (falsa B), e come ogni algoritmo ricorsivo in coda, può essere banalmente convertito in un ciclo `while` iterativo (falsa D). Dunque, nessuna affermazione è vera.

- **Domanda 138**

Testo: Dato il frammento di programma, qual è il valore di ritorno di `topolino()`, assumendo **scope statico**?

```
int x, y;  
void pippo (void) { x = 8; y = 4; }  
void pluto (void) { int y; pippo(); y = 3; }  
int topolino (void) {  
    int x, z;  
    x = 5; y = 15; z = x + y;  
    pluto();  
    return z - y - x;  
}
```

- A) Nessuna delle altre risposte
- B) Non è possibile dirlo
- C) 0
- D) -3

Risposta esatta: E - 11

Spiegazione: Seguiamo lo Scope Statico (lessicale), dove i nomi non dichiarati localmente si riferiscono sempre alle definizioni globali nel testo. 1. In `topolino`:

setta la x locale a 5. La y si riferisce alla globale (che diventa 15). Calcola z locale: $5 + 15 = 20$. 2. Chiamata a **pluto**: alloca una y locale. 3. Chiamata a **pippo**: setta la x globale a 8, e la y globale a 4 (ignora la catena dinamica). 4. Ritorno a **pluto**: setta la y locale a 3 (ininfluente ai fini logici generali). 5. Ritorno a **topolino**: valuta $z - y - x$. La z locale è rimasta 20. La y globale è stata impostata a 4 da **pippo**. La x locale è sempre stata 5 (intoccata da **pippo** che operava sulla globale). Il risultato è $20 - 4 - 5 = 11$.

- **Domanda 139**

Testo: Dato il frammento di programma (lo stesso della Domanda 13), qual è il valore di ritorno di `f1()`, assumendo **scope dinamico**?

```
int x, y, z;
void f3 (void) { x = 0; y = 5; }
void f2 (void) { int y; f3(); y = 0; z = 10; }
int f1 (void) {
    int x; x = -5; y = 10; z = x + y;
    f2(); return z - y - x;
}
```

- A) Nessuna delle altre risposte
- B) -1
- C) -5
- D) 5
- E) Non è possibile dirlo

Risposta esatta: A - Nessuna delle altre risposte

Spiegazione: Come dimostrato esaurientemente nella Domanda 13, la catena dello scope dinamico per questa esatta espressione risale lo stack delle chiamate, facendo in modo che la x locale in `f1` venga modificata a 0, che la y globale resti intoccata a 10, e che la z globale diventi 10. Il calcolo finale $z - y - x$ produce $10 - 10 - 0 = 0$. Poiché lo 0 non appare tra le opzioni elencate qui (a differenza della Domanda 13 dove compariva), la risposta corretta questa volta è "Nessuna".

- **Domanda 140**

Testo: Si consideri lo pseudo-codice. Qual è il valore di ritorno di `pluto()` se i parametri sono passati **per valore**?

```
int c = 2;
int pippo (int a) {
    c = c + 2;
    return a * 2;
}
int pluto (void) {
    return (pippo (c + 1));
}
```

- A) Nessuna delle altre risposte
- B) Dipende dal tipo di scope (statico o dinamico) utilizzato
- C) Non è possibile passare `c+1` per valore
- D) 6
- E) 10

Risposta esatta: D - 6

Spiegazione: Riproposizione del caso già valutato: il passaggio per valore processa in toto l'argomento al momento della chiamata. `c=2`, quindi `c+1` calcola istantaneamente 3. Viene invocato `pippo(3)`. All'interno la variabile globale incrementa (`c = 2 + 2 = 4`), ma il record locale possiede la copia costante `a=3`. Il return matematico chiude eseguendo `3 * 2 = 6`.

- **Domanda 141**

Testo: Si consideri lo pseudo-codice. Qual è il valore di ritorno di `f(1,r(1),1)` se i parametri sono passati **per valore**?

```
int r(int x) {
    return r(x-1);
}
int f(int a, int b, int c) {
    if (c == 1) return a;
    else return b;
}
```

- A) Nessuna delle altre risposte
- B) Si ha ricorsione infinita
- C) 1
- D) Non è possibile dirlo senza conoscere il tipo di scope (statico o dinamico) utilizzato

Risposta esatta: B - Si ha ricorsione infinita

Spiegazione: A differenza del passaggio per nome visto nella Domanda 6, il passaggio di parametri per valore impone che tutti gli argomenti forniti a una funzione vengano calcolati rigorosamente *prima* di entrare nella funzione stessa. Il compilatore tenta quindi di valutare l'argomento `r(1)`. Tale chiamata innesca `r(0)`, che innesca `r(-1)` e così via all'infinito, poiché la funzione `r` manca del tutto di un caso base (condizione di terminazione). Il programma va in crash per stack overflow prima ancora di poter iniziare l'esecuzione di `f`.

• **Domanda 142**

Testo: Dato il frammento di programma (espresso in pseudo-codice), qual è il valore di ritorno di `topolino()`, assumendo **scope statico**?

```
int x, y, z;
void minni (void) { x = 4; y = 8; }
void paperino (void) { int y; minni(); y = 1; z = 666; }
int topolino (void) {
    int x; x = 5; y = 15; z = x + y;
    paperino();
    return z - y - x;
}
```

- A) -3
- B) 0
- C) 14
- D) Nessuna delle altre risposte
- E) Non è possibile dirlo

Risposta esatta: D - Nessuna delle altre risposte

Spiegazione: Seguiamo il flusso a scope lessicale (statico). 1. `topolino`: setta la propria `x` locale a 5. La `y` globale diventa 15. La `z` globale diventa $5 + 15 = 20$. 2. Chiama `paperino`, che dichiara una sua `y` locale e chiama `minni`. 3. `minni` opera solo su variabili globali: imposta la `x` globale a 4 e la `y` globale a 8. 4. Ritorno in `paperino`: imposta la sua `y` locale a 1 e la `z` globale a 666. 5. Ritorno in `topolino`: deve calcolare $z - y - x$. Le variabili a cui ha accesso sono la `z` globale (ora 666), la `y` globale (divenuta 8 in `minni`) e la sua `x` locale (rimasta 5). Il calcolo è $666 - 8 - 5 = 653$. Non essendoci il 653 tra le opzioni proposte, la risposta è "Nessuna".

- **Domanda 143**

Testo: Si consideri lo pseudo-codice. Qual è il valore di ritorno di `pluto()` se i parametri sono passati **per riferimento**?

```
int c = 2;
int pippo (int a) {
    c = c + 2;
    return a * 2;
}
int pluto (void) {
    return (pippo(c + 1));
}
```

- A) Dipende dal tipo di scope (statico o dinamico) utilizzato
- B) 10
- C) Non è possibile passare `c+1` per riferimento
- D) Nessuna delle altre risposte
- E) 6

Risposta esatta: C - Non è possibile passare `c+1` per riferimento

Spiegazione: Il passaggio per riferimento impone che alla funzione venga inviato l'indirizzo di memoria (l-value) della variabile sorgente, affinché la funzione ne crei un alias. Tuttavia, l'espressione `(c + 1)` è un risultato intermedio calcolato al volo (r-value), e come tale risiede solo momentaneamente nei registri della CPU: non possiede un indirizzo fisico di memoria a cui un puntatore o un riferimento possa agganciarsi. Pertanto l'operazione è illegale in sede di compilazione.

- **Domanda 144**

Testo: Usando la "bounded iteration" (numerically controlled) indicare cosa viene stampato dal seguente codice:

```
z = 1;
for i = 1 to 1 + z by 1 do {
    write(i);
    z++;
}
write(z);
```

- A) 1234... (infinite loop)
- B) 12
- C) 123
- D) Nessuna delle altre risposte

Risposta esatta: C - 123

Spiegazione: La caratteristica principale dell'iterazione determinata (o limitata) è che il limite superiore del ciclo `(1 + z)` viene valutato matematicamente *una e una*

sola volta, istanti prima che il ciclo abbia inizio. Prima di iniziare, *z* vale 1, quindi il limite superiore viene "congelato" al valore fisso 2. - Iterazione 1: *i*=1, stampa "1", *z* diventa 2. - Iterazione 2: *i*=2, stampa "2", *z* diventa 3. Il ciclo termina perché si è raggiunto il limite prefissato di 2 (l'incremento interno di *z* non allunga il ciclo). Alla riga successiva, viene stampata la variabile *z* che è diventata 3, per un output sequenziale di "123".

- **Domanda 145**

Testo: Dato il frammento di programma, quanto vale la variabile globale *c* dopo aver eseguito `topolino()`, assumendo **scope statico**?

```
int a, b, c;
void pippo (void) { int a; a = 6; b = 5; }
void pluto (void) { int c; int b; pippo(); c = 3; a = 4; }
void topolino (void) { int a; a = 1; b = 10; pluto(); c = a + b; }
```

- A) Nessuna delle altre risposte
- B) 6
- C) Non è possibile dirlo
- D) 14
- E) 3

Risposta esatta: B - 6

Spiegazione: Tracciamo rigorosamente lo scope statico. 1. In `topolino`: *a* locale a `topolino` = 1. *b* globale = 10. 2. Chiama `pluto`: `pluto` dichiara *c* e *b* locali. Chiama `pippo`. 3. In `pippo`: la sua *a* locale diventa 6. La *b* usata è quella globale (ora vale 5). 4. Ritorno a `pluto`: la *c* locale diventa 3, la *a* usata è quella globale (ora vale 4). 5. Ritorno a `topolino`: esegue `c = a + b`. La *c* in questione è la globale. La *a* è la locale (intoccata da chiunque e ferma a 1). La *b* è la globale, che `pippo` aveva modificato portandola a 5. Il calcolo finale assegnato alla globale è `1 + 5 = 6`.

- **Domanda 146**

Testo: Dato il frammento di programma (espresso in pseudo-codice) qual è il valore di ritorno di `pluto()` se i parametri sono passati **per riferimento**?

```
int b = 666;
int pippo (int x) {
    x = 666; b = 1; return x / 2;
}
int pluto (void) {
    int a, c; a = b / 333; c = pippo(a); return a + c;
}
```

- A) 333
- B) Nessuna delle altre risposte

- C) Non è possibile passare a per riferimento (perché $a=b/333$)
- D) 666
- E) 999

Risposta esatta: E - 999

Spiegazione: Innanzitutto l'opzione C è logicamente errata: una variabile allocata e assegnata (come `a`) possiede un indirizzo di memoria e può assolutamente essere passata per riferimento, indipendentemente dal modo in cui le abbiamo assegnato il valore. Passaggio: `a` viene calcolata ed è uguale a 2. Viene passato il riferimento (l'indirizzo) di `a` a `pippo`. Dentro `pippo`: l'argomento `x` funge da "alias" di `a`. Scrivendo `x = 666`, viene modificata permanentemente anche la `a` all'interno del chiamante `pluto`. Il ritorno ammonta a $666 / 2 = 333$. Tornati in `pluto`: `c` è 333. Eseguendo `a + c`, usiamo la nuova `a` (666) mutata per riferimento. $666 + 333 = 999$.

- **Domanda 147**

Testo: Supponendo sia impiegata la tecnica delle "tombstone" per ovviare al problema dei dangling references, indicare quale delle seguenti affermazioni è vera alla fine dell'esecuzione del codice:

```
p = malloc();
q = malloc();
*p = 222;
*q = 999;
*q = *p / 2;
q = p;
free(p);
```

- A) `q` punta alla tombstone, `p` punta alla tombstone
- B) Nessuna delle altre risposte
- C) `*p=999`, `q` punta alla tombstone
- D) `p` punta alla tombstone, `*q=111`
- E) `q` risulta deallocato, `*p=222`

Risposta esatta: A - `q` punta alla tombstone, `p` punta alla tombstone

Spiegazione: Il codice definisce due blocchi memoria con rispettivi tombstones. Viene assegnato il valore 222 in `p` e viene calcolato $*q = 222 / 2 = 111$. Con l'istruzione di assegnamento di puntatori `q = p`, il riferimento `q` abbandona la sua vecchia memoria per far capo esattamente allo stesso tombstone di `p`. Eseguendo `free(p)`, la memoria fisica viene invalidata, marcando il tombstone con la "pietra tombale". A causa dell'aliasing precedente, ora entrambi i puntatori locali, che non sono stati distrutti e continuano a vivere sullo stack, puntano al medesimo tombstone ormai "morto" (dangling).

26 Simulazioni d'Esame: Esercizi Pratici in ML

In questa sezione finale raccogliamo i problemi di programmazione pratica in linguaggio Standard ML proposti durante le simulazioni ufficiali e gli appelli passati. In sede d'esame, la valutazione di questi esercizi si basa sulla corretta applicazione del paradigma funzionale: divieto assoluto di usare costrutti imperativi (come i cicli o i riferimenti mutabili), uso elegante del *Pattern Matching*, corretta inferenza dei tipi e gestione efficiente dell'ambiente locale tramite i blocchi `let ... in ... end`.

26.1 Esercizio 1: Estrazione dei percorsi su Alberi (Appello 15/07/2025)

Testo del problema:

Dato il seguente tipo di dato che rappresenta un albero di numeri interi:

```
datatype Tree = Leaf of int | Node of int * Tree * Tree | Empty;
```

Scrivere due funzioni. La prima, `all_paths (Tree -> int list list)`, deve restituire tutti i percorsi dalla radice dell'albero alle foglie (`Empty` non è considerato una foglia). La seconda, `filter_paths ('a list * ('a -> bool) -> 'a list)`, prende in input una lista di path e un predicato generico, e restituisce una lista di liste contenente solo i path che soddisfano tale condizione. *È ammesso l'utilizzo della funzione `map` ma non di altre funzioni di libreria.*

Soluzione (Valutata 30L):

```
datatype Tree = Leaf of int | Node of int * Tree * Tree | Empty;
fun all_paths(t: Tree) =
let
  fun aux (Empty) = []
    | aux (Leaf n) = [[n]]
    | aux (Node (n, l, r)) =
      let
        val left_paths = aux(l)
        val right_paths = aux(r)
      in
        (* Aggiunge il nodo corrente 'n' in testa a
           TUTTI i percorsi calcolati dai figli *)
        map (fn path => n::path) (left_paths @ right_paths)
      end
in
  aux(t)
end;

fun filter_paths ([], f) = []
  | filter_paths (x::xs, f) =
    if (f(x)) then x :: filter_paths(xs, f)
    else filter_paths(xs, f);

(* Esempio di utilizzo (filtro basato sulla somma degli elementi): *)
fun sum_lower_than_30 list = (foldl (op +) 0 list) < 30;
```

Analisi didattica: L'utilizzo del `map` con una funzione anonima (`fn path => n::path`) per concatenare il nodo radice a *tutte* le liste figlie in un colpo solo dimostra un'ottima padronanza delle funzioni di ordine superiore. Il blocco `let` evita di ricalcolare i percorsi più di una volta.

26.2 Esercizio 2: Astrazione del Controllo (Il ciclo FOR)

Testo del problema (Prog 3):

Si consideri il tipo di dato `datatype FOR = For of int * (int -> int)`, i cui valori `For(n, f)` rappresentano funzioni che implementano un ciclo `for` imperativo che applica iterativamente n volte la funzione f all'argomento. Si scriva una funzione `eval` (avente tipo `FOR -> (int -> int)`) che riceve come argomento un valore di tipo `FOR` e ritorna una funzione curried pronta ad applicare tale ciclo. Se f moltiplica per 2, `eval (For(3, f))` ritornerà una funzione che dato un input lo moltiplicherà per 8 (2^3).

Soluzione:

```
datatype FOR = For of int * (int -> int);

(* Caso base: un ciclo che itera 0 volte restituisce la funzione Identità *)
fun eval (For (0, f)) = (fn x => x)
  | eval (For (n, f)) =
    let
      (* Scongela il passo successivo (n-1) *)
      val rest_eval = eval(For(n - 1, f))
    in
      (* Restituisce una closure che compone f con il resto delle
        iterazioni *)
      (fn x => f(rest_eval(x)))
    end;

(* Alternativa sintatticamente perfetta usando la composizione 'o': *)
fun eval2 (For (0, f)) = (fn x => x)
  | eval2 (For (n, f)) = f o eval2(For (n - 1, f));
```

Analisi didattica: Questo esercizio testa la comprensione profonda della *Partial Instantiation* e delle *Closure*. La funzione `eval` non calcola il risultato finale matematico, ma si limita ad "assemblare" la catena di funzioni $f(f(f(x)))$ e la restituisce in attesa che l'utente fornisca l'input x .

26.3 Esercizio 3: Somma Filtrata per Parità (Prog 1)

Testo del problema:

Si scriva una funzione `sommali` (avente tipo `int -> int list -> int`) che riceve un intero n ed una lista di interi L . La funzione somma ad n gli elementi di L che hanno posizione pari (indici 0, 2, 4...). Se la lista contiene meno di 2 elementi, la logica si applica regolarmente. (Es: `sommali 2 [1,2,3,4] = 2 + 1 + 3 = 6`).

Soluzione:

```
fun sommali n [] = n
  | sommali n [x] = n + x
  (* De-strutturiamo la lista afferrando i primi due elementi:
     x (pari) e y (dispari). Sommiamo x al totalizzatore, e scartiamo y. *)
  | sommali n (x::y::xs) = sommali (n + x) xs;
```

Analisi didattica: Raggiunge un'efficienza ottimale sfruttando la **Tail Recursion** (Ricorsione in coda) in modo che l'accumulatore logico venga trascinato nelle chiamate successive, non riempiendo mai il *Call Stack*. L'uso del pattern `x::y::xs` permette di "saltare" le posizioni dispari con una sintassi compatta.

26.4 Esercizio 4: Manipolazione di stringhe (Prefissi e Suffissi)

Testo del problema (Exam Examples):

Scrivere due funzioni `suffixes` e `prefixes` (tipo `string -> string list`) che, data una stringa, restituiscano la lista di tutti i suoi suffissi/prefissi, ignorando la stringa vuota. Esempio: `suffixes "ciao" = ["ciao", "iao", "ao", "o"]`.

Soluzione:

```
(* SUFFISSI: Semplice, poiché possiamo usare l'estrazione substring *)
fun suffixes "" = []
  | suffixes str =
    str :: suffixes (String.extract(str, 1, NONE));

(* PREFISSI: Convertiamo in char list per poter comporre gli elementi *)
fun pref_aux [] = []
  | pref_aux (x::xs) =
    (* Il primo carattere si abbina a tutti i prefissi generati in coda *)
    [x] :: map (fn p => x::p) (pref_aux xs);

fun prefixes "" = []
  | prefixes str =
    (* implode trasforma le char list interne di nuovo in stringhe *)
    map implode (pref_aux(explode str));
```

26.5 Esercizio 5: Addizione Binaria Nativa su Liste

Testo del problema (Exam Examples):

Scrivere la funzione `sum_binary` di tipo `int list * int list -> int list` che prende in input una coppia di liste di interi (composte da 1 e 0) rappresentanti un numero binario, e ne restituisca la somma binaria. Esempio: `sum_binary ([1,0], [1,0]) = [1,0,0]`.

Soluzione:

```
(* Funzione ausiliaria che incapsula la logica del riporto (carry) *)
fun sum_bin_carry ([], [], 0) = []
  | sum_bin_carry ([], [], 1) = [1]

(* Se una delle due liste è più corta, aggiungiamo [0] fittizi *)
| sum_bin_carry (x::xs, [], carry) = sum_bin_carry(x::xs, [0], carry)
| sum_bin_carry ([], y::ys, carry) = sum_bin_carry([0], y::ys, carry)

(* Passo generale per l'elaborazione dei bit *)
| sum_bin_carry (x::xs, y::ys, carry) =
  let
    val sum = x + y + carry
    val new_bit = sum mod 2    (* Se la somma è 2, il bit è 0 *)
    val new_carry = sum div 2 (* Se la somma è 2 o 3, il carry è 1 *)
  in
    new_bit :: sum_bin_carry(xs, ys, new_carry)
  end;

(* Wrapper esterno che inverte le liste (Little-Endian vs Big-Endian) *)
fun sum_binary (L1, L2) =
  rev (sum_bin_carry (rev L1, rev L2, 0));
```

Analisi didattica: In ML, l'operatore `cons (::)` permette di aggiungere elementi solo *in testa* alla lista, operazione logica $O(1)$. Poiché un'addizione richiede l'allineamento a destra (sui bit meno significativi), è obbligatorio ribaltare prima gli operandi con la funzione nativa `rev`, processare i carry in avanti e, infine, ribaltare il risultato. L'uso dei pattern per gestire la discrepanza di lunghezza tra gli operandi è un'altra peculiarità vitale del codice.

26.6 Esercizio 6: Aggregazione su Strutture Ramificate (get_words)

Testo del problema (Exam Examples):

Dato il tipo datatype `ctree = Empty | Leaf of char | Node of char * ctree * ctree`. Scrivere una funzione `get_words` che restituisce una coppia di stringhe (`string * string`). La prima stringa è la concatenazione dei caratteri contenuti nelle foglie, la seconda stringa è la concatenazione di quelli nei nodi interni. (Es: `get_words Node("#a", Leaf #"b", Leaf #"c") = ("bc", "a")`).

Soluzione:

```
datatype ctree = Empty | Leaf of char | Node of char * ctree * ctree;

fun get_words Empty = ("", "")
  | get_words (Leaf c) = (str(c), "")
  | get_words (Node (c, left, right)) =
    let
      (* Estraiamo i risultati pre-calcolati dai sotto-alberi *)
      val (leavesL, nodesL) = get_words(left)
      val (leavesR, nodesR) = get_words(right)
    in
      (* Ricomponiamo la tupla assemblando i rispettivi domini *)
      (leavesL ^ leavesR, str(c) ^ nodesL ^ nodesR)
    end;
```

Analisi didattica: Questa soluzione sfrutta l'ambiente locale `let` in congiunzione con l'operatore nativo di concatenazione di stringhe (`^`) e il convertitore `str()` per operare il casting da carattere a stringa, aggirando costose manipolazioni di liste di caratteri intermedie.